



Technische Universität Berlin
Faculty IV – Electrical Engineering and Computer Science
Research Group Distributed and Operating Systems

Improving LLM Performance via Sparse Autoencoder Feature Steering

Master's Thesis

Author: Jonas Brüggemann
First Examiner (Supervisor): Prof. Dr. habil. Odej Kao
Second Examiner: Prof. Dr. Volker Markl
Advisor: Dr. Thorsten Wittkopp
Submission: 16.08.2026

Abstract

Improving a large language model (LLM) on a task normally means gradient-based training, with labelled data, a training run, and redeployment for every new task. Sparse autoencoders (SAEs) factor a model's hidden states into often interpretable features that an intervention can amplify or suppress at inference. Recent studies report accuracy gains from steering such features, yet the features are usually chosen by interpretation, correlation, or tuning on the questions that score the result. No training-free method yet shows that its gains come from the features it moves.

This thesis contributes that method and asks whether, and under what conditions, it improves accuracy. A five-phase pipeline with no gradient-based training at any stage consumes a frozen LLM, a public SAE, and a labelled multiple-choice benchmark. It screens questions into a model-relative difficulty interval and there discovers features whose activation tracks whether the model answers correctly. Every candidate is validated causally, moved alone on questions its discovery never saw. A configuration search settles which features to combine and how strongly, and the tuned intervention meets untouched holdout questions once, under a paired significance test.

Across 98 experiments on CommonsenseQA and ARC-Challenge, spanning fifteen models of 70 M to 8 B parameters and 52 public SAEs from eight providers, the answer is a conditional yes. Fifteen of the 98 reach significance. Seven pass every further check, gaining 0.48 to 3.70 percentage points, all on two models of 1 B and 2 B parameters. Of the seven, the strongest lifts Llama-3.2-1B-Instruct on CommonsenseQA from 44.7% to 48.4%, and by 15.47 points inside the difficulty interval that yielded its features. The result is an operating envelope, mapping where the method helps, where it fails, and where the evidence cannot yet decide.

Table of Contents

1 Introduction 1
1.1 Research Questions 2
1.2 Contributions 3
1.3 Thesis Structure 4
2 Background 5
2.1 Transformer Decoders and the Residual Stream 5
2.2 Sparse Autoencoders 7
2.3 Activation Steering 10
2.4 Logits, Temperature, and Multiple-Choice Evaluation 11
2.5 Statistical Foundations 12
3 Methodology and Implementation 13
3.1 Feature-Space Intervention 14
3.2 Phase 0: Difficulty Screening 15
3.3 Phase 1: Frequency-Difference Feature Discovery 18
3.4 Phase 2: Causal Feature Validation 21
3.5 Phase 3: Intervention Configuration Search 22
3.6 Phase 4: Holdout Evaluation 24
4 Evaluation 26
4.1 Experimental Setup 26
4.1.1 Benchmarks 26
4.1.2 Models and Sparse Autoencoders 28
4.1.3 Parameters and Compute Budget 30
4.1.4 Measurement Protocol 33
4.2 Results 36
4.2.1 A Reference Experiment 37
4.2.2 Model Scale 41
4.2.3 Layer Depth 44
4.2.4 Intervention Site 46
4.2.5 Cross-Benchmark Transfer 47
4.3 Contribution of the Pipeline Components 49
4.4 When the Measurement Misleads 53
4.4.1 Scrutiny of the Significant Results 53
4.4.2 Power and Numerical Resolution 55
4.4.3 The Composition of the Transfer Gains 56
4.4.4 What Survives Every Check 56
4.5 Synthesis 57
5 Related Work 59
5.1 Fine-Tuning and Model Editing 59

5.1.1 Weight Modification 59
5.1.2 Representation Finetuning 60
5.2 Activation Steering 61
5.2.1 Vector-Space Steering 61
5.2.2 Feature-Space Steering 62
5.2.3 Steering for Task Performance 63
5.3 Positioning 65
6 Conclusion 68
6.1 Summary of Contributions 68
6.2 Limitations and Future Work 69
Bibliography 71
A SAE Inventory 80
B Benchmark Prompts 83
C Experiment Results 83

1 Introduction

By July 2025, under three years after its launch, ChatGPT had grown to 700 million weekly users, roughly one in ten adults worldwide [1]. In the United States, generative artificial intelligence has spread faster overall than either the personal computer or the internet did at the same point after launch [2]. Eloundou et al. estimate that 19% of US workers could see at least half of their work tasks affected by large language models (LLMs) and the software built on them [3].

Large language models have developed into general-purpose systems, a single pre-trained network answering questions, writing code, and following instructions for tasks it was never explicitly trained on [4], [5]. Improving such a model on a chosen task, however, still normally means changing its weights. That is expensive. Adapting GPT-3 [6] across all of its 175 billion parameters consumes 1.2 terabytes of GPU memory during training, and each task served this way keeps its own 350-gigabyte copy of the weights [7].

The further training that adapts a pre-trained model to a task is called fine-tuning. Preference-based pipelines such as reinforcement learning from human feedback and Direct Preference Optimization retrain the model against curated human judgements [8], [9]. Parameter-efficient methods such as Low-Rank Adaptation shrink that update ten-thousandfold on GPT-3 [7]. In each case the behaviour is acquired through gradient-based training, so every new task demands labelled data, a training run, and a new set of weights to ship. Preference training also measurably degrades performance on some tasks it never targeted, a cost Ouyang et al. call an alignment tax [8].

Each of these routes ends by changing the weights the model runs on. The wish is to improve the model as it stands, nothing retrained, nothing redeployed. A second research direction opens that possibility, leaving the weights untouched and editing the model's hidden states while the forward pass runs [10], [11], [12], [13], [14], [15], [16]. Its instrument is activation steering, the addition of a vector to a per-token hidden state at a chosen layer so that the output shifts in an intended direction [17], [18]. The edit is transient, existing only in the forward pass it is injected into, and it costs one vector addition rather than a training run. Such interventions can carry substantial behaviour. Removing a single direction from the hidden states of thirteen open chat models disables their refusal behaviour, and injecting the same direction induces it [19].

The construction of these vectors is blunt. A steering vector is conventionally the difference of averaged activations between two contrasting prompt sets [18]. The result is a single dense vector. It blends every property that separates the two sets, so the concepts it moves cannot be named. Where such vectors are audited, their effects prove highly variable across inputs, and much of the supporting evidence rests on demonstration rather than measurement [20], [21].

To its user, a large language model is a black box. Its answers are visible, but the computation that produces them is not. Mechanistic interpretability is the research field that works to open this box [22]. One of its principal instruments is the sparse autoencoder (SAE), a pre-trained auxiliary network that decomposes a model's hidden states into features, many of them tracking

recognisable properties such as a topic, a language, or an abstract concept like deception [23], [24], [25]. A single feature can act as a handle on behaviour. Templeton et al. demonstrated this at production scale by clamping a feature of Claude 3 Sonnet that tracks the Golden Gate Bridge to ten times its maximum activation [25]. The model began to self-identify as the bridge. Asked what its physical form was, it replied that it was the bridge itself, and it raised the bridge in answer to almost any query [26]. The instrument built to read a model thus also steers it.

Trained to reconstruct a model's hidden states at one layer, an SAE re-expresses each activation as a combination of a few active features drawn from a learned dictionary holding far more directions than the hidden state has dimensions [23], [24]. A learned dictionary is necessary because the model's own neurons are typically polysemantic, responding to mixtures of unrelated properties [27]. Bricken et al., for instance, find a single neuron that responds to academic citations, English dialogue, HTTP requests, and Korean text all at once [23]. Matched SAEs are now released publicly for many open models, often at several layers [28], [29]. Expressed in this feature basis, an intervention can name the concepts it moves and amplify or suppress each of them separately, while requiring no training of its own.

Feature steering has so far been aimed at behaviour, keeping generation on a topic, strengthening refusal of unsafe prompts, and inducing reasoning strategies such as backtracking [30], [31], [32]. The approach fares poorly against alternatives, one systematic comparison placing SAE-based steering behind both prompting and fine-tuning for concept control [33]. Whether steering can instead make a model measurably better at a task was asked only recently. Dense-vector interventions answered first, more than doubling the truthfulness score of an instruction-tuned LLaMA at inference time [34]. Since 2025 the question has reached the SAE feature basis, with gains reported on mathematical-reasoning and knowledge benchmarks [32], [35], [36], [37]. Reporting a gain is not the same as having a method to produce one. Which features to move, and how strongly, is typically settled by interpretation, by correlation, or by trial on the very questions that then measure the improvement. What is missing is a training-free method whose gains are attributable to the features it selects, and a characterisation of where feature steering helps and where it does not.

Closing that gap is the purpose of this thesis. Its object of study is a training-free pipeline that consumes nothing beyond three public artefacts, a frozen LLM, a pre-trained SAE for one of its layers, and a labelled multiple-choice benchmark. From these inputs the pipeline discovers features whose activation separates the model's correct answers from its errors, validates every candidate causally, assembles a sparse intervention vector, and judges that vector once on new questions. A reported improvement must be attributable to the intervention, and every design choice serves that standard.

1.1 Research Questions

One central question asks whether the method improves accuracy at all, and five refining questions then ask what any such improvement depends on.

Central Question. *Can SAE-based feature steering produce measurable and statistically significant accuracy improvements on multiple-choice benchmarks?*

An improvement counts only if it appears on new questions that played no part in constructing the intervention, and only if a paired significance test separates it from sampling noise. Such a test needs the same question answered with the intervention and without it, which a multiple-choice benchmark makes possible.

RQ1 (Model Scale). *Does the effectiveness of feature steering depend on model scale?*

An SAE is trained on one model's activations. Every change in model size is therefore also a change of dictionary. Comparing models within one provider's suite of SAEs is the closest control the public releases allow.

RQ2 (Layer Depth). *Does the position of the intervention layer influence the outcome accuracy?*

That position describes how early or late in the model the layer sits, and the rest of this thesis calls it the layer depth. Edits made early pass through every layer above them, edits made late through almost none.

RQ3 (Intervention Site). *Does the location of the intervention within a layer influence the outcome accuracy?*

Such a location is called a site, a place within a layer where an activation can be read and edited. A layer exposes several sites, and two of them are compared here. The residual-stream output is the per-token state the layer passes forward, whereas the multi-layer perceptron (MLP) write is one contribution to that state, edited before it is added (Section 2.1).

RQ4 (Cross-Benchmark Transfer). *Does an intervention built on one benchmark also help on other benchmarks?*

Every intervention is discovered, validated, and tuned entirely within one benchmark. Scoring it unchanged on benchmarks it never saw shows whether the signal it carries is general or specific to the one that produced it.

RQ5 (Causal Validation). *Does testing each candidate feature by moving it change what the pipeline delivers?*

A feature whose activation accompanies success need not cause it [38]. The test costs computing time, and whether it earns its keep is measured rather than assumed.

1.2 Contributions

Three contributions follow, in the order the chapters develop them. The first is the method, a five-phase training-free pipeline that leads from a frozen LLM, a public SAE, and a labelled benchmark to a single measured intervention (Chapter 3). Every candidate feature is validated by moving it alone, and the questions that build the intervention are never the ones that judge it. No earlier training-free steering method combines these safeguards (Section 5.3).

The second is the study, 98 experiments pairing fifteen models of 70 M to 8 B parameters with 52 public SAEs on two primary and two transfer-only multiple-choice benchmarks (Chapter 4).

Model scale, layer depth, and intervention site vary as controlled factors, as far as the public SAE inventory permits. The study characterises the method on its own terms, so no fine-tuning baseline is run.

The third is the operating envelope, naming where feature steering helps, where it fails, and where it cannot yet be judged (Section 4.5). No prior work provides such a characterisation (Section 5.3).

1.3 Thesis Structure

The five remaining chapters run from foundations to conclusions. Chapter 2 assembles the technical background, the Transformer's residual stream, sparse autoencoders, activation steering, multiple-choice scoring, and the statistical tests the evaluation depends on. Building on it, Chapter 3 develops the pipeline and its implementation together, with every experimental setting left as a named variable. The values, the material, and the results follow in Chapter 4, which surveys the full study, walks one reference experiment end to end, and answers all six questions (Section 4.5). Only then does Chapter 5 situate the method among fine-tuning, model editing, and activation steering, a deliberate order that lets the positioning rest on measured results rather than promised ones. Finally, Chapter 6 summarises the contributions and pairs each limitation with the future work that would move past it.

2 Background

This chapter assembles the technical foundation for the rest of the thesis, moving from the structure of the underlying models to the tools used to interpret, manipulate, and evaluate them. Each section supplies what the next one assumes. Section 2.1 introduces the decoder-only Transformer and the residual stream, the additive channel that makes a single hidden state both readable and writable. Building on that channel, Section 2.2 presents sparse autoencoders, which decompose a layer's activations into a dictionary of interpretable features. Section 2.3 then turns interpretation into control, describing how a feature-level intervention shifts a model's behaviour at inference time. Measuring such a shift requires reducing a model's output to a single decision, which Section 2.4 develops through logits, temperature, and multiple-choice scoring. The closing Section 2.5 supplies the statistical machinery for deciding whether an observed change is real.

2.1 Transformer Decoders and the Residual Stream

The models studied in this thesis are autoregressive large language models (LLMs) built on the Transformer architecture of Vaswani et al. [39]. Given a prompt, such models generate text one token at a time. Each prediction is conditioned on the prompt and on all tokens generated previously, as in the original GPT model of Radford et al. [40]. Two related Transformer families lie outside this scope: encoder-only models such as BERT [41], which produce contextual representations but are not trained for autoregressive generation, and encoder-decoder models such as T5 [42], which split input encoding and output generation into separate stacks. Accordingly, every model considered here is a decoder-only Transformer trained by *next-token prediction*, with *masked, causal self-attention* that allows each token to attend only to itself and to earlier tokens in the sequence.

Architecturally, each LLM consists of a single stack of identical transformer layers. Each layer comprises two learnable sub-layers that operate on a per-token hidden state $h \in \mathbb{R}^{d_{\text{model}}}$, where d_{model} is the *model dimension*, the width of the state:

- a *self-attention* sub-layer, which aggregates information across tokens, and
- a *multi-layer perceptron* (MLP) sub-layer, which acts on each token in isolation.

Self-attention computes, for each token, a weighted sum of value vectors over the current and preceding tokens, with the weights produced by a learned query-key matching [39]. This lets each token incorporate context from whichever earlier tokens are most relevant to it. The MLP, by contrast, is a position-wise feed-forward network applied independently to each token. It first projects the hidden state h up into a wider inner space of learned neurons, gates them with a nonlinearity so that only some remain active, and projects the result back down to d_{model}, matching the layer's input dimension. Each layer additionally interleaves a *normalisation* substep, written LN in what follows, that stabilises the scale of a token's hidden state before a

sub-layer reads it. The original Transformer used layer normalisation, which subtracts the mean and divides by the standard deviation before a learnable scale and shift [39], [43]. Most of the more recent models studied here instead adopt RMSNorm, which omits the mean subtraction and the shift, dividing each hidden state by its root mean square before the learnable scale [44]. Placement is likewise conventional rather than fixed. The original design applied normalisation after each residual addition (post-norm), whereas every model in this thesis follows the now-standard pre-norm arrangement and normalises the sub-layer input.[1]

A productive view of the layer, introduced by Elhage et al. [45], is the *residual stream*: a per-token vector that flows through the layer, with the attention and MLP sub-layers each adding their output back into it. Letting h^l denote the residual-stream state at the input of layer l, the standard pre-norm computation is

$$h^{l+1} = h^l + \mathrm{Attn}\big(\mathrm{LN}_1(h^l)\big) + \mathrm{MLP}\big(\mathrm{LN}_2\big(h^l + \mathrm{Attn}\big(\mathrm{LN}_1(h^l)\big)\big)\big),$$

where LN_1 and LN_2 are the two LN operations inside the layer.[2] The residual stream is therefore the sum of the embedding and every sub-layer write produced before layer l, and remains the channel through which information propagates between layers and into the final unembedding.

This additive structure gives the residual stream a status that single feed-forward activations do not have. Every sub-layer reads from it (after layer normalisation) and writes additively back into it. Consequently, an intervention applied at a single point in the residual stream is carried forward by this additive structure, reaching every subsequent layer and the final unembedding. The residual stream has become the natural object of study for mechanistic interpretability work on transformers [45], [47].

A transformer layer exposes many locations at which activations can be read or modified: the queries, keys, and values inside attention, the per-head attention outputs, the neuron activations inside the MLP, and the intermediate residual state between the two sub-layers. Three of these are natural sites for intervention: the residual stream and the two sub-layer writes (Figure 1). One is the residual stream, read as the state leaving a layer (`resid_post`). Because the output of layer l is the input of layer $l + 1$, this is the same activation as the state entering the next layer (`resid_pre`), so the two names label one object at adjacent layer indices rather than two independent sites. The others are the individual sub-layer writes, `attention_out` and `mlp_out`, each taken before it is summed into the stream.

These sites are not interchangeable. An intervention at `resid_post` addresses the cumulative state, everything the model has computed for the token up to that point, whereas one at `mlp_out` or `attention_out` addresses a single sub-layer's write before it joins that state. Either edit persists, since the stream carries both forward to every later layer. What the sites offer is therefore not different reach but a different object to edit.

[1] Some models, such as Gemma 2 and Gemma 3, additionally normalise each sub-layer's output.

[2] In a parallel-residual variant, adopted by the Pythia suite among others, both sub-layers read the same layer input, giving $h^{l+1} = h^l + \mathrm{Attn}\big(\mathrm{LN}_1(h^l)\big) + \mathrm{MLP}\big(\mathrm{LN}_2(h^l)\big)$ [46]. Each sub-layer still writes additively into the stream, so the residual-stream view applies unchanged.

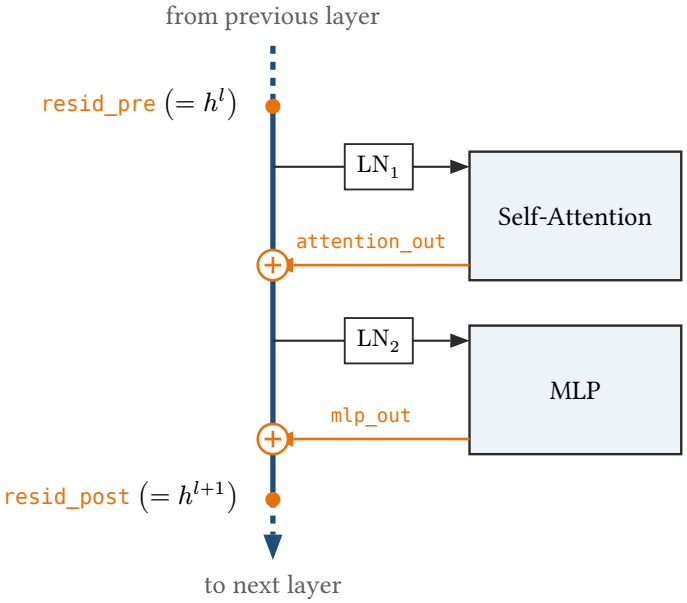


Figure 1: A (serial pre-norm) Transformer layer viewed as a residual stream, top to bottom: each sub-layer reads a layer-normalised copy of the stream and adds its output back into it. `resid_post` is the cumulative post-layer state. `attention_out` and `mlp_out` are the individual sub-layer writes. Orange marks the positions at which an intervention can be applied.

2.2 Sparse Autoencoders

A model's behaviour is easier to reason about once its activations are decomposed into meaningful units, and mechanistic interpretability calls such a unit a *feature*, taking it to be the fundamental object of analysis and to correspond to a direction in activation space [22]. What exactly counts as a feature is unsettled. Elhage et al. offer three candidate definitions, a feature as any function of the input, as a human-interpretable property, or as a property a sufficiently large model would reliably dedicate a neuron to, without committing to one [27], and some authors sidestep the ambiguity by calling the units a sparse autoencoder recovers *latents* instead [48]. This thesis adopts the broad, interpretable reading: a feature is a meaningful property represented in a model's activations, such as the topic of a text, its sentiment, or the language it is written in.

Less obvious is where such a property resides in the activation vector. The residual stream of Section 2.1 is a dense vector, and a feature of this kind seldom aligns with any single coordinate. Any single coordinate participates in representing many unrelated features, and any single feature is in general distributed across many coordinates. This entanglement is not incidental. The *linear representation hypothesis* holds that such features are encoded as directions in activation space, so that semantically meaningful operations correspond to vector arithmetic on those directions, a view that reaches back to the regularities found in static word embeddings [49] and has since been argued, though not settled, for the internal representations of large language models [50]. A space of width d_{model} holds only d_{model} mutually orthogonal directions,

but exponentially many *almost*-orthogonal ones.[3] To explain how a model can track far more features than it has dimensions, Elhage et al. demonstrate *superposition*: when the underlying features are sparse, the model exploits this geometric slack to encode many more than d_{model} of them in non-orthogonal directions, accepting the bounded interference that results and relying on its nonlinearity to suppress the cross-talk [27]. Superposition offers one account of why individual neurons are typically *polysemantic*, firing on collections of seemingly unrelated inputs, and why a model's computation cannot be read reliably off its raw neuron basis. Polysemanticity is itself an observed phenomenon, not a consequence the hypothesis derives, and both the linear and superposition claims remain working conjectures supported by toy models and probing evidence rather than proof. They are nonetheless the premises on which sparse-autoencoder interpretability rests.

A sparse autoencoder (SAE) attempts to undo this entanglement, learning an overcomplete dictionary that aims to lay the superposed features out one per latent coordinate [23], [24]. What the training recovers is a sparse code (a feature vector with few active entries) that reconstructs the activations well. Whether it captures the model's true generating features is an identifiability assumption, not a guarantee. The objective admits many near-equivalent solutions, a non-uniqueness returned to below. Given an activation $h \in \mathbb{R}^{d_{\text{model}}}$, the per-token hidden state of Section 2.1 read at a fixed site, the SAE encodes it into a high-dimensional, sparse feature vector $f \in \mathbb{R}^{d_{\text{sae}}}$ and decodes that vector back into the activation space,

$$\pi = W_{\text{enc}}(h - b_{\text{dec}}) + b_{\text{enc}}$$

$$f = \sigma(\pi)$$

$$\hat{h} = W_{\text{dec}} f + b_{\text{dec}}$$

where the encoder pre-activation π is passed through a sparsity-inducing nonlinearity σ. Subtracting the decoder bias before encoding is the canonical convention of Bricken et al. [23]. The feature dimension is typically overcomplete, $d_{\text{sae}} = E \cdot d_{\text{model}}$, with an expansion factor E that runs from a few-fold to over a hundredfold across the released suites used here, while the largest published SAEs reach millions of features [51]. Training minimises a reconstruction loss together with a sparsity objective, in the canonical formulation a mean-squared reconstruction error penalised by the L_1 norm of the feature activations,

$$\mathcal{L} = \left\| h - \hat{h} \right\|_2^2 + \lambda \|f\|_1$$

with the decoder columns constrained to unit norm so the penalty cannot be evaded by rescaling. The encoder is thereby pressured to explain each activation with as few active features as possible [23], [24].

The columns of W_{dec} constitute the learned dictionary, and an active feature contributes its column, scaled by its activation, to the reconstruction. Because the dictionary is overcomplete and the code is sparse, individual features are *intended* to be monosemantic, each ideally

[3] Such directions have pairwise cosine similarity below a small fixed bound, and their number grows exponentially in the dimension by the Johnson–Lindenstrauss lemma [27].

responding to a single human-recognisable concept. Such concepts range from concrete lexical regularities, for instance DNA sequences or base64 strings [23], to abstract notions such as sycophantic praise, deception, or code-security vulnerabilities [25]. In practice this ideal holds only partially. The two objectives pull against each other: tighter sparsity raises reconstruction error, so even a well-trained SAE reproduces h only approximately and leaves a reconstruction residual $h - \hat{h}$ [51], [52]. Partial monosemanticity and imperfect reconstruction both shape any intervention built on these features [38].

The activation function σ is what distinguishes the SAE families on which this thesis builds. Each enforces sparsity by a different mechanism. In the original formulation, plain ReLU SAEs rely on the L_1 penalty alone to push activations toward zero [23]. Because that penalty acts on magnitude rather than on count, it also shrinks the activations of the features it keeps, a bias the later families were designed to remove [51], [52]. *TopK* SAEs instead retain only the k largest pre-activations per input and zero the remainder, fixing the active-feature count exactly and decoupling sparsity from magnitude [51]. Relaxing this per-example rule to a per-batch one, *BatchTopK* SAEs keep the $k \cdot B$ largest activations across a batch of B inputs jointly, so that the same average budget of k features per input is met while an individual input may use more or fewer as the data demand [53]. *JumpReLU* SAEs gate each feature through a learned per-feature threshold θ, passing the pre-activation π through unchanged when it exceeds the threshold and zeroing it otherwise,

$$f_i = \pi_i \cdot \mathbb{1}(\pi_i > \theta_i),$$

where $\mathbb{1}(\cdot)$ is the indicator function [52]. No single family is uniformly best: stronger reconstruction at a given sparsity does not reliably translate into more interpretable or better-separated features [54]. All families nonetheless share the same encode-decode interface.

An SAE is always trained to reconstruct activations at a single layer and site inside the LLM, where a *site* is a within-layer location such as the residual-stream output or the MLP output of Section 2.1. Its dictionary is meaningful only there, and encoding an activation drawn from a different layer or site yields features without semantic warrant. The site also determines what the dictionary decomposes. Features learned on the residual stream describe the full cumulative state, whereas features learned on a sub-layer write describe that sub-layer's contribution alone. This locality carries a direct operational consequence. An intervention that manipulates SAE features must read and write the activation at exactly the layer and site on which the SAE was trained.

A trained feature is read off the inputs that activate it most strongly. These maximally activating dataset examples, shown alongside the feature's activation distribution and the output tokens it most promotes or suppresses, suggest which concept a direction tracks [23]. Neuronpedia, a large public catalogue, now indexes such *feature dashboards* for millions of features across many models [55]. Templeton et al. report at production scale that features found this way can act as causal handles [25]. Clamping a single feature that tracks the Golden Gate Bridge made Claude 3 Sonnet reference the bridge in almost any context and even self-identify as it [25], [26]. That evidence is qualitative rather than measured on a held-out task.

These dictionaries are powerful but imperfect, and the imperfections bear directly on what a feature-level intervention can achieve. A trained SAE commonly carries *dead* features that never activate, and as the dictionary grows, coherent concepts *split* into ever finer fragments, so the same concept may be represented by different features at different expansion factors [23], [51]. More troubling for control, a more specific feature can *absorb* part of a general feature's direction into its own decoder vector, so the general feature then fires inconsistently and the one-feature-one-concept ideal breaks [56]. The decomposition is not even unique [57]. On the indirect-object-identification task in GPT-2 Small, judged against supervised feature dictionaries, SAEs recover interpretable structure yet support less reliable intervention than the supervised features they approximate [38]. None of this refutes the SAE programme, but it bounds when such an intervention can succeed. If the exposed features are partial, split, or absorbed, the intervention is likely to help only where a genuinely causal feature happens to be cleanly represented.

Pre-trained SAEs are now openly released for every family introduced above. They cover models across a wide range of scales. ReLU SAEs in the style of Bricken et al. [23] appear in open releases such as Goodfire's for Llama-3.1-8B-Instruct [29]. TopK and BatchTopK SAEs following Gao et al. and Bussmann et al. [51], [53] are available in EleutherAI's open suites for the Pythia, SmolLM2, and Llama model families, alongside further community releases. JumpReLU SAEs come mainly from the Gemma Scope release of Rajamanoharan et al. and Lieberum et al. [28], [52], which supplies matched SAEs at every layer and site of the Gemma-2 2B and 9B models, with a successor release extending the same design to Gemma-3 [58].

2.3 Activation Steering

The SAE becomes an instrument of control only once its features are used to change what a model does, and activation steering is the general mechanism for doing so without touching weights. It manipulates behaviour by adding a vector to a hidden state during the forward pass: at a chosen layer and site the intervention takes the form $h \leftarrow h + \Delta h$, with $\Delta h \in \mathbb{R}^{d_{\text{model}}}$ a *steering vector* and the remaining computation left untouched [17]. Because the residual stream of Section 2.1 carries any such edit to every later layer and into the unembedding, a single well-placed displacement can shift the output distribution without retraining. The open question is how to choose Δh. Two approaches divide on whether the steering vector stays a single undecomposed direction or is expressed through the sparse feature basis of an SAE.

In the vector-space approach, Δh is one direction in activation space, formed without any decomposition. Contrastive Activation Addition (CAA), the representative method, collects prompt pairs that differ only in whether they exhibit a target behaviour, averages the activation at a layer on each side, and takes the difference of means as the steering vector [18]. Turner et al. had earlier derived a comparable vector from a single contrastive pair rather than an averaged set [17]. The result is one dense vector. It blends whatever features distinguish the two prompt sets, which makes it effective yet hard to interpret, since one cannot say which of the model's concepts the direction actually moves.

In the feature-space approach, the steering vector is expressed through the sparse basis of an SAE, so that the intervention names the concepts it moves. A sparse code $s \in \mathbb{R}^{d_{\text{sae}}}$ selects and weights individual features. The decoder turns that code into a displacement, $\Delta h = W_{\text{dec}} s$, which is added to the hidden state. A roundtrip variant instead adds the code in feature space and decodes the result, $h \leftarrow \text{decode}(\text{encode}(h) + s)$, applying the edit entirely within the dictionary but discarding the reconstruction residual $h - \hat{h}$ unless it is restored. Templeton et al.'s clamping experiments are an instance of this approach [25], and Joshi et al. select topic-relevant SAE features to steer generation [30]. Feature-space steering buys sparsity and a measure of interpretability, at the cost of depending on the quality of the underlying dictionary, whose limits Section 2.2 has already set out. This thesis takes the feature-space route, in an exact form that additionally restores the reconstruction residual.

2.4 Logits, Temperature, and Multiple-Choice Evaluation

Steering changes what a model outputs. Measuring that change requires reducing the model's output to a single choice. A language model does not emit probabilities directly. At each position it assigns every vocabulary token a real-valued score, its *logit*, where a higher logit marks a token the model favours. A *softmax* then converts the vector of logits into a probability distribution over the vocabulary,

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}},$$

where z is the logit vector. A *temperature* $T > 0$ divides the logits by T before the softmax, reshaping the distribution: $T = 1$ leaves it unchanged, larger values flatten it toward uniform, and as $T \to 0$ all mass concentrates on the highest-scoring token. The zero-temperature limit is therefore $\text{argmax}_i z_i$, deterministic greedy decoding. This determinism is what lets a multiple-choice answer be read straight from the logits, with no sampling.

Evaluating a model on a multiple-choice benchmark means turning this distribution into one chosen option, and two paradigms do so differently. Generation-based scoring decodes a continuation, by sampling or greedy decoding, and matches the produced text against the candidate answers. It mirrors deployment but introduces sampling variance at $T > 0$ and an ambiguous decision point when the output is verbose. Logit-based scoring instead reads the logits at the final prompt position, restricts them to the token identifiers of the answer letters, and takes the argmax over that small set [59], [60]. The second paradigm is deterministic and fast. It commits the model to a single token and so cannot credit an answer the model would have reached only through intermediate reasoning, a limitation that bounds what logit-based accuracy measures. The scoring setup introduces biases of its own. A first is a prior over the answer-letter tokens themselves, a tendency to favour some letters regardless of content, so that part of a measured preference reflects token identity rather than the option it labels [61]. A second is order sensitivity, since the letter a model picks may change when the options are permuted [62]. A third is tokenisation, the outcome depending on which token stands for an answer letter under a given

tokeniser and prompt format, so the scored token must be the one the model would produce as its answer. Together they mean a logit-based accuracy reflects the evaluation format as well as the model, so any fair comparison must hold the prompt, the letter set, and the option order fixed.

2.5 Statistical Foundations

Every accuracy reported in this thesis is an estimate from a finite sample, so a claim that an intervention helped must be separated from sampling noise by a significance test. For a single proportion, the *binomial test* gives the probability of observing at least as many successes as were seen, were the true success rate equal to a null value. The comparison that matters here is paired, however: each question is answered once without the intervention and once with it, on the same item, so the two outcomes are not independent. *McNemar's test* is built for this matched-binary setting [63]. It discards the questions both passes answer alike and examines only the *discordant* pairs, those the intervention flips. Under the null that an intervention is as likely to fix a question as to break one, the number of improvements among the discordant pairs is binomial with $p = 0.5$. A one-sided binomial test on the discordant counts is thus the exact form of McNemar's test.

A significance test reports whether an effect is distinguishable from zero, not how stable it is across the evaluation set. To estimate stability, the holdout is partitioned into K near-equal folds. The intervention and the baseline are scored on each fold, and the per-fold accuracy differences give K effect estimates. Their mean estimates the effect. A 95% confidence interval follows from the Student-t quantile with $K - 1$ degrees of freedom [64], appropriate for the small K used here. The interval is itself diagnostic. One that straddles zero signals an effect that varies from fold to fold rather than a dependable improvement.

A final caution concerns multiplicity. Evaluating many configurations and keeping the best raises a multiplicity problem. Reporting the winner's significance as though it were a single planned test would understate the risk of a false positive, because testing many hypotheses inflates the family-wise type-I error rate. Remedies range from the conservative Bonferroni correction [65] to false-discovery-rate control [66].

3 Methodology and Implementation

This chapter presents the steering pipeline as a single coherent contribution, combining the conceptual method and its implementation. The proposed method constitutes a five-phase, training-free pipeline that turns a large language model (LLM) with frozen weights, a matching sparse autoencoder (SAE), and a multiple-choice benchmark into a sparse intervention vector applied at inference time. Phase 0 partitions the benchmark once. Every later phase then works on its own disjoint share of that partition, building on the artefact the phase before it produced. The settings that govern the phases appear as named variables throughout this chapter, their concrete values reported in Chapter 4. Figure 2 gives an overview of the full pipeline, the phases in sequence, the data each consumes, and the artefact each hands to the next.

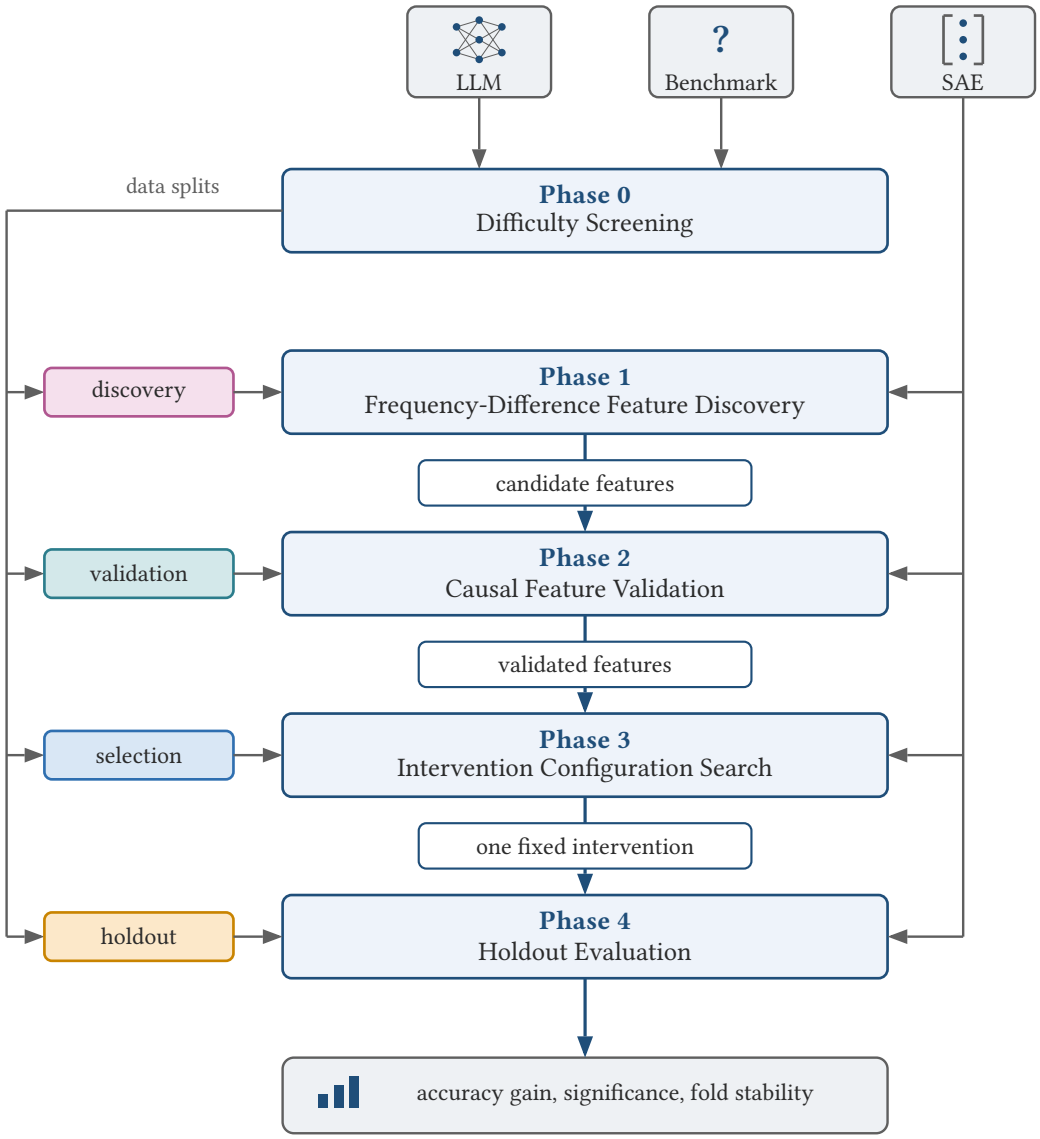


Figure 2: The five-phase steering pipeline, read top to bottom, runs the frozen model in every phase, the sparse autoencoder only from Phase 1 on.

The sequence is designed to keep the roles of the data separate. One subset is used to find candidate features, a second subset tests whether those features have a causal effect, a third subset selects the final intervention settings, and the holdout is reserved for the final evaluation. This separation ensures that the reported improvement is measured only on questions that played no role in discovering features, validating candidates, or choosing hyperparameters. Without this constraint, the pipeline would risk reporting performance on the same examples it had already used to decide which intervention looked best.

3.1 Feature-Space Intervention

Before the phases that build it are described, the intervention itself is defined. Those phases settle which features to move and how strongly, while what moving them means is fixed here. Expressed in the sparse feature basis of the SAE, the edit can name the features it moves and amplify or suppress each on its own terms.

The edit is carried by an *intervention vector* $s \in \mathbb{R}^{d_{\text{sae}}}$, the sparse code of Section 2.3, non-zero only at the selected features and signed by the direction each is moved, positive to amplify a feature and negative to suppress it. This feature intervention follows the precedent of Templeton et al., whose feature-clamping showed at scale that moving a single SAE feature can steer a model's behaviour [25]. Yet the model reads and writes hidden states of dimension d_{model}, not feature vectors, so applying s means crossing into the feature basis and back.

Installed as a forward hook at the SAE's layer and site, the intervention intercepts the hidden state $h \in \mathbb{R}^{d_{\text{model}}}$. Encoding maps this hidden state to its feature vector $f = \text{encode}(h)$, where individual features become addressable. There the intervention vector is added, and the sum is clamped to non-negative values. Decoding returns the result to the hidden-state space, and the part of h the SAE cannot reconstruct is added back, so that the intervention changes only what it intends to,

$$h' = \text{decode}(\max(f + s, 0)) + (h - \text{decode}(f)).$$

Only the final prompt position receives this edit. Under the scoring convention of Section 2.4, the answer is read from the logits at that position, so its hidden state is the one the decision passes through. Editing an earlier position would alter the representation of the question itself, and the layers above the hook would carry that corruption through attention into the final position (Section 2.1). Confining the edit to the final position therefore leaves the question's representation intact and moves only the state from which the answer is read.

Because decoding is a matrix multiplication plus a fixed bias, $\text{decode}(x) = W_{\text{dec}}x + b_{\text{dec}}$ (Section 2.2), the operator reduces to a single displacement (the vector added to the hidden state h),

$$\begin{aligned} h' &= (W_{\text{dec}} \max(f + s, 0) + b_{\text{dec}}) + h - (W_{\text{dec}}f + b_{\text{dec}}) \\ &= h + W_{\text{dec}}(\max(f + s, 0) - f). \end{aligned}$$

Even in this reduced form the displacement is not a fixed vector, since it is computed from $f = \text{encode}(h)$ and f differs for every input. The SAE must therefore run inside each steered forward

pass, encoding the hidden state before the displacement can be formed, so the intervention carries the SAE alongside the model at inference.

Two properties follow from this form. First, the displacement lies in the column space of W_{dec}, the subspace the SAE can represent. The reconstruction residual $h - \text{decode}(f)$, the component of h the SAE cannot express, is carried through unchanged. Only the part the SAE can represent is therefore moved. Restoring that residual matters because an SAE reconstructs its input only approximately (Section 2.2). A roundtrip that discarded it would move the activation by the SAE's reconstruction error as well as by the intended edit, so part of the steering would come from that error rather than from the chosen features.

Second, the clamp keeps every entry of the modified feature vector non-negative, as an SAE activation is. Because those activations start non-negative, an amplified feature rises by the full magnitude carried in s, while a suppressed feature falls no further than zero. An already inactive feature is thus left untouched by suppression.

The exact intervention runs the SAE at inference, as already established. As an alternative that needs no SAE at inference, an *approximated intervention* can be built. Its displacement is fixed in advance as $W_{\text{dec}}s$, which does not depend on the hidden state, so it can be precomputed once and added directly at the same position, with no encoding or decoding. The SAE is then needed only to build this vector, not to apply it, so the deployed model carries it alone. It matches the exact displacement wherever $f + s$ stays non-negative, since $\max(f + s, 0) - f$ is then just s, and differs only where a suppressed feature would fall below zero, which the clamp floors but the fixed vector does not. How much this difference costs in practice is measured in Chapter 4.

3.2 Phase 0: Difficulty Screening

With the intervention defined, Phase 0 selects which benchmark questions the later phases will use and partitions them into disjoint subsets. Because not every question is equally informative, only some are worth working on. A steering intervention can only improve an answer the model gets wrong: in a multiple-choice setting a correct prediction cannot score higher under argmax scoring, and steering a question the model already handles well risks converting a right answer into a wrong one. Identifying the informative ones therefore requires scoring every question against the model, and doing so before any sparse autoencoder is involved, since the difficulty assessment must be independent of whichever SAE configuration is later tested. Figure 3 shows this construction in full.

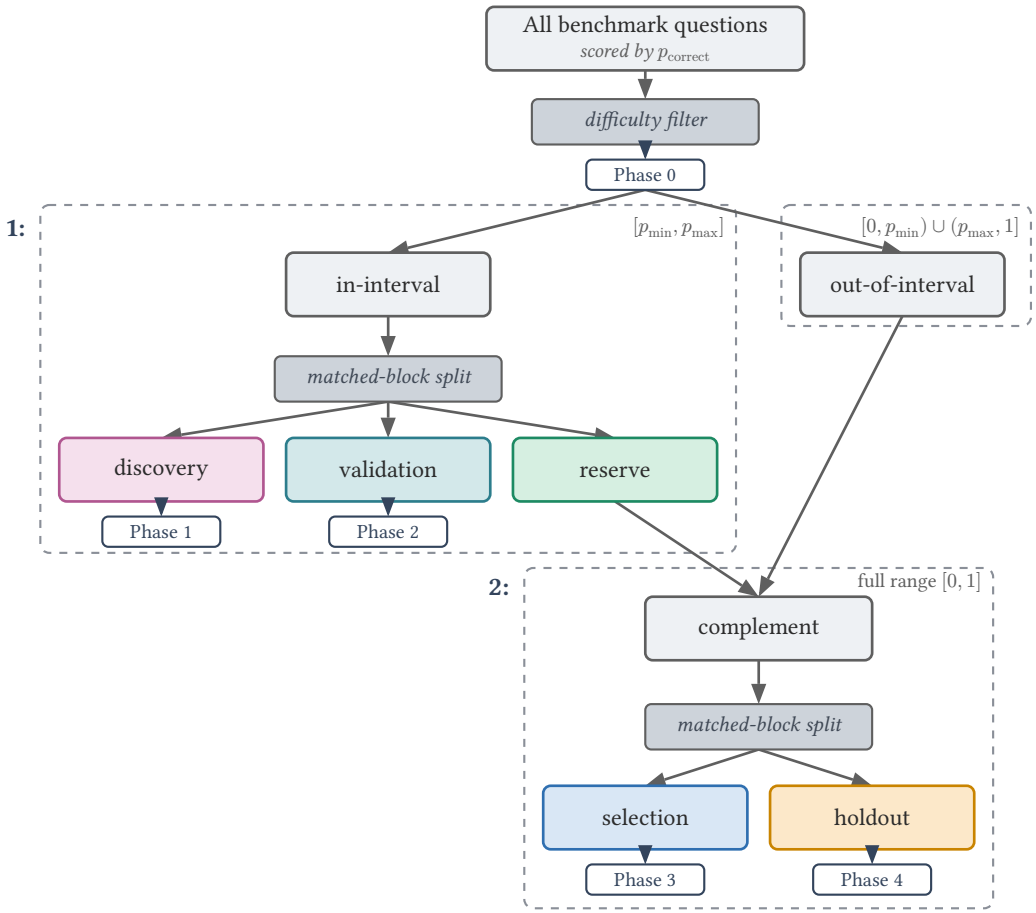


Figure 3: The data subsets. Dashed boxes group them by the difficulty range of their questions, shown in each corner.

A language model does not merely pick an answer but grades the options. At the final prompt position it assigns every vocabulary token a real-valued logit (Section 2.4). Restricting these logits to the answer-letter tokens $\mathcal{A}$ and normalising by softmax yields a probability distribution over the answer options alone, and the probability it places on the correct letter y is the difficulty score

$$p_{\text{correct}} = \frac{e^{z_y}}{\sum_{i \in \mathcal{A}} e^{z_i}},$$

where z_i is the logit of answer letter i. Near one, almost all of the mass sits on the correct letter. A value near zero leaves almost none there. Phase 0 records this score for every benchmark question.

These scores follow the scoring convention used throughout the pipeline. The predicted answer is the argmax over the same answer-letter logits, so a single forward pass yields both the hard prediction and the soft score p_{correct}, with no autoregressive decoding (Section 2.4). Each read is fully deterministic under the conditions fixed in Chapter 4. That determinism matters structurally: an intervention is measured by comparing a question's prediction with and without it on the same input, and only a deterministic baseline ensures the comparison is unconfounded by sampling noise. Because the difficulty assessment does not involve the SAE, one screening

pass at the start of an experiment suffices for every intervention configuration that experiment evaluates, and the p_{correct} values are computed once per experiment and reused across its phases.

The recorded scores then determine the working set. A difficulty interval $[p_{\min}, p_{\max}]$ on p_{correct} defines the questions from which features will be discovered and validated (Figure 3, 1). That later work draws its signal from the contrast between the model's successes and its failures, so it needs questions on which both outcomes occur. Above $p_{\max}$, the model concentrates its probability mass on the correct letter and answers nearly every question correctly. Included in the working set, such questions would flood the success side with easy examples, so the contrast would favour features that track the model's confidence rather than features that separate success from failure. Below $p_{\min}$, the reverse holds. The model answers nearly every question incorrectly, so these questions would contribute failures and almost nothing to contrast them against. Both bounds are held fixed across every experiment, since adjusting them based on downstream outcomes would corrupt the separation between selecting questions and evaluating the intervention. The interval is applied against each model's own p_{correct} distribution and is therefore model-relative: the bounds select the questions this particular model finds hard, not those that any external measure or human assessment would call difficult.

Questions within this interval are divided into three disjoint subsets by a matched-block split, which sorts them by p_{correct} and randomly assigns within consecutive blocks of similar difficulty: a discovery subset, a validation subset, and a reserve set aside for later evaluation. The first two supply the questions for the feature-selection work described in the phases that follow, while this reserve is held back from that work. Questions outside the interval, those scoring below $p_{\min}$ or above $p_{\max}$, are likewise excluded from feature work. Together, this reserve and the out-of-interval questions form the full complement of questions not assigned to discovery or validation (Figure 3, 2). This complement spans the complete difficulty range of the benchmark. A second matched-block split divides it equally into a selection subset, on which the best intervention configuration will be chosen, and a holdout, reserved for the final accuracy estimate.

Splitting the questions at random would leave each subset's difficulty distribution to chance, so one subset could end up markedly easier or harder than another. Instead, the questions are sorted by p_{correct} and divided among the subsets in consecutive blocks, so that every subset draws evenly from the full difficulty range. Smaller blocks hold questions of more similar difficulty, so they are made as small as they can be while still dividing exactly into the requested shares. Within each block the questions are assigned to the subsets at random, in the requested proportions, with the draw fixed by a seed so that the split can be reproduced. Figure 4 illustrates the construction.

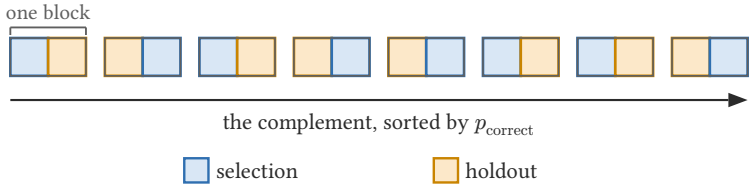


Figure 4: The matched-block split, here dividing the complement equally into selection and holdout.

This near-matching matters differently for each split. For the first, it keeps discovery, validation, and reserve drawn from closely matched difficulty profiles, so no subset receives a systematically easier or harder sample. Applied to the second, the same construction makes selection and holdout comparably hard, so the settings are chosen on questions about as difficult as the holdout they are judged on. Because that holdout is never seen by any earlier step, the accuracy reported on it is not inflated by optimisation toward those specific questions.

3.3 Phase 1: Frequency-Difference Feature Discovery

Where Phase 0 sorted questions by difficulty, Phase 1 turns to the features, searching for the few whose activation separates the questions the model gets right from those it gets wrong. The SAE holds far more features than the causal screen of Phase 2 could examine one by one (Section 2.2), so a cheaper criterion must narrow them to a shortlist first. Correlation supplies that criterion. Every question in the discovery subset arrives marked correct or incorrect by the screening pass, supplying the labels this criterion needs. Figure 5 previews the phase on an illustrative example. Each discovery question is answered by the LLM, marked correct or incorrect, and its active SAE features recorded (Figure 5, 1). The phase then counts each feature's activation on correct versus incorrect answers (Figure 5, 2) and ranks it by the difference into amplify and suppress lists (Figure 5, 3).

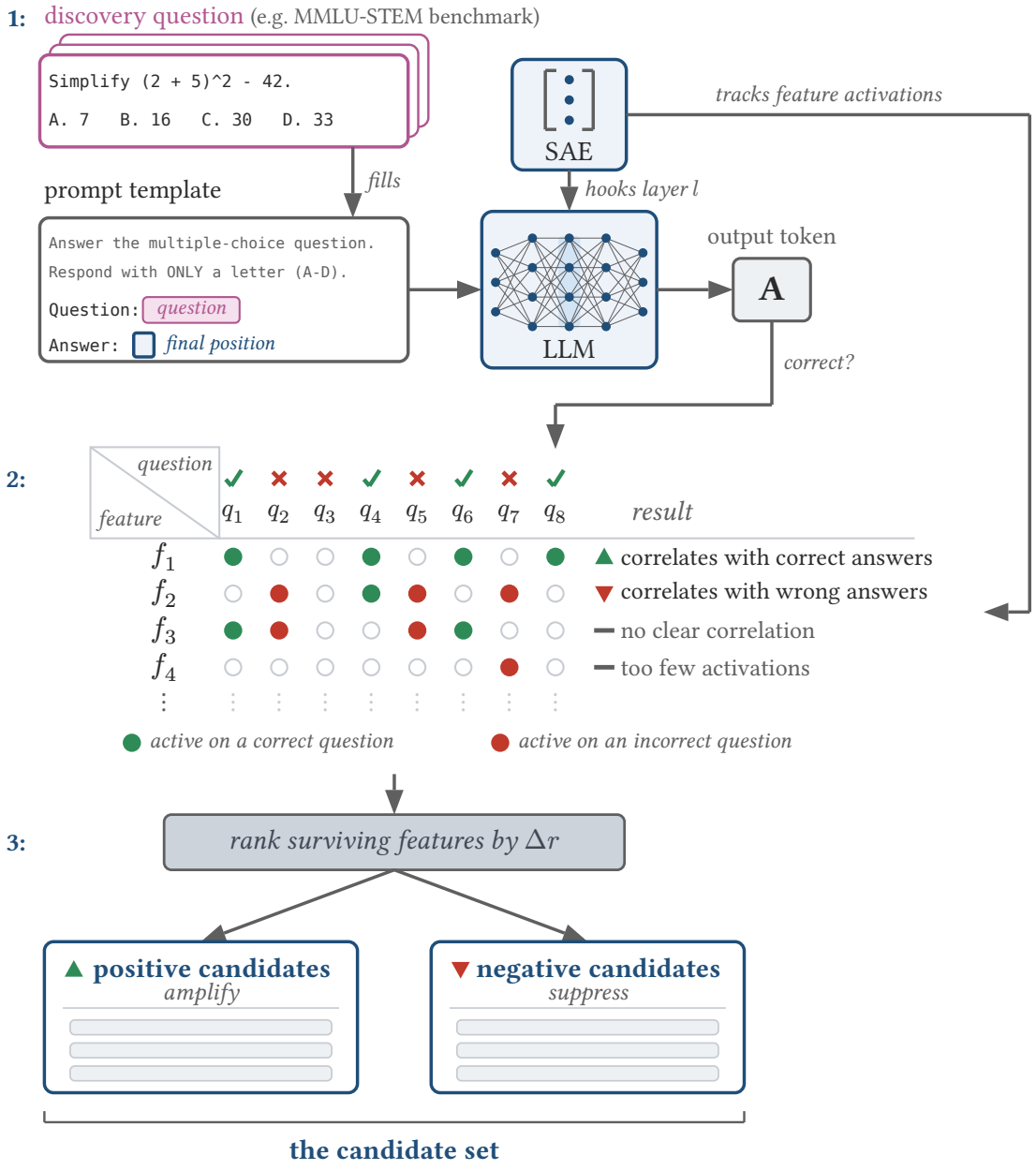


Figure 5: Feature discovery in Phase 1, on an illustrative example. Features that fire mainly on correct answers (green) become candidates to amplify, those that fire mainly on incorrect ones (red) candidates to suppress.

The premise is intuitive. A feature that tends to be active when the model answers correctly, and quiet when it errs, is a candidate for promoting the correct behaviour, whereas a feature with the opposite profile is a candidate for suppression. The evidence this search yields is correlational. It surfaces features whose activation co-occurs with success, not features shown to bring it about. That distinction matters, because an apparently relevant SAE feature need not be a causal one [38]. The method is therefore organised to keep this correlational step separate from the causal screen that follows.

Each question in the discovery subset is run through the model once, on the unmodified prompt and under the same logit read used for screening (Figure 5, 1). From that single pass two quantities are recorded: whether the answer was correct, and the feature vector at the final prompt position, obtained by reading the hidden state at the SAE's layer and site, then passing it through the encoder defined in Section 2.2. The final position is the one from which the answer is read, so the features there describe the representation on which the decision turns. Since that read is deterministic, the correct and incorrect labels recovered here coincide with those Phase 0 assigned, and discovery is conditioned on the outcomes the screening measured. A feature is counted as active when its encoded activation is positive. Given the sparse code of an SAE, only a small fraction of the d_{sae} features are active on any one question, so each pass is summarised by its active indices rather than a dense vector, keeping the bookkeeping tractable even when the feature space runs to hundreds of thousands of dimensions.

Aggregated over the subset, each feature acquires two activation frequencies. The first, $r_{\text{correct}}(j)$, is the fraction of correctly answered questions on which feature j is active. Its counterpart, $r_{\text{incorrect}}(j)$, is the fraction of incorrectly answered questions on which the same feature is active. Each estimates a conditional probability, that feature j is active given a correct answer and given an error respectively. Because the correct and incorrect groups differ in size, a raw count would depend on the size of each group as much as on how often the feature fires within it. Dividing each count by the size of its group removes this dependence, leaving two frequencies that can be compared on equal terms. The discovery score is the difference of the two,

$$\Delta r(j) = r_{\text{correct}}(j) - r_{\text{incorrect}}(j),$$

its *activation-frequency difference*. A large positive $\Delta r(j)$ marks a feature far more often active on success than on failure, hence a positive candidate, and a large negative value identifies the reverse, a negative candidate. Scores near zero, by contrast, mark features that are active about as often on success as on failure, qualifying as neither positive nor negative candidates (Figure 5, 2). The construction mirrors, in the sparse feature basis, the difference of class means that Contrastive Activation Addition forms in the raw activation space [18], with a feature's binary on-or-off event in place of a real-valued activation and the success-and-failure split in place of the behavioural contrast.

By design, the score counts activation as a binary event and ignores how strongly a feature fires. Two properties of SAE activations motivate that choice. First, activation magnitudes are not calibrated for comparison across features. Different features fire at different natural scales in any SAE family, and in the L_1-penalised SAEs of Section 2.2 the penalty additionally shrinks the activations it retains [51], [67], an effect Wright and Sharkey find most pronounced for features that fire frequently but weakly [68], so a difference in mean magnitude would partly reflect each feature's idiosyncratic scale rather than its association with the outcome. Second, a mean can be dominated by a few extreme activations, whereas a frequency is bounded in $[0, 1]$ and, by construction, cannot. Counting occurrences therefore captures what discovery needs, namely how reliably a feature accompanies success or failure, without entangling that signal with magnitudes the training objective never calibrated.

A frequency estimated from very few observations is unreliable [69]. If a feature is active on only one or two questions, its frequency rests on almost no evidence, and the difference of two such frequencies can reach an extreme value by chance, crowding the top of the ranking with artefacts [70]. To prevent this, a feature is admitted to the ranking only when it is active on at least $n_{\min}$ of the subset's questions, successes and failures counted together. The threshold scales with the size of the discovery subset, subject to a small fixed floor, so that a feature must clear a minimum count before its frequencies are trusted. Its value is fixed in advance rather than tuned, exactly as the difficulty bounds were. Features below the threshold are discarded, and those above it are ordered by $\Delta r(j)$.

The output of Phase 1 is a pair of ordered lists, the features with the most positive activation-frequency difference and those with the most negative, each cut to a fixed length (Figure 5, 3). Together they constitute the candidate set, a shortlist distilled from the full SAE feature space and reduced to those whose activation is most strongly associated with the model's successes and failures. Each feature earned its place by correlation alone. Whether moving it actually changes an answer is a separate question, and resolving it is the role of the causal screen to which the candidate set now passes.

3.4 Phase 2: Causal Feature Validation

The candidates arrive from Phase 1 ranked by correlation, a weak foundation for intervention, and the task now is to test which of them actually cause the model's successes. A feature that activates more often when the model succeeds may carry no responsibility for that success, responding instead to some property that merely accompanies the questions it gets right. The difficulty screening of Phase 0 sharpens the worry. Within the difficulty interval, the model answers correctly only when the correct option outscores the others, so the questions it gets right tend to carry a higher p_{correct} than those it gets wrong. Correct and incorrect questions therefore differ in the model's confidence, not only in the outcome. A feature that merely tracks that confidence would then fire more often on the correct questions, earning a large activation-frequency difference without bearing on what makes an answer right. Phase 2 confronts this by intervening, manipulating each candidate feature directly and recording whether the manipulation changes the model's answers, the evidence that bears directly on causal relevance [25], [38]. The test runs on the validation subset, kept disjoint from the discovery subset, so that every feature is judged on questions that played no part in proposing it.

Each candidate is tested on its own. For a single feature, an intervention vector is built that is zero everywhere except at that feature, where it carries a fixed test strength α_{test}, positive for a feature taken from the positive list and negative for one from the negative list. The vector is applied through the exact intervention of Section 3.1. Each question in the validation subset is then re-scored under the single-feature intervention and set against a baseline pass that applies no intervention. Because both passes are deterministic, any flip in an answer is attributable to the intervention and not to sampling. The screen applies a single test strength rather than a range. A sweep would multiply the cost of this phase by the number of strengths tested, and the

goal is only to detect whether moving a feature helps, not to tune how strongly to move it. That tuning happens later, with the other intervention settings.

The change in validation accuracy that a feature's intervention produces is its *isolated causal effect*, written $\Delta a(j)$ for feature j and equal to the intervention accuracy minus the baseline accuracy on the same questions. On top of the correlational ranking, the phase thus adds a causal criterion: a feature advances only if $\Delta a(j) > 0$, meaning its intervention corrects more questions than it breaks. Features that show no effect under this screen are set aside, not disproven, since they might still act at another strength or in combination with others. The fraction set aside is reported as a diagnostic, indicating how many correlational candidates fail to show an isolated effect rather than proving their correlation spurious. No significance threshold is imposed at this stage. This screen narrows the field rather than delivering a verdict, so the bar is kept permissive, and the rigorous statistical test falls instead on the assembled intervention, later, on the untouched holdout.

Features are tested one at a time, never in combination. Isolating a single manipulation is what licenses the causal reading, since when only one feature moves, any change in the answer can be ascribed to that feature's manipulation and to nothing else, as a controlled experiment varies one factor while holding the rest constant. Joint manipulation would forfeit this. Two features moved together can interfere, one masking or cancelling the other, so a combined change could not be cleanly apportioned between them. The number of combinations is prohibitive in any case, growing exponentially in the size of the candidate set, while the isolated test grows only linearly. Combined behaviour is not dismissed. Whether features still help once assembled into one vector is settled when the final intervention is configured, but the screen that decides which features are eligible is kept deliberately simple, each tested alone.

What leaves Phase 2 is a smaller set of features, those whose isolated manipulation measurably improves the answers, divided into a group that helps when amplified and a group that helps when suppressed. Each retains its measured effect $\Delta a(j)$ beside the activation-frequency difference it arrived with. The meaning of a feature plays no part in this screen, and no candidate is inspected for the concept it might encode. This validated set, narrower than the correlational shortlist before it and selected for showing an effect rather than mere co-occurrence, is what Phase 2 carries into the configuration search that assembles and tunes the intervention.

3.5 Phase 3: Intervention Configuration Search

Phase 3 consolidates the selected features into a single working intervention. Discovery offers the correlational shortlist, validation the smaller set whose isolated manipulation improved the answers, and the search may draw on either. What these lists fix is which features are available, not how to assemble them into one vector. That assembly is the task of this phase, settling how the available features combine and how strongly each is moved.

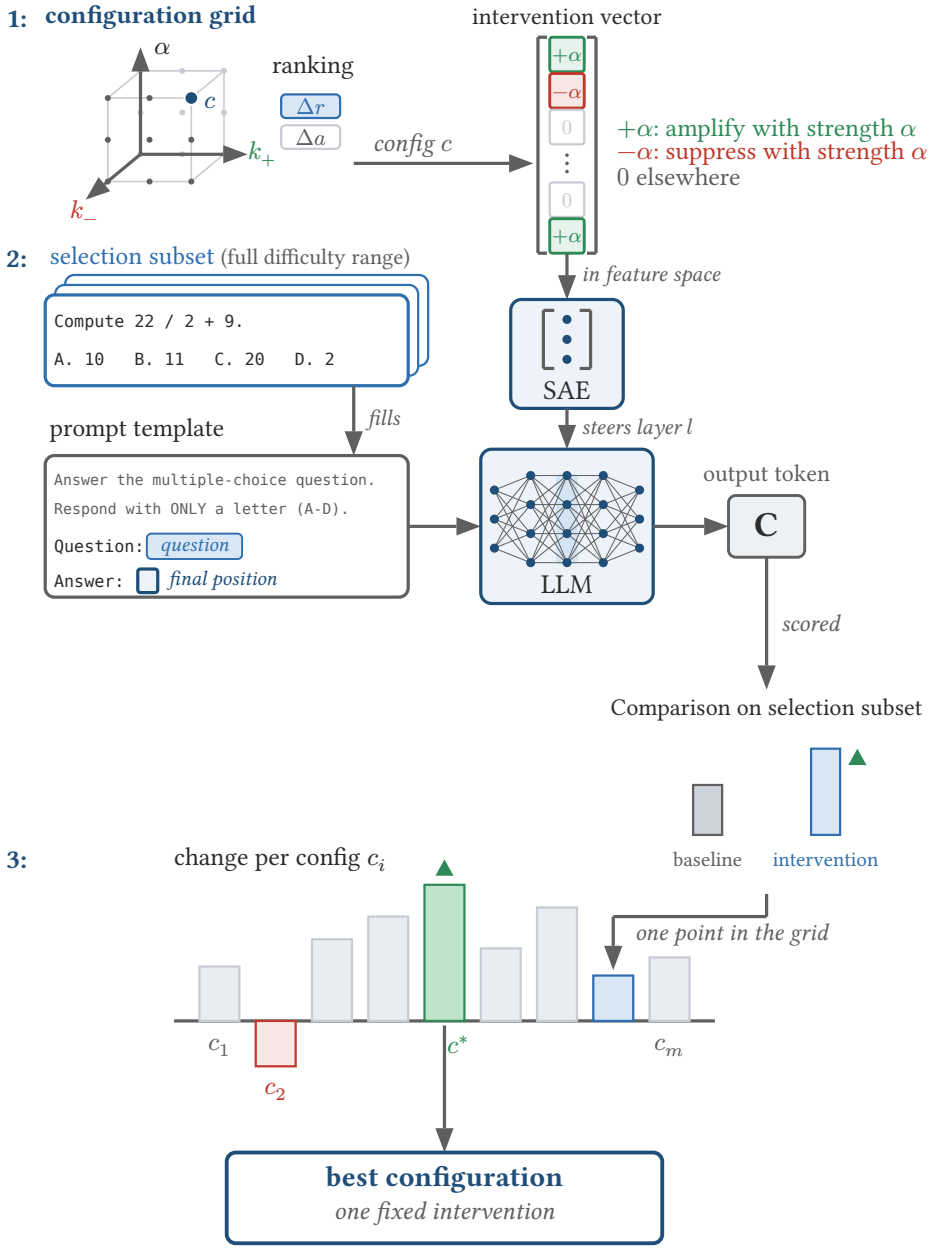


Figure 6: Configuration search. Green marks amplification and the best configuration, red marks suppression and configurations that underperform the baseline.

As an overview, an illustrative example is shown in Figure 6. One configuration is drawn from the configuration grid and assembled into a sparse intervention vector (Figure 6, 1). Applied to the model, the vector steers a pass over the selection subset (Figure 6, 2), and the accuracy gain this yields ranks the configuration against the rest of the grid (Figure 6, 3).

Four settings define the search. The first two, k_+ and k_-, count the features drawn from each side, the number amplified and the number suppressed, tuned independently so the two directions need not contribute in equal measure. Third is the intervention strength α, the magnitude carried by every selected feature. Fourth is the ranking criterion, which decides whether features are taken from the discovery shortlist by frequency difference or from the validated set by causal

effect. The three numeric settings span the configuration grid, and the ranking criterion stands beside that grid as a two-way switch (Figure 6, 1).

A configuration of the four yields a single intervention vector, sparse in the feature basis: each selected feature carries the strength, positive on the amplified side and negative on the suppressed. Through the exact intervention of Section 3.1, the vector then enters the forward pass and steers the model over the selection subset (Figure 6, 2). It is the first vector to move several features at once, and the first to move them at strengths other than the one validation tested. Phase 2 screened each candidate alone, at a single test strength. Whether the candidate features help in combination, and at the swept strengths, is therefore measured by the search rather than assumed.

Choosing the ranking criterion does more than reorder features. Building from the validated set confines the intervention to features that survived the causal screen, while building from the shortlist reaches back past it and admits features on correlation alone, including those it rejected. The choice therefore decides whether Phase 2′s validation constrains the final intervention at all, which makes it a direct test of whether that validation earns its keep.

Each configuration is scored by a single number, the gain in accuracy it brings on the selection subset over the baseline (Figure 6, 3). The largest gain wins by a strict maximum with no tolerance band, and ties are broken by the grid's fixed enumeration order. No significance test is applied at this stage, by design. The chosen configuration is the best of many the grid evaluates on the selection subset. Part of its winning gain is therefore chance, the luck of fitting those particular questions, and a test on the same questions would mistake that chance for a real effect [71]. A fair test has to use questions the search never saw. That is the role of the holdout, and Phase 4 applies it there once, to the chosen intervention.

What leaves Phase 3 is a single configuration, with every setting fixed. Nothing in it remains open, so the final stage has only to measure it, not to search.

3.6 Phase 4: Holdout Evaluation

The configuration fixed in Phase 3 now meets the holdout for the first time. Discovery, validation, and the configuration search each drew on their own disjoint slice of the benchmark, and the holdout took no part in any of them. The estimate it yields is therefore unbiased by the choices that produced the intervention, and the significance test that Phases 2 and 3 deferred can now be applied where it is valid. Phase 4 measures the fixed intervention on this set, reporting the gain it brings, whether that gain is significant, and whether it holds steady across the holdout.

The evaluation is organised along two axes. One axis varies the question set. Spanning the benchmark's entire difficulty range, the full holdout matches the scope of the central question and of the selection in Phase 3, so any claim about the benchmark as a whole rests on it. Its inside-interval part, reported beside it, isolates the effect on the questions the pipeline targeted, those inside the difficulty interval $[p_{\min}, p_{\max}]$. A gap between the two readings can come only from the out-of-interval questions, since nothing else separates them. The second axis is the

form of the intervention. Its exact form applies the intervention of Section 3.1 in full, while its approximated form adds the precomputed displacement, with no SAE in the forward pass. Crossing the two axes gives four readings, each reporting a baseline accuracy, an intervention accuracy, and the gain between them.

Whether a gain marks a real effect or only chance is decided by a significance test. Because each question is answered once with the intervention and once without, on the same item, the two outcomes are paired, and only the questions whose answer the intervention flips carry information. Among those discordant questions, the test asks whether corrections outnumber breakages by more than chance would produce, a one-sided binomial test on the discordant counts that is the exact form of McNemar's test (Section 2.5). Only an imbalance unlikely under that even-odds null counts as more than sampling noise.

A single significance verdict does not reveal whether the gain is steady across the holdout or driven by a handful of questions. To gauge this, the full holdout is partitioned into K near-equal folds under a seeded shuffle, and the exact intervention, already fixed, is scored on each fold in turn. The spread of the per-fold gains around their mean, summarised by a Student-t confidence interval (Section 2.5), measures the effect's stability rather than testing it afresh. An interval clear of zero indicates a gain that recurs across folds. One straddling zero marks an effect too variable to trust. Because the intervention is never refitted, the folds are disjoint evaluation partitions rather than cross-validation splits, so no question informs more than one fold's estimate.

These four readings invite a multiplicity problem: reporting whichever one clears the threshold would let the most favourable of several gains reach significance by chance alone (Section 2.5). The exact intervention on the full holdout is therefore fixed in advance as the primary endpoint, since it matches the benchmark-wide scope of the central question and applies the method in its defining form. It alone receives the stability check above, and its significance is read as confirmatory. The other three readings are reported beside it as descriptive support.

Taken together, the phases form an economy of computation, each stage shrinking what the next must handle. The difficulty screen sends only a band of questions, not the whole benchmark, into the feature work that follows. Discovery reduces a feature space of up to hundreds of thousands of dimensions to a short candidate list. Validation tests only those candidates. The configuration search ranges over the candidate list and the validated subset within it, not all the SAE's features. Holdout evaluation comes last, a single fixed intervention measured on data no earlier phase used to choose it. What began as an open-ended search over an enormous feature space narrows, stage by stage, to one vector and a single clean measurement of it.

4 Evaluation

A pipeline described in variables is not yet an experiment. Chapter 3 fixed the phases and named the settings that govern them, deferring every concrete value to this chapter, which supplies them together with the material the pipeline was run on and the conditions under which it ran. The study comprises 98 completed experiments. Each pairs one large language model with one public sparse autoencoder (SAE) at one layer and one site, runs all five phases against one of two primary benchmarks, and is measured once on a holdout that no earlier phase touched. Fifteen models spanning 70 M to 8 B parameters and 52 SAEs are represented across them.

4.1 Experimental Setup

The setup is fixed in four steps, from the benchmarks and the models through the parameter values to the measurement protocol. Its last step also describes the measurement itself, since the difficulty interval reshapes each benchmark differently and what it leaves behind governs how every result in this chapter must be read. Properties of that kind are better stated before any result exists than discovered after one.

4.1.1 Benchmarks

Four multiple-choice benchmarks enter the study. CommonsenseQA [72] and ARC-Challenge [73] are the primary benchmarks, while OpenBookQA [74] and HellaSwag [75] enter only as transfer targets. Every item consists of a short text followed by lettered answer options, exactly one of which is correct. Scoring follows the convention Section 2.4 sets out and Section 3.2 adopts, reading the logits at the final prompt position and taking the argmax over the answer-letter tokens.

Benchmark	Options	Published splits used	Questions
Primary			
CommonsenseQA [72]	A–E	`train` 9,741 + `validation` 1,221	10,962
ARC-Challenge [73]	A–D	`train` 1,119 + `test` 1,172	2,291
Transfer			
OpenBookQA [74]	A–D	`train` 4,957 + `test` 500	5,457
HellaSwag [75]	A–D	`validation` 10,042	10,042

Table 1: The four benchmarks. *Published splits used* lists the benchmark providers' own release labels with their question counts, which are then combined into one question pool per benchmark. The study's own subsets are drawn from that pool in Phase 0.

Each experiment takes one primary benchmark through all five phases, from difficulty screening to the holdout measurement. Once it has fixed its intervention, that intervention is additionally scored on the full question pool of the three benchmarks that played no part in its discovery, validation, or selection. A CommonsenseQA experiment is therefore scored on ARC-Challenge,

OpenBookQA, and HellaSwag. What that asks is whether the effect carries beyond the benchmark on which the intervention was built. Neither transfer target is ever run through the pipeline, so transfer to them is measured rather than assumed.

The four differ in what they ask of a model. CommonsenseQA poses questions whose answers rest on everyday knowledge the question text does not supply [72]. Talmor et al. sample a source concept from ConceptNet [76] with three target concepts standing in the same relation to it, and a crowdworker writes one question per target, adding two further distractors, one again from ConceptNet, to reach five options. Since the three targets relate to the source identically, the question itself has to carry the commonsense that separates them, and surface association with the source does not settle the answer. ARC-Challenge draws on grade-school science examinations, and its Challenge partition is defined by exclusion: it retains exactly those questions that both a retrieval solver and a word-co-occurrence solver answered incorrectly, which removes the items answerable by lexical overlap with a corpus [73]. Modelled on an open-book examination, OpenBookQA ties each question to one of 1,326 elementary science facts and requires that fact to be combined with broad common knowledge the book itself does not contain [74]. HellaSwag is not a question-answering task at all. It presents a short context taken from video captions or a how-to article and asks which of four endings continues it plausibly, the three wrong endings machine-generated and then repeatedly replaced, over several rounds, until a trained classifier could no longer reliably pick them out [75].

The original release splits are used only to form the question pools in Table 1. Phase 0 (Section 3.2) then creates the study's own discovery, validation, selection, and holdout subsets within each primary pool. Where the other three draw on two published splits, HellaSwag contributes only its `validation` split, which was designed to combine categories seen during training with held-out zero-shot categories [75]. Its 10,042 labelled questions are nearly twice the size of the largest primary holdout in this study and provide ample data for the transfer evaluation. Public benchmark questions may already have appeared in a model's pretraining or fine-tuning data. Deng et al. document overlaps between contemporary benchmarks and pretraining corpora and note that such contamination can inflate benchmark scores [77]. Such exposure cannot be ruled out here. For every reported holdout and transfer effect, however, the same model answers the same questions without and with steering. Any prior exposure is therefore shared by both passes rather than introduced by the intervention. The paired difference remains a valid estimate for these question pools, although the absolute accuracies may be inflated and generalisation to unseen questions remains uncertain.

Template shape. The four share one template. It comprises an instruction line naming the task, a second line restricting the response to a single letter, the item's text, the options one per line in the order the dataset stores them, and a final "`Answer:`" line. Three of them label that text "`Question:`". HellaSwag heads it "`Context:`", heads its options "`Possible endings:`", and asks for the best ending rather than the correct answer, since what it presents is a passage to be continued rather than a question to be answered. Appendix B gives the instruction and restriction lines of all four benchmarks and one prompt as a model receives it. The options are never permuted, so every question retains the same order in the baseline and steered passes.

Applying different permutations would mix the intervention's effect with the change in order. Where a tokeniser ships a chat template, the prompt is wrapped in it before scoring, so each model receives the format declared by its own tokeniser. Holding the prompt, letter set, and option order fixed between the baseline and steered passes satisfies the condition for a fair paired comparison established in Section 2.4. Consequently, the measured difference cannot be attributed to a change in any of these format variables. It estimates the intervention's effect under the evaluation format used here. Whether that effect generalises to alternative prompts, letter sets, or option orders was not tested.

Why these four. Five properties distinguish the four as a set. The first is the number of answer options. Each presents four or five, which places chance accuracy at 20% or 25%. On a two-option benchmark it would sit at 50%. Every question the model answered wrongly would then have exactly one alternative left, the correct one, so an intervention that merely unsettled the model's choice could convert wrong answers into right ones without supplying anything. The same holds in reverse for correct answers. Four or five options reduce this concern, since a uniformly random redirection from an incorrect answer reaches the correct option only one time in three or four. Volume is the second property. Phase 0 divides one benchmark into four disjoint subsets, and each of the four phases that follow draws on one of them.

Contrast between the two primary benchmarks is a third property. Beyond the option count, they differ in what an answer has to draw on, everyday knowledge in one case and grade-school science in the other, so the study is not tied to a single subject matter [72], [73]. The fourth property is the size gap between them. CommonsenseQA pools 4.8 times as many questions as ARC-Challenge (Table 1), which lets the same pipeline be observed where discovery data is abundant and where it is scarce. This scarcity is a condition and a limitation at once, since the smaller pool limits how many questions can enter each subset. The fifth property is the distance in task and prompt format between a transfer target and the benchmark an intervention was built on. OpenBookQA sits closest to ARC-Challenge on these dimensions, sharing its domain, its four options and its prompt instruction verbatim, whereas HellaSwag is furthest from the three question-answering benchmarks, the only one asking for a continuation rather than an answer, which is why its template differs from the other three [74], [75].

4.1.2 Models and Sparse Autoencoders

Table 2 lists the fifteen models on which the pipeline was run. Five families are represented: Pythia [46], SmolLM2 [78], three generations of Gemma [79], [80], [81], Llama 3 in its 3, 3.1 and 3.2 releases [5], and a DeepSeek-R1 distillation of `Qwen2.5-Math-1.5B` [82], [83]. Every model is named by its HuggingFace identifier throughout this chapter, because the shorthand several releases go by is ambiguous. In `google/gemma-2b` the digit is the parameter count, whereas in `google/gemma-2-2b` the first digit is the generation, and `meta-llama/Llama-3.2-1B` is a base model while `meta-llama/Llama-3.2-1B-Instruct` is its instruction-tuned sibling.

Model	Params	d_{model}	Layers	Intervened at	Sites	SAEs	Exp.
`EleutherAI/pythia-70m` [46]	70 M	512	0–5	2, 5	both	4	8
`HuggingFaceTB/SmolLM2-135M` [78]	135 M	576	0–29	15, 27	`mlp_out`	2	4
`EleutherAI/pythia-160m` [46]	160 M	768	0–11	5, 6, 11	both	3	5
`EleutherAI/pythia-160m-deduped` [46]	160 M	768	0–11	5, 11	both	4	8
`google/gemma-3-270m` [81]	270 M	640	0–17	9, 15	both	4	8
`EleutherAI/pythia-410m` [46]	410 M	1024	0–23	11, 23	`mlp_out`	2	4
`google/gemma-3-1b-pt` [81]	1 B	1152	0–25	12, 13, 22, 24	both	5	6
`meta-llama/Llama-3.2-1B` [5]	1 B	2048	0–15	7, 8, 11	`resid_post`	3	6
`meta-llama/Llama-3.2-1B-Instruct` [5]	1 B	2048	0–15	7, 9, 15	both	3	6
`deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B` [83]	1.5 B	1536	0–27	13, 27	`mlp_out`	2	4
`google/gemma-2b` [79]	2 B	2048	0–17	0, 6, 10, 17	`resid_post`	4	8
`google/gemma-2b-it` [79]	2 B	2048	0–17	12	`resid_post`	1	2
`google/gemma-2-2b` [80]	2 B	2304	0–25	1, 12, 19, 24	both	11	22
`meta-llama/Llama-3.1-8B-Instruct` [5]	8 B	4096	0–31	15, 27	`resid_post`	2	3
`meta-llama/Meta-Llama-3-8B-Instruct` [5]	8 B	4096	0–31	15, 30	`resid_post`	2	4
15 models						52	98

Table 2: The models, ordered by parameter count. *Params* carries each model's own size label. *Layers* gives the model's zero-based layer index range and *Intervened at* the indices the pipeline acted on. *Sites* names the within-layer locations used, *SAEs* the number of distinct SAEs applied to the model, and *Exp.* the number of completed experiments it contributes.

SAE inventory. The interventions are built on 52 pre-trained SAE checkpoints, published as 27 releases across 24 public repositories by eight providers. A *release* is one provider's set of SAEs trained under a shared recipe, so one repository may hold several. Appendix A groups the checkpoints by release and gives each one's architecture, repository identifier, resolved commit, width, layer, and site, so the exact weights behind any experiment can be fetched. All four SAE families of Section 2.2 are represented among the checkpoints, so the activation function that defines a family varies across the inventory, while the intervention of Section 3.1 reaches every SAE through the same encode-and-decode interface. Dictionary width runs from 14,336 to 131,072 features. Model width runs from 512 to 4,096. Each SAE's expansion factor, the number of dictionary features it carries per model dimension, runs from 7.0 to 85.3.

An opportunistic grid. Availability, rather than the steering method, is what bounds the design space. Three design axes organise the study, model scale, layer depth, and site. A comparison along any of them requires a public SAE to exist at the matching model, layer, and site, and no provider covers that space evenly. Every repository serves a single model, so breadth across models comes only from a provider publishing many repositories. EleutherAI covers eight of the fifteen models across nine repositories, reaching both sites on three, and Gemma Scope [28] covers three Gemma models with both sites at a shared layer. Six further providers contribute between one and three models apiece at one site only. The grid is therefore opportunistic, and three consequences follow. Coverage is sparse in both directions, since eight of the fifteen models carry one site only and none is represented at more than four layers. A comparison along layer or along site can hold the release and the width fixed. Along scale no release can be held fixed, since each serves one model, and the closest available control is one provider's suite with width matched where possible. No SAE used here was trained or independently validated by this work,

each being taken as published on its provider's own data and hyperparameters, so dictionary quality enters every result as a property of the material rather than as a controlled variable.

The site axis. It carries two levels, the residual-stream output `resid_post` and the MLP write `mlp_out`, and excludes the attention write that Section 2.1 names as the third architectural site. Within the public releases considered here, coverage is concentrated at the first two sites, and the available attention SAEs do not provide a comparable third level. Attention divides its work among several *attention heads*, whose separate outputs are concatenated and projected back to the width of the residual stream before being added to it. Gemma Scope trains its attention SAEs on those concatenated outputs before the projection, and for `google/gemma-3-270m` they are 1,024 wide where the residual stream is 640 [58]. Defined on $h \in \mathbb{R}^{d_{\text{model}}}$, the operator of Section 3.1 cannot be applied to these SAEs without being extended to the pre-projection space. Attention is therefore not treated as a third comparable site in this study.

4.1.3 Parameters and Compute Budget

Every setting Chapter 3 carried as a variable takes the value in Table 3. The order follows the phases, with the batch size and the seed after them since neither belongs to a single phase.

Setting	Value	
Phase 0: Difficulty screening		
Difficulty interval	$p_{\text{correct}} \in [p_{\min}, p_{\max}] = [0.15, 0.45]$	
In-interval split	discovery 50%, validation 25%, reserve 25%	
Complement split	selection 50%, holdout 50%	
Phase 1: Feature discovery		
Admission threshold	$n_{\min} = \max\left(3, \lfloor 0.01 \cdot n_{\text{discovery}} \rfloor\right)$	
Candidate shortlist	50 positive and 50 negative features	
Phase 2: Causal validation		
Test strength	$\alpha_{\text{test}} = 1.0$	
Phase 3: Configuration search		Options
Amplified features	$k_+ \in \{3, 5, 7, 10, 15\}$	5
Suppressed features	$k_- \in \{0, 3, 5\}$	$\times$ 3
Intervention strength	$\alpha \in \{0.5, 1.0, 1.5\}$	$\times$ 3
Ranking criterion	frequency difference or causal effect	$\times$ 2
Grid size	$\longrightarrow$	90
Phase 4: Holdout evaluation		
Significance level	0.05	
Evaluation folds	$K = 5$	
Confidence level	95%	
Across the run		
Batch size	1	
Random seed	42	

Table 3: Concrete values for every experiment setting.

Difficulty interval. Phase 0 (Section 3.2) asks one thing of the interval, that the questions inside it produce both outcomes, since feature discovery needs both successes and failures to contrast. Under the scoring convention of Section 2.4 that requirement takes an exact form. The difficulty score p_{correct} is the probability the model places on the correct answer letter, one of n letter probabilities that sum to one. Whichever letter holds the largest of those probabilities is the model's answer. With p_{correct} below $1/n$, the correct letter holds less than an equal share and is therefore outweighed by another option, so every such question is a failure. Above one half the correct letter leaves less than half to be shared among all the other options, so none can overtake it and every such question is a success. Both outcomes therefore occur only between the chance level $1/n$ and one half.

Where that range falls depends on the benchmark. Chance is 0.20 on the five options of CommonsenseQA and 0.25 on the four options of ARC-Challenge, so the range runs from 0.20 to 0.50 on the first and from 0.25 to 0.50 on the second. The two agree at the top and differ at the bottom, so one interval serving both would run from 0.20 to 0.50. What the experiments use instead is $[0.15, 0.45]$, each bound sitting 0.05 lower. The upper margin gives up questions the model is very likely to answer correctly, since at 0.45 a distractor must already hold more than 0.45 of the 0.55 that remains in order to win. Below chance the lower margin takes in a strip in which every question is a failure. Both margins point the same way, so the questions inside the interval lean towards errors an intervention can improve rather than successes it can only break. Successes remain available above chance, so the contrast keeps both of its sides. Covering five sixths of the CommonsenseQA range and four fifths of the ARC-Challenge one, the interval serves both benchmarks without adjustment.

Subset splits. After screening, Phase 0 assigns each later phase its own questions through two splits. Inside the interval, the discovery subset takes half, the validation subset a quarter and the reserve the remaining quarter. Discovery holds the largest share because it ranges over the whole dictionary, up to 131,072 features, while every later phase works only with features drawn from that shortlist. Due to the SAE's sparsity, only a small fraction of its features are active for a given question (Section 2.2), so the discovery subset yields relatively few observations per feature on average. A larger discovery subset makes the resulting frequency differences more reliable (Section 3.3). Validation, by contrast, works one feature at a time on that shortlist, the features whose activation already separates success from failure most sharply, so its subset is spent on testing each of them rather than on searching the dictionary. The reserve is held back as the only route by which in-interval questions reach the second split, so without it the holdout would have no in-interval part to report. That second split takes the reserve together with every question outside the interval and halves it into a selection subset and the holdout, so the configuration is chosen on as much data as it is judged on.

Admission threshold. Only this setting in Table 3 is expressed as a rule. Fixed in advance like the difficulty bounds, it requires a feature to be active on one percent of the discovery subset, rounded down, or on three questions, whichever is greater. Discovery subsets run from 36 questions to 5,467, a factor of 152, so the same fixed count would represent very different activation frequencies at the two ends. The floor covers the end where a share fails. One percent

of that smallest subset is 0.36, so the percentage alone would admit a feature active on a single question, and the sign of its score would then depend entirely on whether that question was answered correctly or incorrectly (Section 3.3). Across the study the threshold takes values from 3 to 54 with a median of 11, and the floor binds in seven of the 98 experiments. Those are the seven smallest discovery subsets, five of them on the two 8 B instruction-tuned models and both experiments on `google/gemma-2b-it`. Where the floor binds, a feature active on as few as 3 of 36 questions can enter the ranking, so the floor bounds that weakness rather than removing it.

Phase	Reference experiment	Share	All experiments	Share
Phase 0: Difficulty screening	10,962	2.2%	658,068	2.1%
Phase 1: Feature discovery	2,841	0.6%	184,243	0.6%
Phase 2: Causal validation	125,048	25.0%	7,427,970	23.8%
Phase 3: Configuration search	304,850	60.9%	17,370,717	55.7%
Phase 4: Holdout evaluation	21,522	4.3%	1,237,018	4.0%
Evaluation on the other three transfer benchmarks	35,580	7.1%	4,319,256	13.8%
Total	500,803		31,197,272	

Table 4: Forward-pass counts for the reference experiment presented in Section 4.2.1 and for all 98 experiments combined. One pass is one scored evaluation of one benchmark question, with or without steering.

Compute budget. Table 4 reports the distribution of forward-pass workload across the five pipeline phases and the transfer evaluation. Across the 98 experiments, causal validation and configuration search account for 79.5% of 31.2 million forward passes over benchmark questions, compared with 2.7% for difficulty screening and feature discovery combined. This concentration follows because Phase 2 (Section 3.4) tests every shortlisted feature at one strength on the validation subset, while Phase 3 (Section 3.5) evaluates all 90 grid configurations on the selection subset. The other evaluations scale only with question-set size. Transfer nevertheless accounts for 13.8%, 3.5 times the holdout workload, because each experiment scores the full question pool of each of its three transfer benchmarks both without and with the intervention.

Relative to the workload in Table 4, adding a fourth grid strength would increase the study-wide total by 18.4%, while testing each shortlisted feature at three strengths instead of one would add 47.0%. Doubling the number of candidates actually tested in every experiment would add 23.5%. The two Phase 2 changes interact multiplicatively, yielding six times as many feature-intervention passes. Including the fourth grid strength would raise the study-wide total from 31.2 million to 73.6 million forward passes, a factor of 2.4. The largest observed Phase 2 workload reached 276,134 passes in two `EleutherAI/pythia-70m` experiments on CommonsenseQA, where one baseline and 100 feature interventions each covered 2,734 validation questions.

Grid counts. The counts k_+ and k_- specify upper bounds on intervention size rather than guaranteed feature counts. If a_+ and a_- denote the numbers of positive and negative features available under the chosen ranking, the constructed vector contains $\min(k_+, a_+) + \min(k_-, a_-)$ features. A winning configuration can therefore contain fewer features than its grid label requests.

Evaluation folds. The experiments use $K = 5$ for the stability estimate defined in Phase 4 (Section 3.6). Because the folds partition rather than resample the holdout, K changes neither sample size nor forward-pass count. Five folds give the Student-t interval (Section 2.5) four degrees of freedom while leaving about one fifth of the holdout in each fold. Increasing K would raise the interval's degrees of freedom but shrink each fold to roughly $\frac{N}{K}$ questions, making every changed answer contribute approximately $\frac{K}{N}$ to the fold's accuracy difference.

4.1.4 Measurement Protocol

Each experiment ran on a single NVIDIA RTX A6000 or A100 GPU, with model weights loaded in `bfloat16`. Together, the 98 experiments consumed 631 GPU-hours, with a median runtime of 4.7 hours and a maximum of 20.1 hours.

Across all experiments, batch size is one and seed 42 determines the randomised split and fold assignments. Baseline and steered predictions for each question are produced within the same run, on the same device, and using the deterministic logit-scoring procedure specified in Section 3.2. The intervention is their only experimental difference, so a changed answer is attributable to it rather than to sampling noise. This attribution does not imply that separate executions on different hardware or software environments produce bit-identical outputs.

Holdout gain. All reported accuracy differences compare baseline and steered predictions on the same questions. The primary endpoint of Phase 4 (Section 3.6) carries one name throughout this chapter, the *holdout gain*. It is the accuracy of the exact intervention minus the unsteered baseline accuracy, both measured on the experiment's own full holdout dataset, and it is stated in percentage points. Every reported baseline is thus the unsteered accuracy on that same holdout. Transfer evaluation instead applies the approximated intervention of Section 3.1 at the same layer and site to each transfer benchmark's full question pool. Each transfer result compares this steered pass with an unsteered pass over the same questions and uses the same statistical test as the primary endpoint. Exact and approximated interventions are compared on the primary holdout, whereas transfer evaluates only the approximated form.

Shape of the holdout. Figure 7 shows that the fixed difficulty interval partitions the benchmark differently for each of the 30 model-and-benchmark pairs. Because it is applied to each model's own p_{correct} distribution (Section 3.2), the resulting coverage is model-relative and ranges from 2.7% to 99.7%. Across the 98 experiments, the median in-interval holdout contains 284 questions. For `google/gemma-2b-it` on ARC-Challenge, 3.1% of the benchmark lies inside the interval, and only 9 of those questions enter the holdout, the minimum across the study. Accuracy on that subset changes in increments of one question, or 11.1 percentage points. Because the in-interval holdout varies so widely in size, comparisons across experiments use the result of the exact intervention on the full holdout.

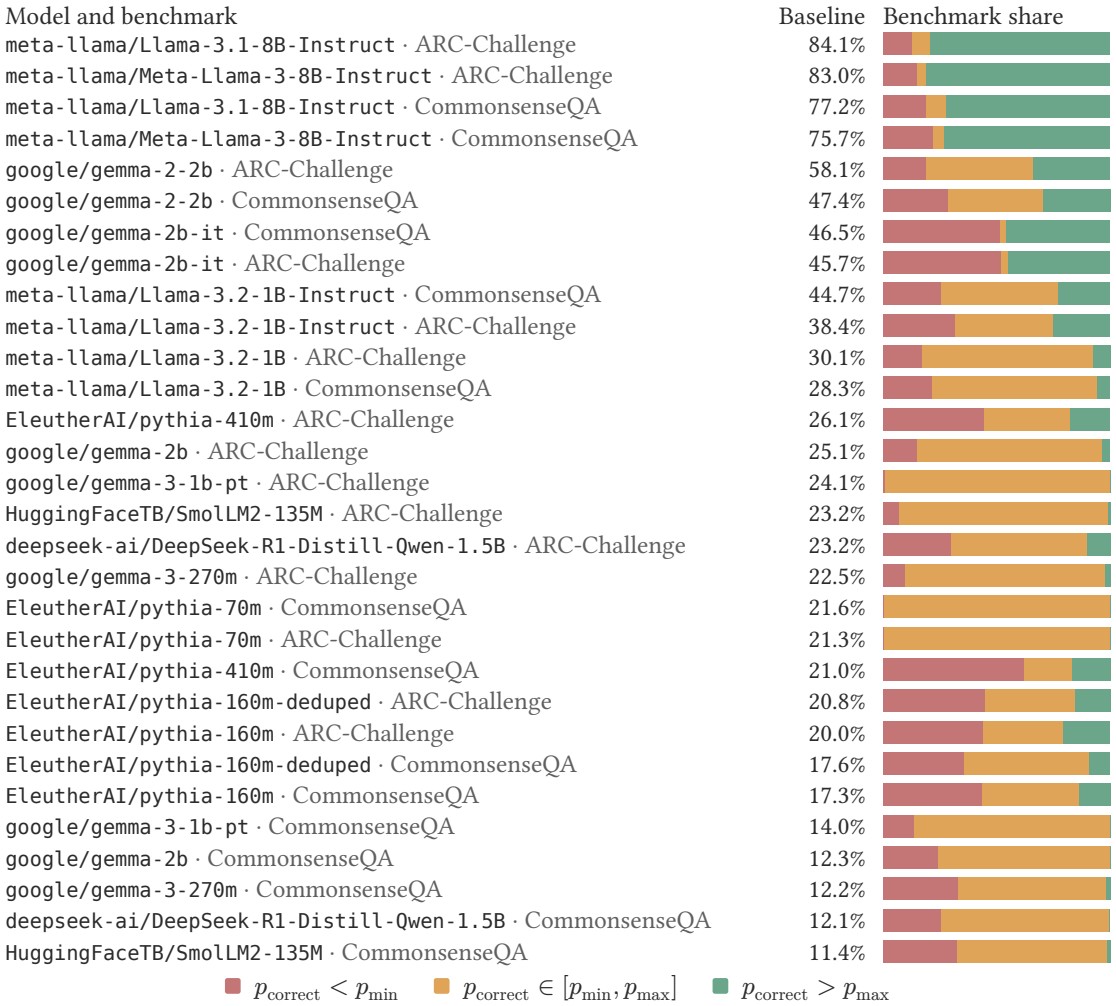


Figure 7: Difficulty composition of each model-and-benchmark pair, sorted by unsteered holdout accuracy. Each bar partitions the full benchmark pool by the screening score p_{correct}. Below the interval the model answers no question correctly, by construction, and above it almost every one. The baseline column reports unsteered holdout accuracy.

Interval coverage is not a restatement of baseline accuracy. On CommonsenseQA, `EleutherAI/pythia-410m` and `EleutherAI/pythia-70m` achieve unsteered holdout accuracies of 21.0% and 21.6%, yet their interval coverage is 20.8% and 99.7% of the full benchmark, respectively (Figure 7). Baseline accuracy is the share of holdout questions answered correctly under argmax scoring, whereas interval coverage is the share of benchmark questions whose p_{correct} lies within $[0.15, 0.45]$ (Section 3.2). Similar holdout accuracy can therefore coexist with markedly different p_{correct} distributions.

The 99.7% interval coverage of `EleutherAI/pythia-70m` largely reflects an almost constant difficulty score rather than graded variation across questions. Across all 10,962 CommonsenseQA questions, the recorded p_{correct} takes only eight different numerical values, and 10,919 questions receive the same value of 0.200 when rounded to three decimals. The 99.6% sharing that value therefore cannot be ordered by relative difficulty using this score (Section 3.2). The model also predicts A on every question, so its 19.6% full-benchmark accuracy equals the proportion of questions whose correct answer is A.

Reading the baselines. The preceding case illustrates a broader limitation of answer-letter scoring. As discussed in Section 2.4, a selected option can reflect a model's prior preference for particular answer-letter tokens rather than only its assessment of option content [61]. In 27 of the 98 experiments, one letter accounts for more than 90% of unsteered predictions. These comprise all 25 experiments on Pythia models and two of the four on `deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B`. Consequently, baseline accuracy provides only a limited measure of task performance in these experiments because it also reflects the dominant answer-letter preference.

The region below $p_{\min}$ creates an asymmetry in the paired comparison on the full holdout. It accounts for 0.9% to 73.8% of the holdout across experiments, with a median of 41.9%. Baseline accuracy in this region is 0% by construction. Questions from the region remain part of the full-holdout accuracy, but the paired significance test uses only discordant outcomes (Section 3.6). Unchanged errors are therefore excluded from the test, while every correction counts as an improvement and no worsening is possible. To determine whether this one-sided contribution changes the significance verdict, the paired test is recomputed for every experiment after excluding the region (Section 4.4.1).

High baseline accuracy on the holdout does not guarantee a balanced discovery subset. Across the 98 experiments, discovery-subset accuracy ranges from 2.7% to 40.0%, with a median of 26.5%. On CommonsenseQA, `meta-llama/Meta-Llama-3-8B-Instruct` has a holdout baseline of 75.7%, but only 15 of its 278 discovery questions are answered correctly. The smallest correct-answer group occurs for `google/gemma-2b-it` on ARC-Challenge, with only 3 correct answers among 36 discovery questions. Phase 1 (Section 3.3) divides each feature's activation count by the size of the corresponding outcome group, making the correct- and incorrect-answer frequencies comparable despite this imbalance. This normalisation does not compensate for the uncertainty of estimating a frequency from so few correct answers.

Baseline accuracy falls below chance for 14 of the 30 model-and-benchmark pairs, representing 43 of the 98 experiments. Because logit-based scoring can reflect a systematic answer-letter preference [61], an intervention may move a model toward chance by weakening that preference rather than by improving its assessment of option content. Such a change is still an improvement in measured benchmark accuracy, but it does not by itself establish improved task reasoning.

4.2 Results

Where Section 4.1 fixed the material and the conditions, this section reports what the pipeline produced. One reading convention holds throughout. Unless stated otherwise, every reported effect is the holdout gain defined in Section 4.1.4, the accuracy change of the exact intervention on the experiment's own full holdout dataset, in percentage points. Significance comes from the one-sided test of Phase 4 (Section 3.6) at level 0.05. An overview of all 98 experiments comes first. Section 4.2.1 then follows a single experiment end to end, and four further subsections survey the design axes of the first four refining questions, model scale (RQ1), layer depth (RQ2), intervention site (RQ3), and cross-benchmark transfer (RQ4). Whether the causal screen earns its cost (RQ5) is answered in Section 4.3, and the verdicts the axes support jointly are consolidated in Section 4.5.

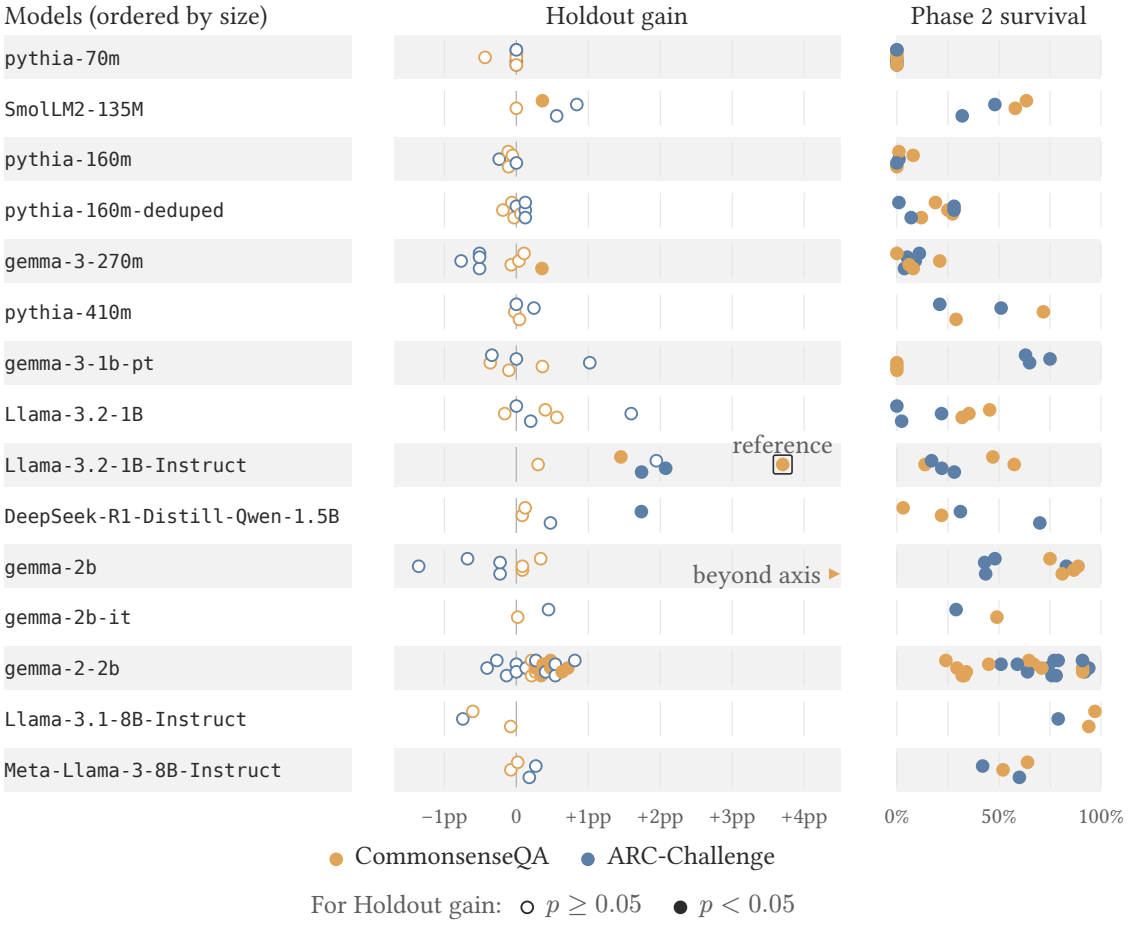


Figure 8: All 98 experiments, each contributing one dot to both panels, with the marker beyond the axis standing at +8.83pp. The right panel gives the share of candidates from Phase 1 (Section 3.3) that survived Phase 2 (Section 3.4). Dots are jittered vertically where experiments overlap.

Figure 8 places every experiment on a common scale. A more detailed view of these results is set out in Appendix C, one row per experiment, with layer, site, SAE release, holdout gain, and transfer results. Across the 98, the primary endpoint runs from -1.36 to $+8.83$ percentage points with a median of $+0.07$. Fifty-four experiments end above zero, 29 below, and 15 exactly at it. Eight of those zeros belong to interventions that changed no answer at all and seven to interventions whose improvements and worsenings cancelled exactly. In all eight the causal screen had validated no candidate. The pipeline-component analysis of Section 4.3 separates these outcomes systematically.

An entirely different fifteen reach significance. The 98 tests at level 0.05 would return 4.9 significant results on average even if no intervention anywhere helped, the multiplicity Section 2.5 warns of. Fifteen is three times that, and fifteen or more arises under that global null hypothesis with probability 1.1×10^{-4}. Asking instead which individual experiments can be called discoveries, Benjamini and Hochberg's false-discovery-rate control retains eight of the fifteen at 0.05 [66]. Neither number is a success rate, since SAE availability rather than sampling fixed which experiments exist (Section 4.1.2). What they establish is that chance does not account for the significant results this study contains.

Where the fifteen sit carries as much information as their count. Thirteen belong to models between 1 B and 2 B parameters. The remaining two, one experiment on `HuggingFaceTB/SmolLM2-135M` and one on `google/gemma-3-270m`, are among the smallest significant effects at $+0.36$ points each. Twelve of the fifteen have CommonsenseQA as their primary benchmark and three ARC-Challenge, out of 50 and 48 experiments overall. A CommonsenseQA holdout contains a median of 3,211 questions against 738 on ARC-Challenge, so the same true effect faces a harder test on the smaller benchmark. On a 771-question ARC-Challenge holdout $+1.95$ points misses significance at $p = 0.062$, while $+0.26$ points on a 3,774-question CommonsenseQA holdout passes at $p = 0.026$. Ten of the fifteen gains are also steady across the five evaluation folds of Phase 4, their 95% interval staying above zero. The remaining five are all `google/gemma-2-2b` experiments at $+0.72$ points or less.

A final convention concerns the eight experiments on `EleutherAI/pythia-70m`. Their difficulty screen returns an almost constant score, one value covering more than 99% of each benchmark (Section 4.1.4). That value is chance itself, 0.20 on CommonsenseQA and 0.25 on ARC-Challenge. Phase 0 (Section 3.2) had no ordering to split by, so the pipeline ran on subsets that difficulty did not stratify. Every result below counts all 98 and was recomputed over the 90 experiments whose screen produces a varying score. All eight sit at the smallest model in the study, so an aggregate over scale could inherit them rather than measure the axis. Both populations are given wherever the two differ in what they support.

4.2.1 A Reference Experiment

Before the axes are surveyed, one of the 98 experiments is followed end to end, so that the summaries that follow correspond to a concrete procedure whose intermediate products have been seen once in full. The experiment steers `meta-llama/Llama-3.2-1B-Instruct` on CommonsenseQA at the MLP write (`mlp_out`) of layer 7, just before the midpoint of its sixteen layers.

Its SAE, published as `EleutherAI/sae-Llama-3.2-1B-131k`, belongs to the TopK family. Of its 131,072 features, $k = 32$ are active at each position. The expansion factor is 64 over the model's 2,048 dimensions. That checkpoint was trained on the base model `meta-llama/Llama-3.2-1B` and is the only release the inventory holds for the instruction-tuned sibling at the MLP write, so the pipeline applies it off-distribution, a provenance condition Appendix A records. The run completed in 5.9 hours against the study median of 4.7, and Table 4 has already itemised its half-million forward passes. It serves as the reference for two reasons. Every phase leaves visible structure in its record, whereas in many experiments one or more phases pass through with little to show. Its result also stands up to the re-tests of Section 4.4, so the walk-through is not spent on an artefact.

Phase 0 (Section 3.2) scored all 10,962 questions once. The resulting difficulty distribution is graded, in contrast to the constant-score screening Section 4.1.4 reports: p_{correct} takes 681 distinct values, no single value covers more than 0.5% of the pool, and the middle 80% of questions span scores from 0.09 to 0.61. Applied to this distribution, the interval $[0.15, 0.45]$ selects 5,683 questions, 51.8% of the pool, leaving 25.3% below and 22.9% above. The first split assigns 2,841 in-interval questions to discovery, 1,421 to validation and 1,421 to the reserve. From the reserve and the out-of-interval remainder, the second split yields a selection subset and a holdout of 3,350 questions each, and 711 of the holdout's questions lie inside the interval. Unsteered accuracy stands at 44.7% on the full holdout and 34.5% on its in-interval part.

Phase 1 (Section 3.3) compared, for each of the 131,072 features, how often it was active on the 1,013 discovery questions the model answered correctly against the 1,828 it did not. At 35.7% correct, that subset is among the most balanced in the study. The admission threshold of Table 3 resolves to $n_{\min} = 28$, one percent of the discovery subset. That threshold rather than the fifty-per-side cap bounded the shortlist: 46 features cleared it with a positive frequency difference and 41 with a negative one, so 87 candidates went forward. Even at the top of the ranking the correlations are weak. The strongest positive candidate, feature 74778, is active on 36.2% of successes against 33.2% of failures, a difference of 3.0 percentage points. At the other end, the strongest negative, feature 32216, is active on 30.5% against 36.7%, a difference of 6.2 points. A shortlist this weakly separated is the situation the causal screen exists for.

Phase 2 (Section 3.4) measured each of the 87 candidates alone, at the test strength $\alpha_{\text{test}} = 1.0$, over all 1,421 validation questions, the quarter of the run's forward passes Table 4 attributes to it. Survival was asymmetric. Of the 46 amplification candidates, 14 improved validation accuracy. Among the 41 suppression candidates, 36 did. Averaged over every candidate tested, amplification moved accuracy by -3.0 points and suppression by $+0.7$. The two sides also differ at their best: the strongest single amplification, feature 40681, gains 8.0 points, three times the strongest suppression's 2.6 from feature 130481. Rejection therefore concentrates on the amplification side, yet it carries the largest single positive effects.

How little the rankings of the two phases agree is the observation that carries beyond this experiment. The three strongest correlates of success, features 74778, 58984 and 29608 at $+3.0$, $+2.6$ and $+2.5$ points of frequency difference, all fail causal validation, the second of them costing 2.4 points of validation accuracy when amplified. By contrast, the feature with the largest

causal benefit, 40681 at $+8.0$ points, stands only sixth in the correlational ranking, at $+1.7$ points of frequency difference. On the suppression side the best causal effect, the $+2.6$ points of feature 130481, belongs to a candidate whose frequency difference of -0.02 points is smaller than one question of either group would move it. The strongest correlate of failure, 32216 at -6.2 points, survives with a gain of only $+1.0$. Within this experiment, the correlational score that discovers the candidates predicts almost nothing about which of them help. That is the premise of Phase 2 observed in the data. It also gives substance to the choice Phase 3 leaves open, whether the final intervention is assembled by that score or by the measured causal effect, and the grid answers it.

Phase 3 (Section 3.5) evaluated all 90 configurations of Table 3 on the 3,350-question selection subset, whose unsteered baseline is 45.2%. Figure 9 lays out the surface. Twelve configurations beat the baseline, and they share one description: every one ranks by causal effect at strength $\alpha = 0.5$, filling all but three of that block's fifteen count combinations. At $\alpha = 1.0$ and 1.5 the same causally ranked features lose ground everywhere, the best such configuration finishing 0.3 points below the baseline, so combinations overshoot at the very strength at which their members were validated alone. The frequency-difference half never reaches the baseline at any strength or count, its best configuration sitting 0.5 points short. Its worst corner, fifteen amplified features at strength 1.5, falls to 21.9% accuracy, within two points of the 20% five-option chance level. A single setting, the ranking criterion, thus separates a half-grid containing every viable configuration from a half-grid containing none. The winning configuration amplifies the top 15 and suppresses the top 5 causally ranked features at $\alpha = 0.5$. Only 14 amplification survivors exist, so under the bound stated in Section 4.1.3 the intervention carries 19 features. It gains 2.27 points on the selection subset, 199 questions improved against 123 worsened.

k_+	k_-	Ranked by causal effect			Ranked by frequency difference		
		$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 1.5$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 1.5$
3	0	+1.4	−0.3	−6.4	−1.7	−4.8	−10.3
	3	+1.6	−0.9	−6.7	−0.9	−4.5	−10.2
	5	+1.9	−0.7	−5.8	−1.1	−4.3	−9.9
5	0	+1.3	−8.0	−15.3	−1.5	−3.0	−5.1
	3	+0.2	−8.9	−15.5	−0.5	−2.9	−5.4
	5	+0.7	−7.6	−13.6	−0.7	−2.5	−5.0
7	0	+0.5	−7.2	−15.6	−2.1	−8.7	−11.7
	3	−0.1	−7.3	−15.4	−1.1	−7.7	−11.1
	5	+0.3	−6.4	−15.0	−1.0	−7.6	−10.6
10	0	−0.1	−7.3	−14.8	−6.8	−21.8	−22.3
	3	−0.4	−8.2	−14.8	−4.5	−20.8	−21.9
	5	+0.1	−6.6	−15.3	−4.5	−20.7	−22.0
15	0	+1.5	−8.2	−19.7	−20.1	−23.3	−23.3
	3	+1.3	−8.4	−19.9	−18.7	−23.3	−23.3
	5	**+2.3**	−6.7	−20.1	−18.8	−23.3	−23.3

Figure 9: The 90 configurations of Phase 3. Each entry is the change in selection-subset accuracy, in percentage points against the unsteered baseline.

Phase 4 (Section 3.6) carried that intervention to the holdout. Table 5 reports the four readings that phase defines. On the primary endpoint, the full holdout under the exact intervention, accuracy rises from 44.7% to 48.4%, a gain of 3.70 percentage points. It is built from 233 improved questions against 109 worsened, and the one-sided test on those discordant counts gives $p = 8.8 \times 10^{-12}$. The five evaluation folds put the same estimate at a mean of $+3.70$ points with a fold standard deviation of 1.26 and a 95% interval of $[+2.13, +5.27]$, every fold positive, the smallest at $+2.69$ and the largest at $+5.82$. On the in-interval part the gain is 15.47 points from the 34.5% baseline, the concentration expected where the pipeline aimed its discovery and validation.

Evaluation	Questions	Baseline	Steered	Gain	Flips	p
Holdout, exact intervention						
Full holdout	3,350	44.7%	48.4%	+3.70	$+233 / -109$	8.8×10^{-12}
In-interval part	711	34.5%	49.9%	+15.47	$+188 / -78$	6.0×10^{-12}
Holdout, approximated intervention						
Full holdout	3,350	44.7%	48.4%	+3.67	$+308 / -185$	1.7×10^{-8}
In-interval part	711	34.5%	52.5%	+18.00	$+218 / -90$	1.0×10^{-13}
Transfer pools, approximated intervention						
ARC-Challenge	2,291	29.7%	41.8%	+12.13	$+377 / -99$	1.5×10^{-39}
OpenBookQA	5,457	35.6%	44.6%	+8.94	$+703 / -215$	2.0×10^{-61}
HellaSwag	10,042	25.7%	27.7%	+2.03	$+399 / -195$	2.0×10^{-17}

Table 5: Holdout readings of the reference experiment and transfer to the three benchmarks that played no part in its phases. *Gain* is in percentage points, *Flips* counts the questions improved and worsened, and p is the one-sided binomial of Phase 4.

The near-coincidence of the exact and the approximated form on the full holdout, $+3.70$ against $+3.67$ points, is a balance rather than an agreement. In both directions the approximated pass flips more questions, 308 improved and 185 worsened against the exact form's 233 and 109. On the in-interval part it finishes 2.53 points ahead, $+18.00$ against $+15.47$. Question-level behaviour thus differs, though not systematically in the exact form's favour. Section 3.1 left the price of removing the SAE from the forward pass to be measured. Here the price is negligible on the full holdout, with one net answer in 3,350 questions. Inside the interval it is a gain instead.

Table 6 decomposes the primary gain by the difficulty interval. Below $p_{\min}$, where the baseline is 0% by construction, the intervention answers 44 of 1,386 questions correctly, a 3.17-point rise within the region that contributes $+1.31$ of the $+3.70$ points. The in-interval region contributes $+3.28$ points from a net 110 flips, and the region above the interval, starting from a 99.9% baseline, takes back 0.90 points with 31 questions broken against one gained. Earned inside and below the interval, the gain loses a fifth of itself above it. Re-testing with the below-interval region excluded, where no worsening is possible, as Section 4.1.4 announced for every experiment, leaves 189 improvements against the same 109 worsenings and a verdict of $p = 2.1 \times 10^{-6}$, so the significance does not rest on that region.

Region of the holdout	Questions	Baseline	Steered	Gain	Flips	Net	Contribution
Below the interval	1,386	0.0%	3.2%	+3.17	$+44 / -0$	+44	+1.31
Inside the interval	711	34.5%	49.9%	+15.47	$+188 / -78$	+110	+3.28
Above the interval	1,253	99.9%	97.5%	−2.39	$+1 / -31$	−30	−0.90
Full holdout	3,350	44.7%	48.4%	+3.70	$+233 / -109$	+124	+3.70

Table 6: The primary gain of the reference experiment, decomposed by each question's screening score relative to $[0.15, 0.45]$. *Net* is improvements minus worsenings, and *Contribution* the region's share of the full-holdout gain in percentage points.

The transfer evaluation of Section 4.1.4 applied the approximated intervention, unchanged, to the full pools of the three benchmarks that played no part in this experiment's phases. All three move upward, none with p above 10^{-16}: ARC-Challenge by 12.13 points from a 29.7% baseline, OpenBookQA by 8.94 points from 35.6%, HellaSwag by 2.03 points from 25.7% (Table 5). An intervention assembled from 19 features chosen on CommonsenseQA therefore improves three benchmarks it never saw, two of them by more than it improves its own holdout, though all three baselines lie 9 to 19 points below that holdout's 44.7%. What transfer looks like across the study is examined in Section 4.2.5.

Taken together, the record shows every phase doing observable work. Screening produced a graded difficulty distribution and a discovery subset with both outcome groups populated. From 131,072 features, the discovery scan distilled 87 weakly separated candidates. Causal validation rejected 32 of the 46 amplification candidates, among them the three strongest correlates, and found the intervention's strongest member sixth in the correlational ranking. All twelve viable combinations ranked by causal effect at a single strength. Without the causal screen the configuration search would have had nothing above baseline to select. Finally, the holdout confirmed the selected configuration at $p = 8.8 \times 10^{-12}$, and the intervention carried to three untouched benchmarks. One caution frames what follows: at $+3.70$ points this is the second-largest primary effect among the 98 experiments. That is the pipeline at its best. The four subsections that follow establish where in the design space that behaviour was and was not found.

4.2.2 Model Scale

The first refining question asks whether the effectiveness of the method depends on model scale, across the 70 M to 8 B range that Table 2 spans. Its answer arrives in two parts. One concerns the primary endpoint, the other the causal screen. Significance reaches 2 of the 37 experiments on models at 410 M and below, 13 of 54 from 1 B to 2 B, and none of the 7 at 8 B. Those boundaries are the two widest gaps in the size ladder, 2.4-fold from 410 M to 1 B and 4-fold from 2 B to 8 B. Each model contributes one median gain, and across all fifteen those gains show no ordering by parameter count, the Spearman rank correlation [84] standing at -0.02. The left panel of Figure 8 resolves the counts per model, with every gain beyond one point produced between 1 B and 2 B. Eleven of the thirteen significant results in that range belong to `google/gemma-2-2b` and `meta-llama/Llama-3.2-1B-Instruct`, a concentration the measurement analysis of Section 4.4 returns to.

Neither end supports a conclusion about scale itself, because the measurement changes along with it. At the small end, the eight `EleutherAI/pythia-70m` experiments carry the degenerate screen just described, 25 of the 37 experiments in the lowest group run on Pythia models whose predictions Section 4.1.4 showed to be dominated by a single answer letter, and in 11 of the 37 the causal screen validated not a single candidate. The difficulty interval collapses at the large end instead. Figure 7 showed the two 8 B models leaving 4% to 9% of each benchmark inside the interval, and five of the seven smallest discovery subsets belong to their experiments (Section 4.1.3). The seven experiments yield no significant result, every one of their gains falling between -0.74 and $+0.27$ points. Whether that absence reflects a method that stops working or a measurement that loses power is a question Section 4.4.2 returns to, and the groups alone cannot settle it.

Provider control does not change the endpoint picture. Table 7 applies the scale control Section 4.1.2 identified as the closest the public inventory allows, each ladder holding one SAE provider fixed across models of different size. The Gemma Scope ladder spans eight experiments on `google/gemma-3-270m`, three on `google/gemma-3-1b-pt`, and eight on `google/gemma-2-2b`, all at a dictionary width of 16,384. Two releases meet inside it, Gemma Scope at 2 B and its successor below, so the largest rung carries the older model generation (Section 2.2). Significance appears once at 270 M and twice at 2 B, never at 1 B, and the three significant gains all lie between $+0.34$ and $+0.48$ points. Its largest gain, $+1.02$ points at 1 B, misses significance. The Pythia ladder spans four models from 70 M to 410 M at two dictionary widths, 32,768 and 65,536. None of its 25 experiments reaches significance. Endpoint accuracy is thus not ordered by scale even where the provider is held constant.

Scale orders one quantity, the survival rate of the causal screen shown in the right panel of Figure 8. Median survival climbs from 8.0% on the models at 410 M and below to 48.5% between 1 B and 2 B and 64.0% at 8 B. Ranked by size and ranked by median survival, the fifteen models fall in nearly the same order. A rank correlation puts that agreement at $+0.730$, where $+1$ would be a perfect match, with $p = 0.002$. The same rise appears where provider and dictionary width are fixed, from 7.0% at 270 M to 61.0% at 2 B along the Gemma Scope ladder. Two exceptions mark it as a tendency rather than a rule. That ladder's three 1 B experiments split 0%, 0% and 65%. The two 160 M Pythia models sit at 1.0% and 21.9%, a gap at equal size as wide as the one from there to 40.0% at 410 M.

Small models are also the ones the difficulty screen measures worst, so the ordering could belong to the measurement rather than to scale. Five candidates were tested, the size of the validation subset, the dictionary's width and its expansion factor, and the two weaknesses the small end carries (Section 4.1.4), a screen that returns few distinct difficulty scores and a dominant answer letter. Comparing only experiments alike in one of the five, a partial rank correlation leaves the ordering in place, at $+0.502$ in the weakest case. The reverse comparison is more telling. With model size held fixed, the two strongest candidates fall from $+0.604$ and -0.576 to $+0.165$ and -0.084. Those weaknesses therefore accompany scale rather than account for the ordering.

Model	Layer	Site	Benchmark	Holdout gain	p	Phase 2 survival
Gemma Scope, dictionary width 16,384						
gemma-3-270m	9	mlp_out	ARC	−0.51	0.891	11%
gemma-3-270m	9	mlp_out	CSQA	+0.04	0.500	21%
gemma-3-270m	9	resid_post	ARC	−0.51	1.000	4%
gemma-3-270m	9	resid_post	CSQA	**+0.36**	0.006	8%
gemma-3-270m	15	mlp_out	ARC	−0.51	0.938	5%
gemma-3-270m	15	mlp_out	CSQA	+0.11	0.227	0%
gemma-3-270m	15	resid_post	ARC	−0.77	1.000	9%
gemma-3-270m	15	resid_post	CSQA	−0.07	0.891	6%
gemma-3-1b-pt	13	mlp_out	ARC	+1.02	0.188	65%
gemma-3-1b-pt	13	resid_post	CSQA	+0.36	0.155	0%
gemma-3-1b-pt	22	resid_post	CSQA	−0.36	0.939	0%
gemma-2-2b	12	mlp_out	ARC	−0.14	0.773	76%
gemma-2-2b	12	mlp_out	CSQA	+0.21	0.092	24%
gemma-2-2b	12	resid_post	ARC	+0.14	0.500	74%
gemma-2-2b	12	resid_post	CSQA	**+0.34**	0.024	33%
gemma-2-2b	19	mlp_out	ARC	−0.27	0.855	77%
gemma-2-2b	19	mlp_out	CSQA	+0.21	0.067	32%
gemma-2-2b	19	resid_post	ARC	+0.00	0.688	51%
gemma-2-2b	19	resid_post	CSQA	**+0.48**	0.003	71%
EleutherAI TopK on Pythia, dictionary width 32,768						
pythia-70m	2	mlp_out	ARC	+0.00	1.000	0%
pythia-70m	2	mlp_out	CSQA	−0.43	0.885	0%
pythia-70m	2	resid_post	ARC	+0.00	1.000	0%
pythia-70m	2	resid_post	CSQA	+0.00	1.000	0%
pythia-70m	5	mlp_out	ARC	+0.00	1.000	0%
pythia-70m	5	mlp_out	CSQA	+0.00	1.000	0%
pythia-70m	5	resid_post	ARC	+0.00	1.000	0%
pythia-70m	5	resid_post	CSQA	+0.00	1.000	0%
pythia-160m	5	resid_post	ARC	−0.24	0.938	1%
pythia-160m	5	resid_post	CSQA	−0.05	0.806	8%
pythia-160m	6	mlp_out	CSQA	−0.11	1.000	1%
pythia-160m	11	resid_post	ARC	+0.00	1.000	0%
pythia-160m	11	resid_post	CSQA	−0.11	0.717	0%
EleutherAI TopK on Pythia, dictionary width 65,536						
pythia-160m-deduped	5	mlp_out	ARC	+0.12	0.500	28%
pythia-160m-deduped	5	mlp_out	CSQA	−0.06	0.806	19%
pythia-160m-deduped	5	resid_post	ARC	+0.00	0.750	28%
pythia-160m-deduped	5	resid_post	CSQA	+0.06	0.430	27%
pythia-160m-deduped	11	mlp_out	ARC	+0.12	0.500	1%
pythia-160m-deduped	11	mlp_out	CSQA	−0.03	0.655	12%
pythia-160m-deduped	11	resid_post	ARC	+0.12	0.500	7%
pythia-160m-deduped	11	resid_post	CSQA	−0.19	0.830	25%
pythia-410m	11	mlp_out	ARC	+0.24	0.250	51%
pythia-410m	11	mlp_out	CSQA	−0.02	0.773	72%
pythia-410m	23	mlp_out	ARC	+0.00	0.750	21%
pythia-410m	23	mlp_out	CSQA	+0.04	0.250	29%

ARC = ARC-Challenge CSQA = CommonsenseQA

Table 7: The two provider-controlled scale ladders, the lower one split by dictionary width. Each heading fixes one SAE provider and one width, so the expansion factor falls as the models grow. *Holdout gain* is the primary endpoint in percentage points, printed bold where its one-sided p falls below 0.05. *Phase 2 survival* repeats the right panel of Figure 8 for these experiments.

Read together, the two parts answer the scale question asymmetrically. Scale does not order the endpoint, and the significant results cluster between 1 B and 2 B, where power and interval coverage are both adequate. It does order how often a shortlisted feature survives the causal screen. That suggests larger models, or the SAEs trained on their activations, expose more decision-relevant structure for the screen to find, though the design cannot separate the model's contribution from the SAE's. The two parts do not join up. No experiment whose screen validated nothing reached significance, yet across the fifteen models survival and the holdout gain agree at $+0.014$, as little as the gain and model size do. At 8 B the highest group median, 64.0%, coexists with no significant result at all. A validated feature must still survive assembly, selection, and a holdout whose difficulty interval has largely collapsed. Section 4.5 weighs these two parts against each other.

4.2.3 Layer Depth

The second refining question asks whether the depth of the intervention layer influences the outcome accuracy. Thirty-nine controlled contrasts answer it. Each holds the model, site, benchmark, and SAE release fixed and takes its checkpoints at two depths, so the endpoints differ in the layer and in the checkpoint trained there. Figure 10 lays out all 39. Between a contrast's two endpoints the median gap is 0.20 points, 30 of the 39 stay within half a point, and three exceed one point. The shallower layer yields the larger gain in 19 contrasts, the deeper in 14, and 6 tie, a split a two-sided sign test [85] places at chance ($p = 0.487$). No depth preference is detectable on the primary endpoint.

The largest of the three lies at the residual-stream output of `google/gemma-2b` on CommonsenseQA, where $+0.34$ at layer 10 stands against $+8.83$ at layer 17, a gap of 8.50 points. That deeper endpoint is the study's largest single effect, whose validity Section 4.4.1 examines separately. The other two favour the shallower layer, 2.25 points for `meta-llama/Llama-3.2-1B-Instruct` on CommonsenseQA and 1.26 points for `deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B` on ARC-Challenge, both at the MLP write. Neither gap carries to the other benchmark its release covers. The Llama contrast prefers layer 7 to layer 15 by $+3.70$ against $+1.45$, and reverses on ARC-Challenge, where layer 15 leads by $+2.08$ against $+1.95$. On DeepSeek the sign holds and the size does not, the same contrast separating its layers by 0.04 points on CommonsenseQA.

The population bounds the claim. Every contrast compares exactly two depths, so the design can detect a direction but cannot trace a curve. Shallower endpoints fall between 0.40 and 0.59 of the stack, so no contrast tests a layer nearer the input than that. In 37 of the 39 contrasts the deeper endpoint falls between 0.76 and 1.00, and in 18 it is the model's final layer. The other two come from a `meta-llama/Llama-3.2-1B` release whose SAEs stop at layer 8 of a sixteen-layer model. Both its depths sit near that cap, the study's only adjacent pair against a median separation of 7 layers. Nor can depth be separated from the SAE trained at it, since a contrast changes both. What the 39 contrasts support is a negative finding with practical content. Gaps large enough to matter do not survive the move to another benchmark, so intervention depth remains a quantity to settle empirically per configuration rather than by rule.



Figure 10: The 39 controlled layer contrasts. Each row holds one model, site, benchmark, and SAE release fixed, and its two dots give the primary endpoint at the release's two layers, in percentage points on the full holdout. Rows are sorted by the difference between them. The marker beyond the axis stands at +8.83. Bracketed tags appear only where several releases share a model, site, and benchmark, at the residual-stream output of `google/gemma-2-2b` (Table 12). One is Gemma Scope [28]. The others are two training runs of a single Chanind Matryoshka release, standard and snap loss. Snap loss is a reconstruction loss aimed at feature hedging [86], which the release reports makes no practical difference [87].

4.2.4 Intervention Site

The third refining question asks whether the site of the intervention influences the outcome accuracy. Answering it fairly requires the same SAE release family at both sites of the same model, layer, and benchmark, a condition the completed experiments meet sixteen times across four models. Table 8 lists the pairs. Within each, the two SAEs match in dictionary width as well, so the comparison is as controlled as Section 4.1.2 found the inventory to allow. Every pair stays under half a point of separation, the largest gap standing at 0.43 points and the median at 0.14. The residual-stream side yields the larger gain in 7 pairs, the MLP side in 4, and 5 tie, a split a two-sided sign test [85] places at chance ($p = 0.549$). No site preference is detectable on the primary endpoint. The sixteen pairs come from only four models, four apiece, so they carry less independent evidence than their count suggests.

Model	Layer	Benchmark	`resid_post`	`mlp_out`	Difference
Gemma Scope, dictionary width 16,384					
`gemma-3-270m`	9	ARC	−0.51	−0.51	+0.00
`gemma-3-270m`	9	CSQA	**+0.36**	+0.04	+0.32
`gemma-3-270m`	15	ARC	−0.77	−0.51	−0.26
`gemma-3-270m`	15	CSQA	−0.07	+0.11	−0.18
`gemma-2-2b`	12	ARC	+0.14	−0.14	+0.27
`gemma-2-2b`	12	CSQA	**+0.34**	+0.21	+0.13
`gemma-2-2b`	19	ARC	+0.00	−0.27	+0.27
`gemma-2-2b`	19	CSQA	**+0.48**	+0.21	+0.26
EleutherAI TopK on Pythia, dictionary width 32,768					
`pythia-70m`	2	ARC	+0.00	+0.00	+0.00
`pythia-70m`	2	CSQA	+0.00	−0.43	+0.43
`pythia-70m`	5	ARC	+0.00	+0.00	+0.00
`pythia-70m`	5	CSQA	+0.00	+0.00	+0.00
EleutherAI TopK on Pythia, dictionary width 65,536					
`pythia-160m-deduped`	5	ARC	+0.00	+0.12	−0.12
`pythia-160m-deduped`	5	CSQA	+0.06	−0.06	+0.12
`pythia-160m-deduped`	11	ARC	+0.12	+0.12	+0.00
`pythia-160m-deduped`	11	CSQA	−0.19	−0.03	−0.16

ARC = ARC-Challenge CSQA = CommonsenseQA

Table 8: The 16 release-matched site pairs. Every row holds one model, layer, benchmark, and release family fixed, giving the primary endpoint at each site in percentage points on the full holdout. The group headings carry the provider and the dictionary width. Bold marks endpoints whose one-sided p falls below 0.05. *Difference* is `resid_post` minus `mlp_out`.

Three pairs contain a significant side, and in all three it is the residual stream on CommonsenseQA, at layer 9 of `google/gemma-3-270m` and at layers 12 and 19 of `google/gemma-2-2b`. That all three fall on CommonsenseQA comes down to power. Its holdouts run about four times the size of ARC-Challenge ones, so significance concentrates there throughout the study. Each MLP partner is positive too, and two of them miss the threshold only narrowly, at $p = 0.092$ and $p = 0.067$. A result this one-sided could mark a real residual-stream advantage or nothing more than noise, and three pairs from two models cannot settle which.

What limits this finding is the pairs themselves, not the sign test. Four of the pairs sit on `EleutherAI/pythia-70m`, whose difficulty screen returns a constant score. In three of them neither side changed an answer, and the fourth carries the largest gap in the table. Dropping all four tightens the bound to 0.32 points and leaves the split at 6 to 4 ($p = 0.754$). No release family spans both sites on the models producing the study's largest effects, since the public SAEs for `meta-llama/Llama-3.2-1B-Instruct`, `deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B`, and `google/gemma-2b` cover one site only or disjoint layers. The site contrast is therefore measured exclusively where effects are fractions of a point. Section 2.1 separates the two sites by what an edit acts on rather than by how far it reaches. Within this region the choice between them does not show in the measured gain, so the site can follow whichever SAE is available.

4.2.5 Cross-Benchmark Transfer

The fourth refining question asks whether an intervention selected on one benchmark transfers to benchmarks that played no part in selecting it. Under the protocol of Section 4.1.4 each experiment contributes three such measurements, the approximated intervention scored on the full pool of every benchmark it never saw, giving 294 in total. Most of them are small. The median is $+0.04$ points, and 228 of the 294 stay within half a point of zero. Their extremes run from -4.68 to $+12.13$ points, and the reference experiment produced the upper one (Section 4.2.1). Sixty-eight measurements reach significance, and every one of them is a gain. The one-sided test of Phase 4 (Section 3.6) asks whether an intervention corrected more answers than it broke, so it can only ever report a gain. Read in the opposite direction at the same level, it marks 16 measurements as losses.

Transfer concentrates among the interventions that pass their own holdout. Eleven of the fifteen significant experiments carry a significant transfer as well, against 22 of the 83 that are not, an association a one-sided Fisher exact test [88] puts at $p = 8.1 \times 10^{-4}$. All eleven belong to `google/gemma-2-2b` and `meta-llama/Llama-3.2-1B-Instruct`, so the association rests on two models, though dropping either alone leaves it standing at $p = 0.034$ and $p = 0.013$. On the 90 varying-score experiments it holds at $p = 1.9 \times 10^{-3}$. The median significant experiment averages $+1.11$ points over its three transfers, against $+0.00$ for the rest. Passing the holdout is nonetheless no requirement for transfer. Of the 33 experiments carrying a significant transfer, 22 have no significant holdout result of their own, and eight of those lose accuracy on it.

Table 9 resolves the fifteen significant experiments measurement by measurement, and the pattern is bimodal. All four `meta-llama/Llama-3.2-1B-Instruct` interventions transfer significantly on every one of their twelve measurements, with gains from $+0.56$ to $+12.13$ points. The seven `google/gemma-2-2b` interventions improve ARC-Challenge and OpenBookQA in all fourteen measurements, every one significant, whereas HellaSwag, the transfer target furthest from the primary benchmarks in task and format (Section 4.1.1), moves by at most $+0.59$ points and reaches significance in three of the seven.

Model	Layer	Site	Bench.	Holdout gain	CSQA	ARC	OBQA	HS
gemma-2b	17	resid_post	CSQA	**+8.83**	–	−0.26	−0.97	−0.88
Llama-3.2-1B-Instruct	7	mlp_out	CSQA	**+3.70**	–	**+12.13**	**+8.94**	**+2.03**
Llama-3.2-1B-Instruct	15	mlp_out	ARC	**+2.08**	**+3.95**	–	**+6.43**	**+1.16**
Llama-3.2-1B-Instruct	9	resid_post	ARC	**+1.74**	**+1.87**	–	**+6.41**	**+0.56**
DeepSeek-R1-Distill-Qwen-1.5B	13	mlp_out	ARC	**+1.74**	+0.10	–	−0.13	−0.33
Llama-3.2-1B-Instruct	15	mlp_out	CSQA	**+1.45**	–	**+7.51**	**+7.86**	**+1.84**
gemma-2-2b	1	resid_post	CSQA	**+0.72**	–	**+2.10**	**+1.32**	+0.11
gemma-2-2b [release: snap loss]	12	resid_post	CSQA	**+0.64**	–	**+1.96**	**+1.59**	−0.07
gemma-2-2b [release: gemma scope]	19	resid_post	CSQA	**+0.48**	–	**+1.53**	**+1.45**	**+0.36**
gemma-2-2b [release: seonglae]	19	resid_post	CSQA	**+0.48**	–	**+1.70**	**+1.47**	**+0.59**
gemma-2-2b [release: standard]	12	resid_post	CSQA	**+0.37**	–	**+1.31**	**+1.08**	+0.05
SmolLM2-135M	15	mlp_out	CSQA	**+0.36**	–	+0.17	+0.20	−0.01
gemma-3-270m	9	resid_post	CSQA	**+0.36**	–	−0.13	−0.11	+0.01
gemma-2-2b [release: gemma scope]	12	resid_post	CSQA	**+0.34**	–	**+1.05**	**+0.95**	+0.16
gemma-2-2b	24	resid_post	CSQA	**+0.26**	–	**+0.65**	**+0.92**	**+0.19**

ARC = ARC-Challenge CSQA = CommonsenseQA HS = HellaSwag OBQA = OpenBookQA

Table 9: The 15 experiments whose holdout gain is significant, sorted by that gain. *Holdout gain* is the primary endpoint, the accuracy change of the exact intervention on the experiment's own full holdout dataset. The four right columns score the approximated intervention on the full question pool of each benchmark that played no part in building it (Section 4.1.4), and a dash marks the experiment's own primary benchmark. Gains are in percentage points, bold where the one-sided p falls below 0.05. Bracketed tags separate rows differing only in the SAE release (Table 12).

The remaining four experiments do not transfer. Those on `HuggingFaceTB/SmolLM2-135M`, `google/gemma-3-270m`, and `deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B` gain no more than $+0.20$ points on any transfer benchmark, none of them significantly. The fourth, at layer 17 of `google/gemma-2b`, is the largest primary effect in the study at $+8.83$ points and moves all three of its transfer benchmarks downward. Its approximated form gains 8.79 of them at home, so the transfer vector is not the weaker one. An intervention gaining that much at home and nothing anywhere else invites the suspicion that its home measurement rewards something other than task ability, a suspicion Section 4.4.1 substantiates.

Losses are rarer and smaller than gains, though they appear on nine models where gains appear on six. Four of the 16 losses cost more than a point, and 33 of the 68 gains bring more than a point. All four experiments behind them move their other two targets upward, two of them significantly. The largest loss belongs to the reference experiment's counterpart, identical in model, layer, site and SAE release, and built on ARC-Challenge instead. It gains 1.95 points on its own ARC-Challenge holdout, short of significance. On OpenBookQA it then gains 9.73 points and on HellaSwag 3.10, the largest transfers this study records on either benchmark. CommonsenseQA is the only one of its three targets that falls, by 4.68 points. The reference intervention runs the same pair the other way round and gains 12.13 points on ARC-Challenge. A loss therefore marks one target rather than an intervention that failed.

Two readings of the transfer evidence have to be separated. Ten of the twelve Llama measurements land on benchmarks the model handles poorly, at baselines from 25.7% to 36.3% against 25% chance, so they show a weak model lifted rather than a competent one improved. A lower

baseline does not mean a larger gain. The four lowest of them gain between $+0.56$ and $+2.03$ points, while the largest gains come from the ARC-Challenge and OpenBookQA measurements higher up the range. Only two of the 68 significant transfers start below the chance level, and both lie within half a point of it, so the below-chance hazard of Section 4.1.4 does not account for them either. The two CommonsenseQA transfers among the Llama measurements weigh most, gaining $+3.95$ and $+1.87$ points from baselines of 41.2% and 42.2% against 20% chance, far from any floor effect. Taken together, the evidence suggests that an intervention which survives its own holdout tends to carry a signal that is not specific to the benchmark it was built on. Whether that signal is task ability or a format-level preference the benchmarks share is the question the next two sections take apart, first by pipeline component (Section 4.3) and then by measurement (Section 4.4).

4.3 Contribution of the Pipeline Components

Section 4.2 varied models, layers, sites, and benchmarks while the pipeline itself stayed fixed. What this section examines is whether the pipeline's causal screen earns its keep. That screen, Phase 2 (Section 3.4), consumes 23.8% of the study's forward passes (Table 4), and a pipeline that ranked candidate features by their correlation with success alone would not spend them. The fifth refining question asks whether that spending changes what the pipeline delivers. Nothing is switched off to answer it, since every reading this section compares is one that each experiment already records. Phase 3 (Section 3.5) carries the question as a setting, its ranking criterion deciding whether the winning configuration keeps to the survivors of the causal screen or reaches past them to candidates admitted on correlation alone. Three further measurements come from every experiment. The first is how much of a winning gain survives the move from the selection subset to the holdout, reported before the others because every later comparison is read through the difference it reveals. How many shortlisted features the causal screen keeps is the second, together with what the search does where it keeps none. The third is how far the gain moves when the approximated intervention drops the SAE from the forward pass.

Phase 3 selects by the largest gain on the selection subset, a subset matched to the holdout in size and difficulty (Section 3.2), and part of any winning gain is specific to the very questions it was selected on. Across the study that part is measurable. The median selection gain is $+0.40$ percentage points across the 98 experiments, against $+0.07$ on the primary endpoint of Section 4.1.4. Experiment by experiment the median shrinkage is 0.21 points, the holdout falling below the selection reading in 70 of them, matching it exactly in seven, and exceeding it in 21. A Wilcoxon signed-rank test [89] on those within-experiment differences puts the imbalance at $p = 2.4 \times 10^{-8}$. This shrinkage from selection to holdout is referred to below as *optimism*.

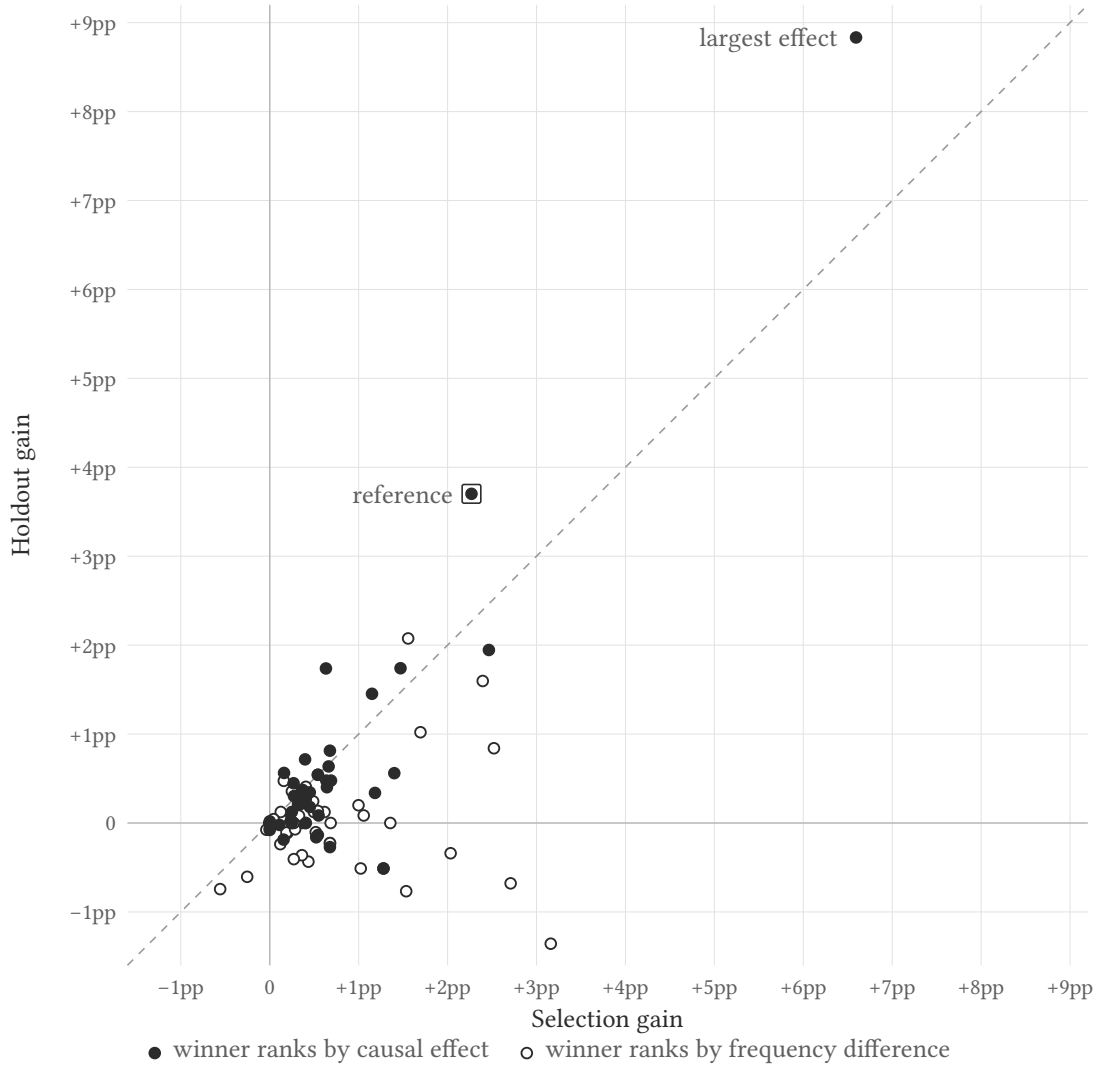


Figure 11: The winning configuration's gain on the selection subset against the primary endpoint on the full holdout, one dot per experiment. On the diagonal the two readings are equal, and dots below it kept less on the holdout than they showed at selection. The labelled point at the top right is the study's largest effect, the residual-stream output at layer 17 of `google/gemma-2b` on CommonsenseQA, at $+6.59$ and $+8.83$pp. Six experiments whose winner is an empty intervention coincide at the origin, and overlapping readings overprint.

Figure 11 places every experiment against the diagonal on which the two readings are equal. Most of the study lies below that line. Had the selection subset served as the report, 85 of the 98 interventions would have counted as improvements, against the 54 the holdout confirms. Nothing in this is a defect of the search. An argmax over 90 evaluated configurations fits the questions it is scored on, as Section 3.5 anticipates, and the shrinkage it produces is the reason Phase 4 (Section 3.6) measures on questions no earlier phase touched.

The ranking criterion itself first appears interchangeable. Every experiment evaluates both halves of the grid, so the best causally ranked configuration can be set against the best frequency-ranked one within the same experiment, the only place where the two criteria meet in a paired comparison. Across the 98 experiments the median difference between those two bests is exactly

zero, and a Wilcoxon signed-rank test [89] finds no separation ($p = 0.153$). The halves separate no better by their mean gain over all 45 configurations ($p = 0.32$) or by how many of them gain anything ($p = 0.37$), so the null does not rest on comparing maxima. If anything the balance tilts the other way, the frequency half offering the better single configuration in 48 experiments against 34, with 16 ties. Among the 83 experiments whose screen validated at least one candidate, so that the causal half is not empty by construction, the count still reads 39 against 34. The winners the search then chose agree, with a median selection gain of $+0.40$ points where the causal criterion won and $+0.36$ where the frequency criterion won. Judged where it is optimised, correlation looks as good as causation.

The holdout undoes that equivalence. Experiments whose winner ranks by causal effect, 44 of the 98, keep a median of $+0.24$ points on it, the 54 whose winner ranks by frequency difference keep exactly zero, and the optimism divides as well, -0.08 against -0.33 points in the median. A frequency-chosen winner thus gives up roughly four times as much of its selection gain on questions no phase touched. Unlike the contrast at selection, this one stands between groups of different experiments, because only the winner of each search reaches the holdout, and membership in the groups is itself an outcome of the search. Where the screen validated nothing, a winner that carries any feature at all can only be frequency-ranked, and nine such experiments sit in the frequency group.

Two objections could explain the separation away. One is that the experiments whose screen validated nothing distort it, since any winner carrying features there can only be frequency-ranked. The other is that the whole finding rests on `google/gemma-2-2b` and `meta-llama/Llama-3.2-1B-Instruct`, the pair on which Section 4.2 saw significance concentrate. Dropping each in turn, and then both, splits the finding. How much each group gives up, the optimism gap, survives every cut. A one-sided Mann-Whitney test [90] places it at $p = 2.8 \times 10^{-4}$ untouched and at $p = 0.032$ under both, the median then -0.19 points on the causal side against -0.36 on the frequency side. What each group ends with does not. That difference survives the first cut, at $+0.32$ against $+0.04$ points among the 83 ($p = 0.0013$), but not the second, both medians falling to zero once the two models go ($p = 0.062$).

Significance concentrates on the causal side as well. Twelve of the 44 causally chosen winners reach it on their holdout against three of the 54 frequency-chosen, an association a one-sided Fisher exact test [88] puts at $p = 3.3 \times 10^{-3}$. The association inherits the concentration of Section 4.2, weakening to $p = 0.133$ without `google/gemma-2-2b` and dissolving at $p = 0.425$ with `meta-llama/Llama-3.2-1B-Instruct` removed as well. That count is therefore fragile too, leaving the optimism gap as the one part that holds every cut. The mechanism behind both was already visible in the reference experiment, where the three strongest correlates of success failed the causal screen and every configuration that beat the selection baseline ranked by causal effect (Section 4.2.1). Causal validation thus buys not a better-looking winner where the search optimises but a winner that keeps more of its gain on questions the search never saw. That is the held-out counterpart of the concern that an apparently relevant SAE feature need not be a causal one [38].

Comparing the two criteria presupposes that the causal side has candidates to offer, and in part of the study it does not. Study-wide, the causal screen rejects a median of 68% of the candidates it tests, at rates ranging from 3% to 100% across experiments. That fraction is the diagnostic Section 3.4 committed to reporting. In fifteen experiments no candidate survived at all, so the causally ranked half of the grid could assemble nothing but the empty intervention there. None of the fifteen belongs to a model above 1 B, and eight of them sit on `EleutherAI/pythia-70m`, the model whose degenerate difficulty screen Section 4.2 set apart.

What the search selected in those fifteen splits them. In six, no configuration anywhere in the grid improved the selection subset, and the winner was an empty intervention that changed no answer on the holdout. The other nine fell back on frequency-ranked features, which gained between $+0.07$ and $+0.68$ points at selection and kept a median of exactly zero on the holdout, all nine staying within half a point in either direction. This also resolves the eight inert interventions of Section 4.2, six of them the empty winners and two of them fallbacks on `EleutherAI/pythia-70m` that carried ten features each and still changed no answer. Every experiment thus ends in one of three ways, fifteen whose screen validated nothing, 68 of the remainder whose intervention fell short of significance, and fifteen that reached it. None of the first group reaches significance, so a screen that validates nothing reads as an early signal that there is nothing to steer, available before the configuration search or the holdout is run. The pipeline as configured does not act on that signal, since its grid keeps the frequency half in play, and the nine fallbacks it chose gained nothing on the holdout.

Removing the SAE from the forward pass, as the approximated intervention does, costs almost nothing except where the winner suppresses features. The clamp of Section 3.1 is what separates them. Where the winning configuration suppresses nothing, the two forms are algebraically identical. The median absolute gap between their holdout gains, 0.079 points across the 44 of them that carry features, is a floor set by numerical resolution rather than a disagreement, a limit Section 4.4.2 quantifies. With at least one feature suppressed the median gap rises by half, to 0.121 points across the remaining 48, since the exact form floors a suppressed feature at zero where the fixed displacement pushes past it. The tail is where it shows most. In the largest case, on `google/gemma-3-1b-pt` with five features suppressed at the strongest grid strength, the two forms differ by 2.38 points and the exact form's gain of $+1.02$ becomes a loss of 1.36. Every gap above one point belongs to a winner that suppresses. The near-coincidence of the reference experiment, $+3.70$ against $+3.67$ points (Section 4.2.1), is thus the common case even among suppressing winners, and the failures of the approximation are confined to that half.

Fifty-one of the grid's 90 configurations win at least once. The winning strength is 1.5 in 48 experiments and 0.5 in 32, so the ends of the range are preferred, leaving 18 for $\alpha = 1.0$, the strength at which Phase 2 tested every candidate in isolation. Half the winners suppress nothing, with $k_{-} = 0$ in 50 experiments against 27 at $k_{-} = 3$ and 21 at $k_{-} = 5$, while the amplified count spreads across all five of its values, each chosen between 12 and 29 times. Which configuration wins is thus a property of the experiment rather than of the method, though what a fixed default would concede is not something these experiments measure, since only each winner reached

the holdout. Whether the fifteen significant gains those searches produced withstand scrutiny of the measurement itself is the question of the next section.

4.4 When the Measurement Misleads

Fifteen experiments ended significant, and Section 4.2 established that chance does not account for their number. Whether it accounts for any single one of them is a different question, and this section asks it of the measurement itself. Two structural hazards were identified in Section 4.1.4 before any result was read. The region of the holdout below $p_{\min}$ starts every question at a 0% baseline, so a paired comparison there can register improvements only. A baseline below the chance level lets an intervention gain by weakening an answer-letter preference rather than by weighing option content differently. Both hazards become checks here, alongside false-discovery-rate control (Section 2.5) and the transfer counts of Section 4.2.5. Table 10 holds the outcome, one row per experiment.

4.4.1 Scrutiny of the Significant Results

Model	Layer	Site	Bench.	Baseline	Gain	p	Gains below $p_{\min}$	Transfers
`gemma-2b`	17	`resid_post`	CSQA	12.3%†	+8.83	7.9×10^{-15}	284 of 476	0 of 3
`Llama-3.2-1B-Instruct`	7	`mlp_out`	CSQA	44.7%	+3.70	8.8×10^{-12}	44 of 233	3 of 3
`gemma-2-2b`	1	`resid_post`	CSQA	47.4%	+0.72	3.3×10^{-5}	0 of 36	2 of 3
`Llama-3.2-1B-Instruct`	15	`mlp_out`	CSQA	45.8%	+1.45	2.5×10^{-4}	0 of 116	3 of 3
`gemma-2-2b` [release: snap loss]	12	`resid_post`	CSQA	47.4%	+0.64	2.7×10^{-4}	0 of 35	2 of 3
`gemma-2-2b` [release: seonglae]	19	`resid_post`	CSQA	47.4%	+0.48	0.0025	0 of 28	3 of 3
`gemma-2-2b` [release: gemma scope]	19	`resid_post`	CSQA	47.4%	+0.48	0.0032	0 of 29	3 of 3
`Llama-3.2-1B-Instruct`	15	`mlp_out`	ARC	38.4%	+2.08	0.0035	0 of 24	3 of 3
`Llama-3.2-1B-Instruct`	9	`resid_post`	ARC	40.6%	+1.74	0.0053	0 of 18	3 of 3
`gemma-3-270m`	9	`resid_post`	CSQA	12.2%†	+0.36	0.0065	0 of 12	0 of 3
`gemma-2-2b` [release: standard]	12	`resid_post`	CSQA	47.4%	+0.37	0.010	0 of 23	2 of 3
`DeepSeek-R1-Distill-Qwen-1.5B`	13	`mlp_out`	ARC	23.2%†	+1.74	0.013	0 of 16	0 of 3
`SmolLM2-135M`	15	`mlp_out`	CSQA	11.4%†	+0.36	0.015	0 of 14	0 of 3
`gemma-2-2b` [release: gemma scope]	12	`resid_post`	CSQA	47.4%	+0.34	0.024	0 of 25	2 of 3
`gemma-2-2b`	24	`resid_post`	CSQA	47.4%	+0.26	0.026	0 of 16	3 of 3

ARC = ARC-Challenge CSQA = CommonsenseQA † baseline below the benchmark's chance level

Table 10: The fifteen significant experiments under the checks of this section, ordered by the p value of the primary endpoint. The rule between rows separates the eight the Benjamini-Hochberg procedure [66] retains at a false-discovery rate of 0.05 from the seven it does not. *Gains below* $p_{\min}$ counts the improvements each experiment draws from the holdout region whose baseline is 0% by construction, out of all its improvements. *Transfers* counts significant gains among the three transfer measurements of Section 4.2.5. A dagger marks a baseline below the benchmark's chance level (Section 4.1.4). Gains are in percentage points, and bracketed tags separate rows differing only in the SAE release (Table 12).

The table orders the fifteen by the p value of their primary endpoint, and the rule after the eighth row is the cut of the Benjamini-Hochberg procedure at a false-discovery rate of 0.05 [66]. Above it, the eight retained experiments remain discoveries under multiplicity control, the cut falling at $p = 0.0035$. The seven below it, from $p = 0.0053$ to $p = 0.026$, are the surplus that correction surrenders. Retention does not follow effect size, since the two largest surrendered gains, $+1.74$ points each, exceed five of the eight retained ones. It follows the holdout sizes Section 4.2 described, which make the same gain easier to certify on CommonsenseQA than on ARC-Challenge. The count also carries one qualification. Seven of the fifteen tests share a holdout, so the 98 tests are not independent. Such dependence is tolerated by the procedure in its positive form, and the variant of Benjamini and Yekutieli, which assumes nothing about it, retains five [91].

How much each gain owes to the region below $p_{\min}$ is the re-test Section 4.1.4 announced, reported in the *Gains below* $p_{\min}$ column. Applied to all 98 experiments, it lifts none of the others to significance. Across the fifteen experiments the region covers between 40.4% and 64.7% of the holdout, and since every question in it starts wrong, it holds an even larger share of the answers available for correction. In thirteen of the fifteen experiments, not a single credited improvement comes from it, so excluding the region repeats their verdicts to the digit. The reference experiment draws 44 of its 233 improvements there and keeps its significance without them, at $p = 2.1 \times 10^{-6}$ against 8.8×10^{-12} before (Section 4.2.1). One experiment remains, and it is the largest in the study.

That remaining experiment, the $+8.83$ points at the layer-17 residual stream of `google/gemma-2b` on CommonsenseQA, fails all three checks that look past its p value. Its dependence on the region has no parallel among the significant experiments. With 56.0% of its holdout below $p_{\min}$, it draws 284 of its 476 improvements from the region, three of every five against 44 of 233 in the reference experiment, the next-highest share. Exclusion does not merely weaken this verdict but reverses it. Outside the region, 192 improvements stand against the unchanged 267 worsenings and the test returns $p = 0.9998$, so where the comparison has two sides the intervention breaks more than it fixes. The below-chance flag marks the same experiment, steering having moved a 12.3% baseline to 21.1% against a 20% chance level, a rise that reads as the removal of an answer-letter bias rather than as ability gained (Section 4.1.4). Transfer agrees, all three of its measurements falling (Section 4.2.5). The aggregate does not rest on this result, since without it fourteen of 97 experiments remain significant, a count arising under the global null with probability 3.4×10^{-4}. What does rest on it is the largest depth contrast of Section 4.2.3, which has this experiment as its deeper endpoint. The one comparison suggesting that depth can move an effect by several points therefore stands on the least trustworthy gain in the study.

Three further experiments carry the below-chance flag, and all three survive the region re-test, so the two checks catch different hazards. The interventions on `HuggingFaceTB/SmolLM2-135M` and `google/gemma-3-270m` add $+0.36$ points to CommonsenseQA baselines of 11.4% and 12.2%, leaving both models, even steered, far below what guessing achieves. On `deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B`, a gain of $+1.74$ points lifts a 23.2% ARC-Challenge baseline

to 25.0%, four hundredths of a point short of the 25% chance level. It is one of the two experiments on this model whose unsteered predictions Section 4.1.4 found concentrated on a single answer letter, here on nine of every ten questions. For all four flagged experiments the most economical reading is the one Section 4.1.4 prepared, a gain in measured accuracy produced by weakening a letter preference, which the benchmark score rewards whether or not any option content is weighed differently. None of the four carries a significant transfer.

Where the fifteen sit completes the pattern. Eleven belong to two models, seven to `google/gemma-2-2b` and four to `meta-llama/Llama-3.2-1B-Instruct`, the concentration Section 4.2.2 set aside. Removing the two models removes 28 experiments and leaves four significant results among the remaining 70, a count indistinguishable from chance, since four or more of 70 tests at level 0.05 arise under the global null with probability 0.466. Those four are exactly the flagged four. The study's significant results thus divide in two, eleven on two models, at baselines at or above chance and every one with a significant transfer, and four elsewhere, all below chance and none transferring. Those eleven share more than their models. All seven `google/gemma-2-2b` results rest on one screening, the same 3,774-question CommonsenseQA holdout at a 47.4% baseline, so they are seven interventions from five SAE releases at four depths, all judged on the same questions, rather than seven independent confirmations of one finding.

4.4.2 Power and Numerical Resolution

The experiments that ended without significance need the converse check, whether the measurement could have produced one. Evidence in the paired test of Phase 4 (Section 3.6) comes only from discordant answers, the questions whose correctness the intervention changes in either direction. The median experiment produces 13 of them. Twenty-four experiments produce fewer than five, and eight among those change no answer at all, the inert interventions Section 4.3 resolved. Below five discordant answers no verdict is reachable in either direction, since even a perfect four-to-zero split returns $p = 0.0625$ and misses the level. That these interventions did little is itself measured, most plainly for the eight that did nothing. What their nulls cannot do is separate a small benefit from none, which makes them weaker evidence than the null of an experiment whose test had answers to work with. Nineteen of the 24 run on ARC-Challenge, whose holdouts are roughly a quarter the size of CommonsenseQA's, and seventeen run on Pythia models. Counting all 24 as refutations would overstate what was measured.

The same accounting extends to the two axes that returned no direction. Of the 33 non-tied depth contrasts in Section 4.2.3, the shallower layer yields the larger gain in 19. A 95% interval of Clopper and Pearson on the underlying preference rate spans 0.392 to 0.745 [92], and against a true preference of 60 to 40 the sign test [85] over 33 contrasts would detect it in 17% of repetitions. The site comparison of Section 4.2.4 stands on less still, seven wins in eleven non-tied pairs, an interval from 0.308 to 0.891, and a power of 11% against a true 70 to 30. Both verdicts therefore bound the magnitude of their axis, which the data constrain to fractions of a point, and say little about direction, since a moderate true preference would usually go undetected.

How small a difference the study can resolve at all is bounded by its arithmetic. Wherever the winning configuration suppresses no feature, the exact and the approximated intervention are

algebraically identical, so any gap between their measured gains reflects rounding in `bfloat16` forward passes that order their operations differently, not a disagreement between methods. Across the 44 such experiments with a non-empty intervention, Section 4.3 reported the median gap as 0.079 points. Only 14 of the 44 measure exactly zero, the upper quartile stands at 0.30 points, and the largest gap reaches 0.60. Identical interventions can therefore differ by a few tenths of a point on this hardware. The axis gaps of Section 4.2.3 and Section 4.2.4, a median of 0.20 points between depth endpoints and 0.43 at the widest site pair, lie inside that range. Effects of that size, in either direction, are indistinguishable from the arithmetic, so the two axes are limited by the study's numerical resolution as much as by their sign tests.

4.4.3 The Composition of the Transfer Gains

What the 68 significant transfer gains consist of is the question Section 4.2.5 left open, and two rates decompose it. The conversion rate of a transfer measurement is the share of the pool's wrong answers the intervention corrects, its breaking rate the share of the right answers it breaks. Together they carry a benchmark-specific null. A redirected wrong answer lands on the correct option one time in three on four options and one time in four on five, whereas a redirected right answer always lands wrong, so undirected shuffling breaks right answers at three or four times the rate at which it fixes wrong ones. All 68 significant transfers fall below their own null, the ratio of breaking to conversion rate reaching a median of 0.75. Every one of them moves answers toward correctness with a direction undirected redistribution would not produce in expectation.

Composition is the other ingredient. Identical rates produce a larger raw gain wherever more of the pool starts wrong, and 60 of the 68 pools start poorer than the home holdout. Recombining each measurement's two rates at the home holdout's baseline mix asks what the same behaviour would have yielded at home. Under that standardisation the median gain falls from $+0.96$ to $+0.54$ points and 22 of the 68 no longer show a gain, so roughly a third of the significant transfers rest on where they were measured rather than on how the intervention behaved. The reference experiment shows both faces. Its ARC-Challenge and OpenBookQA transfers keep $+6.44$ and $+6.13$ points at the home mix, more than its holdout gain of $+3.70$, while its $+2.03$ points on HellaSwag rest on a pool three quarters of which starts wrong, a breaking rate of 7.6% exceeding a conversion rate of 5.3% and the standardised gain turning into a loss of 0.43 points. One alternative remains untestable. A directed shift towards an answer letter that is correct more often than the letters it displaces would pass the ratio test without any task signal, and per-pool letter distributions were not recorded. The question of Section 4.2.5, task ability or a format-level preference the benchmarks share, narrows to that alternative and no further.

4.4.4 What Survives Every Check

Seven of the fifteen experiments pass every check at once, retained by the false-discovery-rate control, at baselines above chance, significant with the region below $p_{\min}$ excluded, and transferring significantly on at least two of their three benchmarks. Three run on `meta-llama/Llama-3.2-1B-Instruct` and four on `google/gemma-2-2b`, with primary gains between $+0.48$

and $+3.70$ points. Within everything this chapter can test, those results stand. How far they generalise is a separate question, since they come from two mid-scale models on two benchmarks, under screenings that are partly shared. Both statements carry into the synthesis that follows.

4.5 Synthesis

The scrutiny of Section 4.4 ended with two statements standing side by side, seven results that withstand every check this chapter can apply, and a concentration of those results on two models. Weighing the one against the other answers the central question of this thesis, whether SAE-based feature steering can produce measurable and statistically significant accuracy improvements on multiple-choice benchmarks. In favour stands the count. Fifteen of the 98 experiments reached significance where 4.9 were expected by chance, a surplus arising with probability 1.1×10^{-4} under a global null, and eight remain discoveries under false-discovery-rate control. Seven of the fifteen withstand every further check at once, gaining 0.48 to 3.70 percentage points on baselines above chance and transferring significantly to at least two of the three benchmarks each never saw. Against the count stands the concentration. All seven run on `meta-llama/Llama-3.2-1B-Instruct` or `google/gemma-2-2b`, and without these two models the study holds four significant results among 70 experiments, a count indistinguishable from chance, all four carrying the below-chance flag of Section 4.4.1. The verdict on the central question is accordingly a conditional yes. Feature steering produced measurable, statistically significant accuracy improvements without any training, and it produced them in a specific corner of the design space rather than wherever it was applied.

Model scale (RQ1) receives an asymmetric answer. The holdout gain shows no ordering by parameter count, and its significant instances concentrate between 1 B and 2 B, where power and interval coverage are both adequate. By contrast, the survival rate of the causal screen rises with scale, from a median of 8.0% at 410 M and below to 64.0% at 8 B, so larger models, or the SAEs trained on their activations, appear to expose more decision-relevant structure. Survival nonetheless acts as a precondition rather than a predictor, since no experiment whose screen validated nothing reached significance, while at 8 B the study's highest survival coincides with a difficulty interval that has largely collapsed, an absence the evidence cannot assign to method or measurement.

Layer depth (RQ2) moves the outcome by less than half a point in 30 of the 39 controlled contrasts, at a median gap of 0.20 points and with no detectable direction, the shallower layer ahead in 19 and the deeper in 14. The one contrast suggesting that depth can shift an effect by several points has the study's least trustworthy gain as its deeper endpoint (Section 4.4.1), and no gap large enough to matter recurred on a second benchmark. Within the sampled region, the middle of the stack and deeper, depth is therefore a setting to fix empirically per configuration rather than by rule.

Intervention site (RQ3) separates its sixteen release-matched pairs even less, by at most 0.43 points at a median of 0.14, the residual stream ahead in seven pairs, the MLP write in four,

and five tied. No matched pair exists on the models with the largest effects, so the contrast is measured only where effects are fractions of a point. Differences of that size lie within the study's numerical resolution (Section 4.4.2), which makes this null and the depth null ceilings on an effect rather than demonstrations of equivalence. The choice of site can therefore follow whichever SAE is available.

Cross-benchmark transfer (RQ4) occurs and concentrates on the interventions that pass their own holdout, eleven of the fifteen significant experiments transferring significantly against 22 of the 83 others. Each of the 68 significant transfer gains moves answers with a direction undirected redistribution would not produce in expectation, though roughly a third of them owe their size to target pools in which more questions start wrong (Section 4.4.3). An intervention that survives its own holdout therefore tends to carry a signal not specific to the benchmark it was built on, one the recorded data can narrow to task ability or a directed shift in answer-letter preference, but no further.

Causal validation (RQ5) earns its keep, though only held-out data shows it. At selection the two ranking criteria are indistinguishable, whereas on the holdout a frequency-ranked winner surrenders roughly four times as much of its selection gain as a causally ranked one, the one separation that survives every robustness cut of Section 4.3. The retained gains point the same way, a median of $+0.24$ points against exactly zero, though that contrast rests on the two dominant models. Abstention carries information as well, since the fifteen experiments in which no candidate survived produced no significant intervention by any route.

Taken together, the verdicts draw an operating envelope, the corner of the design space in which the method as built produced improvements that survive every check. Inside it sits a model of 1 B to 2 B parameters that the scoring convention measures cleanly, with a graded difficulty score, no dominant answer letter, and a baseline above chance. A causal screen that validates candidates belongs to the conditions as well, since no experiment without survivors reached significance. Benchmark size sets how small a gain the paired test can certify, which is why twelve of the fifteen significant results have CommonsenseQA as their primary benchmark. Layer depth and intervention site appear in no condition. Neither showed a measurable preference, so both can follow where public SAEs exist.

The edges of this envelope are boundaries of the measurement before they are boundaries of the method. At 410 M and below, every experiment breaks at least one of these conditions, whether by a screen returning few distinct scores, by a dominant answer letter, or by a baseline below chance, so a gain there cannot be attributed to task ability. The interval collapses at the other end instead, the 8 B models retaining 4% to 9% of each benchmark inside it, so discovery and validation run on some of the study's smallest subsets. Whether the method itself fails at either end is accordingly unresolved rather than refuted. Nor does the envelope amount to a guarantee within its bounds, resting as it does on two mid-scale models and two primary benchmarks under partly shared screenings. What it records is a training-free pipeline that, given nothing but a frozen model, a public SAE, and a benchmark, produced accuracy gains of up to 3.70 points. No check of this chapter could reduce those gains to chance, to the measurement, or to the benchmark the intervention was built on.

5 Related Work

Chapter 3 presented a training-free steering pipeline that leaves model weights untouched, and Chapter 4 measured where it improves multiple-choice accuracy. This chapter situates that contribution in the design space of methods for directing the behaviour of a large language model. Two families organise the space. The first family improves the model itself, whether through preference-based training, parameter-efficient adaptation, or targeted weight edits (Section 5.1). A second family leaves the parameters frozen and manipulates the forward pass at inference time, the family to which the method of this thesis belongs (Section 5.2). Section 5.3 then locates the pipeline against its nearest neighbours along the dimensions that separate it. The sparse autoencoders themselves are not revisited here. They are the instrument the pipeline builds on rather than competing approaches, and Section 2.2 already covers them.

5.1 Fine-Tuning and Model Editing

Two questions separate the methods in this section, what is changed and what is trained. The first kind changes the model by training parameters, either all of them, a small set attached beside them, or only the few that store a single fact (Section 5.1.1). Nothing of the model changes in the second, where an edit of its hidden states is trained instead, showing that untouched weights alone do not make a method training-free (Section 5.1.2).

5.1.1 Weight Modification

The dominant route to changing what a model does is to change its weights. Christiano et al. establish the recipe: annotators are shown two candidate behaviours and mark the better one, a reward model learns to reproduce these judgements, and reinforcement learning then optimises the agent against that learned reward instead of a hand-specified one [93]. Ziegler et al. and Stiennon et al. carry this recipe to language models, training them to continue text in a required style and to write summaries that human judges prefer [94], [95]. Ouyang et al. consolidate the approach into the three-stage pipeline now known as reinforcement learning from human feedback (RLHF), which aligned the InstructGPT models [8]. Supervised fine-tuning (SFT) first teaches the model to imitate human-written example responses, a reward model is then trained on human rankings of model outputs, and the reinforcement-learning algorithm Proximal Policy Optimization (PPO) finally adjusts the model's weights to maximise the reward model's score [96].

Direct Preference Optimization (DPO) later removes much of this machinery. Rafailov et al. show that the model can be fine-tuned on the preference pairs directly, with a simple loss that raises the probability of the preferred response relative to that of the rejected one, so neither a separate reward model nor a reinforcement-learning loop is needed [9]. These methods define the current alignment standard. Its latest extension, reinforcement learning against verifiable

rewards, replaces the learned reward with one a program can check and underlies reasoning models such as DeepSeek-R1 [83]. Each method acquires the target behaviour by optimising the model's parameters against curated data, so a new task requires new data, a new training run, and redeployment of the changed weights. The change is also not free of side effects. Ouyang et al. document an alignment tax, a degradation on unrelated benchmarks caused by the preference training, which they counter by mixing pre-training data back into the optimisation [8].

A second line keeps the paradigm but shrinks the update. Hu et al. freeze the pre-trained weights entirely and train Low-Rank Adaptation (LoRA) modules instead: pairs of small matrices attached to the existing weight matrices, which are tuned in place of the full model [7]. This cuts the number of trainable parameters by orders of magnitude at accuracy comparable to full fine-tuning, an approach known as parameter-efficient fine-tuning (PEFT). Model editing narrows the change further, to individual facts. Rank-One Model Editing (ROME) treats a fact the model has memorised, for instance that the Eiffel Tower stands in Paris, as an association stored at a specific location inside the network. Meng et al. trace which internal computations are causally responsible for recalling such a fact, locate them in the MLP modules of the middle layers, and rewrite the one implicated weight matrix with a minimal update built from a single pair of vectors, after which the model places the Eiffel Tower in Rome [97]. MEMIT extends the same procedure to mass editing, inserting thousands of associations across several layers at once [98]. Surgical as they are, these edits remain permanent modifications of the parameters, present in every subsequent forward pass and removable only by restoring the original weights.

5.1.2 Representation Finetuning

Between weight modification and inference-time control lies a boundary case that sharpens the distinction this thesis rests on. Wu et al. keep every weight of the base model frozen and instead train the intervention itself, an approach they call Representation Finetuning (ReFT) [99]. A small learned function edits the per-token hidden states of the residual stream (Section 2.1) at chosen layers and token positions during the forward pass. It serves the same task-adaptation purpose as LoRA, and the authors report comparable accuracy from far fewer trained parameters. No sparse autoencoder is involved. The mechanism resembles the activation steering of Section 2.3, except that the edit is not computed from recorded activations but fitted to the task by gradient descent. A successor method, Reference-free Preference Steering (RePS), trains such edits on preference pairs so that a target concept can be both promoted and suppressed [100]. No weight of the base model moves, yet something is still trained. That is the property uniting this section. Whether a method fine-tunes every parameter, trains adapter matrices, rewrites a single weight matrix, or fits an edit to the hidden states, the behaviour is in each case bought with gradient-based training on task data. The pipeline of Chapter 3 involves no such training at any stage: its intervention vector is assembled from forward passes, counting, and selection alone.

5.2 Activation Steering

Activation steering belongs to the second family, the methods that leave the model's stored parameters untouched and act on the forward pass while it runs. Instead of changing what the model is, they change what it computes. An edit of this kind is transient. It exists only in the pass into which it is injected, applies per input and per token position, and disappears when its hook is removed, which is precisely what enables the paired comparisons of Chapter 3, in which the same question is scored once with and once without the intervention. The family's most familiar member is prompting, which conditions a frozen model through its input alone and remains a baseline that representation-level methods do not yet consistently beat [33]. Prompting, however, treats the model as a black box, whereas the methods gathered here reach into the hidden states directly, with the mechanics laid out in Section 2.3. Their development runs from single undecomposed directions (Section 5.2.1), through directions expressed in the sparse feature basis of an SAE (Section 5.2.2), to the question this thesis is built on, whether such interventions can improve task accuracy (Section 5.2.3).

5.2.1 Vector-Space Steering

The vector-space approach supplies the field's core repertoire. Subramani et al. first demonstrate that such directions exist, using gradient search to find latent vectors whose addition to the hidden states makes a frozen model reproduce a chosen target sentence almost perfectly [10]. Turner et al. show that this search is unnecessary and compute the vector directly, as the activation difference between a single pair of contrasting prompts, a method they call ActAdd [17]. Contrastive Activation Addition (CAA) stabilises this estimate by averaging the difference over whole datasets of contrastive prompt pairs [18]. Zou et al. widen the toolkit into a programme they call representation engineering, deriving directions from designed stimulus sets and applying them to monitor and control properties such as honesty and harmlessness [11].

A single direction can carry a remarkable amount of behaviour. Arditi et al. show that refusal in thirteen open chat models is mediated by one direction, whose removal disables refusal and whose addition induces it [19]. Hendel et al. and Todd et al. find that in-context demonstrations compress into a transportable task vector, which triggers execution of the demonstrated task even when the demonstrations are absent from the prompt [12], [13]. Liu et al. refine the same idea into in-context vectors that make in-context learning more effective and more controllable [14]. Around the core mechanism an engineering layer has formed. Van der Weij et al. suppress skills as broad as general coding ability, then attempt five behaviours at once. Summed into one vector, the individual directions interfere, and only one behaviour still steers as intended. Injected in parallel instead, one vector per behaviour at its own layer, two of the five steer about as well as alone, at little cost to general performance [15]. Conditional Activation Steering applies a vector only when the input matches a trigger condition, so that a model can be made to refuse harmful requests while continuing to answer harmless ones [16].

How reliably such vectors steer is a separate question, and recent audits answer it cautiously. Tan et al. measure steering both on the kind of data the vectors were extracted from and on data

from outside it, and find the effect highly variable from input to input [20]. Part of the apparent success, they show, stems from surface patterns in the prompts rather than from the target behaviour, and on some inputs the steering even reverses. Braun et al. add that the vector one obtains depends on the prompt type used to extract it, with no type reliably superior [101]. Pres et al. direct the same scrutiny at the evaluations themselves. They argue that steering studies have rested largely on subjective demonstrations, and propose that interventions be judged in contexts resembling the downstream task, with the model's likelihoods taken into account, in a form that allows comparison across behaviours, and against explicit baselines [21]. Two lessons from these audits carry into everything that follows. A dense direction is opaque, so which concepts an intervention moves cannot be named, and the evidence for steering has often been illustrative rather than measured. The first motivates expressing the intervention in a basis in which each coordinate names a concept, the step taken next, and the second motivates the holdout discipline and significance testing that Chapter 3 builds in.

5.2.2 Feature-Space Steering

In the feature basis of an SAE, an intervention can name what it moves. Each coordinate of the learned dictionary corresponds to a candidate concept (Section 2.2), so a feature-level edit addresses the opacity that limits a dense direction. Templeton et al. demonstrate such control at production scale, clamping individual features of Claude 3 Sonnet to steer its behaviour [25]. Joshi et al. turn the same operation into topic steering, amplifying features related to a target topic so that generation stays on it [30]. O'Brien et al. amplify features that mediate refusal to make a model more robust against unsafe prompts, and report a tension between the safety gained and the general capability retained [31].

Which features to move is the harder question. A dictionary offers thousands to hundreds of thousands of candidates per layer (Section 2.2), and the selection signal has evolved in stages. Early demonstrations choose semantically, whether from a feature's dashboard of activating examples or by scoring each feature's semantic similarity to a text representing the target topic [25], [30]. Later methods measure rather than read. Bayat et al. contrast the sparse codes of examples that show a behaviour with those of examples that do not, isolating behaviour-specific features [102]. Chalnev et al. observe that a steering vector's side effects can themselves be read in the feature basis, and construct vectors that move a chosen feature while disturbing few others, a method called SAE-Targeted Steering [103]. Feature-Guided Activation Additions combine this machinery with the contrastive construction of CAA [104]. Arad et al. sharpen the diagnosis, separating input features, which track what the model reads, from output features, which drive what it generates. They find that steering improves when candidates are ranked by their effect on the output rather than by their activation pattern [105]. Information-theoretic criteria follow, selecting features by mutual information with the target behaviour [106] or by Fisher information for targeted unlearning [107]. None of these signals asks whether moving a feature helps the model on the task. Even the output score of Arad et al., which does move each candidate, records only how strongly its concept tokens are promoted on a neutral prompt. A candidate is therefore never judged by intervening and measuring the outcome of the task the steering is meant to serve.

How well feature-space steering works has been assessed from three angles. On the indirect-object-identification task, Makelov et al. judge SAE features against dictionaries learned with supervision for the task and find them usable for steering, though less reliable than the supervised reference [38]. In a case study on social biases, Durmus et al. find a steering sweet spot, a band of strengths within which a feature shifts the targeted bias at little cost to general capability. Beyond that band capabilities degrade sharply, and even inside it an intervention can spill onto biases it was not chosen to move [108]. At benchmark scale the verdict is sobering. AxBench compares prompting, fine-tuning, and representation-level methods on concept steering and detection, and finds prompting ahead of everything, fine-tuning second, and SAE-based steering among the weakest options [33]. All three assessments measure control over the model's behaviour. A different question is whether a feature-level intervention can instead improve a model's accuracy on a task it already performs. Research that asks it has emerged only recently.

5.2.3 Steering for Task Performance

Most steering work aims at behaviour, at what kind of text a model produces. A smaller line asks whether an intervention can make a model measurably better at a task. The earliest affirmative answers come from dense steering vectors. Li et al. introduce Inference-Time Intervention (ITI), training small probes to find attention heads whose activations separate true from false statements and shifting those heads along their truth directions during decoding [34]. On TruthfulQA this more than doubles the truthfulness score of an instruction-tuned LLaMA, from 32.5% to 65.1%. Stolfo et al. derive a vector as the mean difference between activations of prompts with and without an instruction, and add it so that a model follows constraints such as output format and length [109]. The layer receiving the addition is chosen on held-out data, and the gains are confirmed with McNemar's test, one of the few uses of a significance test in this literature. Cheng et al. combine probe-selected attention heads with a backtracking decoder that regenerates a span whenever the output drifts, raising the average accuracy of LLaMA2-7B-Chat over six multiple-choice datasets, MMLU among them, from 65.1% to 78.8% [110]. None of these methods touches an SAE, and each derives its directions from labelled task data, yet together they establish that inference-time interventions can buy measured accuracy.

Reasoning models have become this line's natural testbed. Venhoff et al. switch behaviours such as backtracking and uncertainty estimation on and off in DeepSeek-R1-distilled models with contrastive vectors, characterising the reasoning process rather than improving it [111]. Efficiency is the more common goal. Azizi et al. compress chains of thought with a single verbosity direction, shortening generations by up to two thirds at essentially unchanged accuracy [112], and Huang et al. reach a similar trade-off by removing an overthinking direction identified on a low-dimensional activation manifold [113]. Working at the level of individual neurons, Zhao et al. amplify around 150 contrastively selected MLP activations, together with an inserted trigger token, and elicit long-form reasoning from base models, lifting Qwen2.5-7B from 69.2% to 73.0% on MATH-500 [114]. Across these works the intervention acts while the answer is being generated, at many token positions rather than at one, candidate directions are validated in aggregate if at all, and the reported gains carry no significance test.

In 2025 the accuracy question reaches the sparse feature basis. Galichin et al. score every feature of an SAE trained on DeepSeek-R1-distill activations by how much of its activation mass falls on reasoning-related words, and hand-interpret the top candidates [35]. Steering the best of them yields gains of up to 13.4 percentage points on the thirty problems of AIME-2024, with smaller rises on MATH-500 and GPQA Diamond. Which features count as the best, however, is settled by their accuracy on MATH-500, and the steering strength is tuned there as well. Selection and evaluation therefore overlap on that benchmark. Li et al. steer a single feature, chosen by contrasting activations between verbal and symbolic renderings of a solution, and raise the GSM8K accuracy of a distilled 8B model from 82.7% to 85.7% under an LLM judge [36]. Which feature to steer, and how strongly, is again settled by trying candidates on the reported sets themselves. Those GSM8K figures come from a 300-question sample drawn for exactly that search. The most developed pipeline is that of Fang et al. [32]. Candidate features are recalled when their decoder columns, read through the unembedding, place high logit weight on the keywords of a target reasoning strategy, such as backtracking or multi-perspective verification. Each survivor is then validated by steering on held-out responses, and those that most reliably induce the intended strategy are kept. Applied to previously wrong answers, the winning features raise the fraction of corrected answers by seven percentage points on average over budget forcing, a baseline that simply lets the model reason for longer. Their screen validates features causally and on held-out data, yet what it confirms is that a feature induces its strategy, not that accuracy improves. A trained router, fitted to correction outcomes, additionally selects the strategy for each problem in these runs. The correction gains rest on one author-trained SAE for each of two 8B models at the final layer and are reported with per-method confidence intervals, never with a paired test of the difference.

Closest to this thesis stands CorrSteer. Cho et al. correlate SAE feature activations, collected while the model generates, with task correctness, take the most positively correlated feature of each layer, and set each steering coefficient to the feature's mean activation on correctly answered samples [37]. A pruned variant additionally keeps only those per-layer winners that survive an intervention test on a validation split. The pipeline is fully automated, requires no task-specific tuning, and in its unpruned form lifts Gemma-2 2B by 3.3 percentage points on MMLU. Beyond accuracy, the same automation raises classifier-scored refusal of harmful requests on HarmBench from 46.6% to 73.7%. That gain is real but not free, since the same steering can push the model to over-refuse safe prompts, a cost the authors chart on XSTest. Its two-stage principle, correlation to nominate candidates and intervention to confirm them, is the organising idea of Phases 1 and 2 in Chapter 3.

The designs differ on nearly every axis that Chapter 3 fixes deliberately. CorrSteer reads features from generated tokens and scores unconstrained generation, so part of its MMLU gain stems from features that repair output format. Cho et al. estimate that part at roughly a tenth [37]. A comparison they report for one seed suggests more, since constraining the unsteered model to the four answer letters already recovers 3.2 of the 4.1 percentage points that steering gains. By reading the final prompt position and restricting scoring to the answer-letter logits, the pipeline of this thesis closes that channel by construction.

Even the correlation is not the same statistic. CorrSteer takes the Pearson correlation of activation values [37], while Phase 1 counts activation as a binary event and ranks features by the activation-frequency difference, a choice that keeps uncalibrated activation magnitudes out of the score. Amplification is CorrSteer's only direction, since subtracting the decoder columns of negatively correlated features collapsed accuracy on two benchmarks when applied across all layers. Cho et al. trace that collapse to features often inactive when the model answers correctly, so their directions are subtracted from states in which they were never present. Suppression under the clamp of Section 3.1 takes an active feature at most to zero and leaves an inactive feature untouched, so that mechanism cannot arise. What CorrSteer adds is moreover a fixed vector, precomputed once and blind to the hidden state it lands on, the form Section 3.1 introduces as the approximated intervention. The exact intervention of this thesis encodes each hidden state before displacing it, and what the approximation gives up is not assumed but measured in Chapter 4.

Feature work in CorrSteer draws on raw benchmark samples. Phase 0 instead confines it to questions inside a model-relative difficulty interval and builds its subsets by difficulty-matched splits. CorrSteer prunes only the per-layer winners on a small validation split, whereas Phase 2 screens a broad shortlist feature by feature on a difficulty-matched disjoint subset. Steering strength is fixed to a mean activation rather than searched, the intervention enters the residual stream at many layers rather than one layer and site at a time, and the reported evidence is a mean and standard deviation over seeds. Across the two models they study, the rule yields coefficients from 0.002 to 61.6, a spread Cho et al. attribute to differences in SAE architecture, feature count and model size [37]. The strength applied is thus a property of the SAE and model it was read from. Phase 3 makes it a parameter instead, searched on one grid and settled per experiment by measured effect. The counts CorrSteer tabulates of answers changed for better and for worse are precisely the discordant pairs on which McNemar's test operates (Section 2.5). That test is never run.

Of the works above, few validate each candidate feature in isolation on data held out from its discovery, none varies all three of model scale, layer depth, and site as controlled factors, and only one tests its gains for significance [109]. The remaining difference is one of aim. CorrSteer and its neighbours contribute steering methods, whereas this thesis contributes a characterisation, of where, across scale, depth, and site, feature steering improves accuracy at all, under a protocol built to make that claim testable.

5.3 Positioning

The chapter closes by locating the pipeline of Chapter 3 in the design space the two families span. Table 11 summarises the comparison along five dimensions. The first two record what a method leaves untouched, the model's weights and, more strictly, whether behaviour is acquired without gradient-based training anywhere (Section 5.1). A third dimension is the basis in which the edit is written, a dense direction in activation space or the sparse feature basis of an SAE (Section 5.2.2). The fourth concerns the discipline of the evidence, whether every candidate feature is validated

in isolation, by its effect on the task outcome, on data disjoint from its discovery (Section 5.2.3). A fifth dimension covers three factors that any steering experiment must fix, the scale of the model, the layer the intervention acts on, and the site within that layer. Studying a factor means varying it alone, with everything else held fixed, and reporting how the effect changes. Tuning a factor to a good value does not qualify, and only a work that studies all three earns a checkmark.

Method family	weights untouched	training-free	intervention basis	per-feature validation	model scale, layer and site studied
Preference training (SFT, RLHF, DPO) [8], [9]	×	×	–	–	–
Weight adaptation and editing (LoRA, ROME, MEMIT) [7], [97], [98]	×	×	–	–	–
Trained interventions (ReFT, RePS) [99], [100]	✓	×	dense	–	–
Dense steering (ActAdd, CAA, RepE) [11], [17], [18]	✓	✓	dense	–	(✓)
Probe-based head steering [34], [110]	✓	×	dense	–	×
Instruction steering [109]	✓	✓	dense	–	×
Feature-space steering for behaviour [25], [30], [31], [103], [105]	✓	✓	SAE	×	(✓)
Reasoning-feature steering [32], [35], [36]	✓	(✓)	SAE	(✓)	×
CorrSteer [37]	✓	✓	SAE	(✓)	×
This thesis	✓	✓	SAE	✓	✓

Table 11: Positioning of this thesis against the method families of this chapter. ✓ marks a property that is present, × one that is absent, (✓) a partial form discussed in the text, and a dash a dimension that does not apply. The intervention basis column carries a label rather than a mark.

Read row by row, Table 11 retells the chapter. Weight training gives way to frozen weights, trained interventions to training-free ones, dense directions to a feature basis, and proxy selection to validation by intervention. In the training-free column reasoning-feature steering is bracketed rather than marked outright, since Fang et al. train a router on correction outcomes to select a reasoning strategy for each problem [32]. Among prior work the validation column stays almost empty, carrying only two bracketed marks. Reasoning-feature steering earns one mark through Fang et al., who validate strategy features on held-out responses, and CorrSteer the other by pruning per-layer winners on a validation split. Each falls short in a different way. Fang et al.'s screen establishes strategy induction, not an accuracy gain, and CorrSteer tests only the single per-layer winner rather than a broad shortlist. The last column tells the same story. Instruction steering sweeps a layer only to pick one, while dense steering and feature-space steering for behaviour are bracketed, since CAA and Arad et al. report their results across layers

and across model sizes [18], [105]. No steering row varies the site at all, so none of them studies all three.

Beyond Table 11, three works examine where an intervention attaches, though only in fragments. Billa predicts, from how well each layer's hidden state already encodes the answer, whether and at which layer a dense steering vector will succeed [115], and Gadgil et al. select that layer separately for every input [116]. Das and Gaur map, layer by layer, where the SAE features of Gemma-2 2B are most vulnerable to steering toward harmful behaviour [117]. None of this characterises accuracy-improving feature steering across model scale, layer depth, and site under a paired significance protocol. That cell of the design space stands open.

This thesis occupies that cell. Its contribution is not the two-stage principle, which CorrSteer shares, nor feature steering itself, which the demonstrations of Section 5.2.2 established, but their combination with an evidence protocol. Discovery is labelled by task outcome, every candidate is validated causally in isolation on difficulty-matched disjoint data, and the assembled intervention amplifies and suppresses under a clamp. On a holdout no earlier phase has touched, the verdict comes from McNemar's test and K-fold stability. The study repeats this across model scale, layer depth, and site with matched public SAEs. Under the training-free constraint, no prior work assembles that combination.

6 Conclusion

This chapter summarises what was done and what was found (Section 6.1), then marks the boundaries of those findings and the work that follows from them (Section 6.2).

6.1 Summary of Contributions

The central question of this thesis asked whether SAE-based feature steering can produce measurable and statistically significant accuracy improvements on multiple-choice benchmarks. To make the question testable, the first contribution, the five-phase pipeline of Chapter 3, turns a frozen model, a pre-trained SAE, and a labelled benchmark into one measured intervention, using nothing but forward passes, counting, and selection. Two safeguards discipline the result. Every candidate feature is validated causally, by moving it alone and measuring whether answers improve, and the questions that build an intervention are never the ones that judge it. A reported gain therefore cannot echo its own search.

The second contribution is the study of Chapter 4. Its 98 experiments pair fifteen models, spanning 70 M to 8 B parameters, with 52 public SAEs from eight providers, and each runs the full pipeline against CommonsenseQA or ARC-Challenge, together a workload of 631 GPU-hours. The three design axes are model scale, layer depth, and intervention site, with the public SAE inventory setting how far each can be varied as a controlled factor. Every intervention is additionally scored on three benchmarks that played no part in building it. All 98 experiments are reported, none dropped on its outcome, and the code behind them is public [118].

The study returned a conditional yes. Fifteen of the 98 experiments reached significance. Seven of the fifteen withstand every further check of Section 4.4, with gains of 0.48 to 3.70 percentage points, and all seven sit on `meta-llama/Llama-3.2-1B-Instruct` or `google/gemma-2-2b`, models of 1 B and 2 B parameters. The strongest of them lifts `meta-llama/Llama-3.2-1B-Instruct` on CommonsenseQA from 44.7% to 48.4%. Outside these two models the count of significant results is indistinguishable from chance. That concentration is what makes the answer a conditional yes.

Three further findings stand out. Interventions that pass their own holdout tend to help elsewhere, eleven of the fifteen significant experiments transferring significantly to benchmarks they never saw, sometimes by more than they gained at home. Layer depth and intervention site, by contrast, showed no measurable preference in any controlled comparison, so both can follow whichever public SAE is available. Causal validation earns its keep on new questions. When an intervention meets questions that played no part in choosing it, one built by correlation alone loses about four times as much of its apparent gain as one built from causally validated features. Larger models, or the SAEs trained on them, let more features survive that validation, yet larger gains did not follow.

Assembled from these findings, the third contribution is the operating envelope, a map of where the method helps, where it fails, and where it cannot yet be judged (Section 4.5). Its interior is a model of 1 B to 2 B parameters whose difficulty score genuinely grades the questions, whose predictions are not dominated by a single answer letter, and whose baseline lies above chance. To these conditions the envelope adds a holdout large enough to give the paired test power and a causal validation that keeps at least one candidate, since experiments whose validation kept nothing never reached significance. Layer depth and site appear in no condition. The edges mark where the measurement gives out before the method does. At 410 M and below every experiment breaks at least one condition, and at 8 B the difficulty interval retains only 4% to 9% of a benchmark, so what lies beyond the envelope is open rather than answered in the negative. No prior work provides a characterisation of this kind (Section 5.3).

What makes the result notable is how little it requires. Public models, public SAEs, and no gradient-based training at any stage produced improvements that survived every attempt to attribute them to chance, to the measurement, or to the benchmark each was built on. Where feature steering helps no longer needs to be assumed. The conditions are stated, and each can be tested with the protocol that produced them.

6.2 Limitations and Future Work

Four limitations bound these claims, and each is paired with the future work that would move past it.

A first limitation concerns the concentration of the evidence. All seven surviving results come from two models, and four of them are judged on the same holdout questions. Future work with a larger compute budget could test more strengths, other difficulty intervals, and further parameter variations. A confirmatory study would then set the operating envelope in advance as a hypothesis and test it on more models, SAEs, and benchmarks.

A second limitation is the evaluation format. Accuracy is read from answer-letter logits at the final prompt position, a convention that is deterministic and inexpensive but gives no credit to an answer the model would reach only through intermediate reasoning (Section 2.4). All benchmarks in the study are, moreover, English-language and multiple-choice. The matching future work carries the intervention into generation, steering while a chain of thought is produced and scoring the resulting text. Once an output can be graded correct or incorrect, the paired test transfers to that setting.

A third limitation lies in the material. Every SAE was taken as published, its dictionary quality entering the result as an uncontrolled property, and since no release spans models, the effect of scale is never separated from the SAE trained at it. The site axis likewise stops at two levels, because no comparable dictionary exists at the attention write (Section 4.1.2). One SAE family trained under a fixed recipe, across a scale ladder and at all three sites, would separate model from dictionary and close both gaps.

A fourth limitation concerns deployment. Building an intervention requires labelled questions in every phase. Once built, the vector is applied to every input, even to questions the model already answers correctly, where an answer can only get worse, not better. A label-free difficulty estimate at inference time, or another mechanism that decides input by input when the intervention should act and when not, could therefore increase the gains further.

As public SAE coverage extends along scale, depth, and site, the comparisons this study could not run become possible. Whether feature steering matures into a dependable instrument for improving deployed language models is accordingly an empirical question, and this thesis contributes the protocol under which it can be answered.

Bibliography

[1] A. Chatterji *et al.*, "How People Use ChatGPT," National Bureau of Economic Research, Working Paper 34255, 2025. [Online]. Available: https://www.nber.org/papers/w34255

[2] A. Bick, A. Blandin, and D. J. Deming, "The Rapid Adoption of Generative AI," National Bureau of Economic Research, Working Paper 32966, 2024. [Online]. Available: https://www.nber.org/papers/w32966

[3] T. Eloundou, S. Manning, P. Mishkin, and D. Rock, "GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models," *arXiv:2303.10130*, 2023, [Online]. Available: https://arxiv.org/abs/2303.10130

[4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI, technical report, 2019.

[5] A. Grattafiori *et al.*, "The Llama 3 Herd of Models," *arXiv:2407.21783*, 2024, [Online]. Available: https://arxiv.org/abs/2407.21783

[6] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, 2020.

[7] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *The Tenth International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[8] L. Ouyang *et al.*, "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://arxiv.org/abs/2203.02155

[9] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.18290

[10] N. Subramani, N. Suresh, and M. E. Peters, "Extracting Latent Steering Vectors from Pretrained Language Models," in *Findings of the Association for Computational Linguistics: ACL 2022*, 2022. [Online]. Available: https://arxiv.org/abs/2205.05124

[11] A. Zou *et al.*, "Representation Engineering: A Top-Down Approach to AI Transparency," *arXiv:2310.01405*, 2023, [Online]. Available: https://arxiv.org/abs/2310.01405

[12] R. Hendel, M. Geva, and A. Globerson, "In-Context Learning Creates Task Vectors," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. [Online]. Available: https://arxiv.org/abs/2310.15916

[13] E. Todd, M. L. Li, A. Sen Sharma, A. Mueller, B. C. Wallace, and D. Bau, "Function Vectors in Large Language Models," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[14] S. Liu, H. Ye, L. Xing, and J. Zou, "In-Context Vectors: Making In Context Learning More Effective and Controllable Through Latent Space Steering," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, in PMLR, vol. 235. 2024, pp. 32287–32307. [Online]. Available: https://arxiv.org/abs/2311.06668

[15] T. van der Weij, M. Poesio, and N. Schoots, "Extending Activation Steering to Broad Skills and Multiple Behaviours," *arXiv:2403.05767*, 2024, [Online]. Available: https://arxiv.org/abs/2403.05767

[16] B. W. Lee *et al.*, "Programming Refusal with Conditional Activation Steering," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2409.05907

[17] A. M. Turner *et al.*, "Activation Addition: Steering Language Models Without Optimization," *arXiv:2308.10248v4*, 2023, [Online]. Available: https://arxiv.org/abs/2308.10248v4

[18] N. Panickssery, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. M. Turner, "Steering Llama 2 via Contrastive Activation Addition," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 15504–15522. [Online]. Available: https://arxiv.org/abs/2312.06681

[19] A. Arditi *et al.*, "Refusal in Language Models Is Mediated by a Single Direction," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: https://arxiv.org/abs/2406.11717

[20] D. Tan *et al.*, "Analysing the Generalisation and Reliability of Steering Vectors," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: https://arxiv.org/abs/2407.12404

[21] I. Pres, L. Ruis, E. S. Lubana, and D. Krueger, "Towards Reliable Evaluation of Behavior Steering Interventions in LLMs," in *NeurIPS 2024 Workshop on Foundation Model Interventions*, 2024.

[22] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter, "Zoom In: An Introduction to Circuits," *Distill*, 2020, [Online]. Available: https://distill.pub/2020/circuits/zoom-in

[23] T. Bricken *et al.*, "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning," *Transformer Circuits Thread*, 2023, [Online]. Available: https://transformer-circuits.pub/2023/monosemantic-features/index.html

[24] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, "Sparse Autoencoders Find Highly Interpretable Features in Language Models," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2309.08600

[25] A. Templeton *et al.*, "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet," *Transformer Circuits Thread*, 2024, [Online]. Available: https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html

[26] Anthropic, "Mapping the Mind of a Large Language Model," *Anthropic Research*, 2024, [Online]. Available: https://www.anthropic.com/research/mapping-mind-language-model

[27] N. Elhage *et al.*, "Toy Models of Superposition," *Transformer Circuits Thread*, 2022, [Online]. Available: https://transformer-circuits.pub/2022/toy_model/index.html

[28] T. Lieberum *et al.*, "Gemma Scope: Open Sparse Autoencoders Everywhere All At Once on Gemma 2," *arXiv:2408.05147*, 2024, [Online]. Available: https://arxiv.org/abs/2408.05147

[29] D. Balsam, T. McGrath, L. Gorton, N. Nguyen, M. Deng, and E. Ho, "Announcing Open-Source SAEs for Llama 3.3 70B and Llama 3.1 8B," *Goodfire Research*, 2025, [Online]. Available: https://www.goodfire.com/blog/sae-open-source-announcement

[30] A. Joshi, C. Cintas, and S. Speakman, "Enabling Precise Topic Alignment in Large Language Models via Sparse Autoencoders," *arXiv:2506.12576*, 2025, [Online]. Available: https://arxiv.org/abs/2506.12576

[31] K. O'Brien *et al.*, "Steering Language Model Refusal with Sparse Autoencoders," *arXiv:2411.11296*, 2024, [Online]. Available: https://arxiv.org/abs/2411.11296

[32] Y. Fang *et al.*, "Controllable LLM Reasoning via Sparse Autoencoder-Based Steering," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 21290–21305. [Online]. Available: https://arxiv.org/abs/2601.03595

[33] Z. Wu *et al.*, "AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders," *arXiv:2501.17148*, 2025, [Online]. Available: https://arxiv.org/abs/2501.17148

[34] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, "Inference-Time Intervention: Eliciting Truthful Answers from a Language Model," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2306.03341

[35] A. Galichin *et al.*, "I Have Covered All the Bases Here: Interpreting Reasoning Features in Large Language Models via Sparse Autoencoders," *arXiv:2503.18878*, 2025, [Online]. Available: https://arxiv.org/abs/2503.18878

[36] Z. Li, X. Wang, Y. Yang, Z. Yao, H. Xiong, and M. Du, "Feature Extraction and Steering for Enhanced Chain-of-Thought Reasoning in Language Models," *arXiv:2505.15634*, 2025, [Online]. Available: https://arxiv.org/abs/2505.15634

[37] S. Cho, Z. Wu, and A. Koshiyama, "CorrSteer: Generation-Time LLM Steering via Correlated Sparse Autoencoder Features," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. [Online]. Available: https://arxiv.org/abs/2508.12535

[38] A. Makelov, G. Lange, and N. Nanda, "Towards Principled Evaluations of Sparse Autoencoders for Interpretability and Control," *arXiv:2405.08366*, 2024, [Online]. Available: https://arxiv.org/abs/2405.08366

[39] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017.

[40] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI, technical report, 2018.

[41] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171–4186.

[42] C. Raffel *et al.*, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[43] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv:1607.06450*, 2016, [Online]. Available: https://arxiv.org/abs/1607.06450

[44] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in *Advances in Neural Information Processing Systems*, 2019.

[45] N. Elhage *et al.*, "A Mathematical Framework for Transformer Circuits," *Transformer Circuits Thread*, 2021, [Online]. Available: https://transformer-circuits.pub/2021/framework/index.html

[46] S. Biderman *et al.*, "Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, in PMLR, vol. 202. 2023. [Online]. Available: https://arxiv.org/abs/2304.01373

[47] J. Ferrando, G. Sarti, A. Bisazza, and M. R. Costa-jussà, "A Primer on the Inner Workings of Transformer-based Language Models," *arXiv:2405.00208*, 2024, [Online]. Available: https://arxiv.org/abs/2405.00208

[48] L. Sharkey *et al.*, "Open Problems in Mechanistic Interpretability," *arXiv:2501.16496*, 2025, [Online]. Available: https://arxiv.org/abs/2501.16496

[49] T. Mikolov, W.-t. Yih, and G. Zweig, "Linguistic Regularities in Continuous Space Word Representations," in *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2013, pp. 746–751.

[50] K. Park, Y. J. Choe, and V. Veitch, "The Linear Representation Hypothesis and the Geometry of Large Language Models," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, in PMLR, vol. 235. 2024, pp. 39643–39666. [Online]. Available: https://arxiv.org/abs/2311.03658

[51] L. Gao *et al.*, "Scaling and Evaluating Sparse Autoencoders," *arXiv:2406.04093*, 2024, [Online]. Available: https://arxiv.org/abs/2406.04093

[52] S. Rajamanoharan *et al.*, "Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders," *arXiv:2407.14435*, 2024, [Online]. Available: https://arxiv.org/abs/2407.14435

[53] B. Bussmann, P. Leask, and N. Nanda, "BatchTopK Sparse Autoencoders," *arXiv:2412.06410*, 2024, [Online]. Available: https://arxiv.org/abs/2412.06410

[54] A. Karvonen *et al.*, "SAEBench: A Comprehensive Benchmark for Sparse Autoencoders in Language Model Interpretability," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, in PMLR, vol. 267. 2025, pp. 29223–29264. [Online]. Available: https://arxiv.org/abs/2503.09532

[55] J. Lin, "Neuronpedia: Interactive Reference and Tooling for Analyzing Neural Networks," Software, 2023. [Online]. Available: https://www.neuronpedia.org/

[56] D. Chanin, J. Wilken-Smith, T. Dulka, H. Bhatnagar, S. Golechha, and J. Bloom, "A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. [Online]. Available: https://arxiv.org/abs/2409.14507

[57] P. Leask *et al.*, "Sparse Autoencoders Do Not Find Canonical Units of Analysis," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2502.04878

[58] C. McDougall, A. Conmy, J. Kramár, T. Lieberum, S. Rajamanoharan, and N. Nanda, "Gemma Scope 2: Technical Paper," Google DeepMind, technical report, 2025. [Online]. Available: https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/gemma-scope-2-helping-the-ai-safety-community-deepen-understanding-of-complex-language-model-behavior/Gemma_Scope_2_Technical_Paper.pdf

[59] J. Robinson, C. M. Rytting, and D. Wingate, "Leveraging Large Language Models for Multiple Choice Question Answering," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.12353

[60] L. Gao *et al.*, "A Framework for Few-Shot Language Model Evaluation," *Zenodo*, 2023, doi: 10.5281/zenodo.10256836.

[61] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, "Large Language Models Are Not Robust Multiple Choice Selectors," *arXiv:2309.03882*, 2023, [Online]. Available: https://arxiv.org/abs/2309.03882

[62] P. Pezeshkpour and E. Hruschka, "Large Language Models Sensitivity to The Order of Options in Multiple-Choice Questions," *arXiv:2308.11483*, 2023, [Online]. Available: https://arxiv.org/abs/2308.11483

[63] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.

[64] Student (Gosset, William Sealy), "The Probable Error of a Mean," *Biometrika*, vol. 6, no. 1, pp. 1–25, 1908, doi: 10.1093/biomet/6.1.1.

[65] O. J. Dunn, "Multiple Comparisons Among Means," *Journal of the American Statistical Association*, vol. 56, no. 293, pp. 52–64, 1961, doi: 10.1080/01621459.1961.10482090.

[66] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995, doi: 10.1111/j.2517-6161.1995.tb02031.x.

[67] S. Rajamanoharan *et al.*, "Improving Dictionary Learning with Gated Sparse Autoencoders," *arXiv:2404.16014*, 2024, [Online]. Available: https://arxiv.org/abs/2404.16014

[68] B. Wright and L. Sharkey, "Addressing Feature Suppression in SAEs," *AI Alignment Forum*, 2024, [Online]. Available: https://www.alignmentforum.org/posts/3JuSjTZyMzaSeTxKk/addressing-feature-suppression-in-saes

[69] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval Estimation for a Binomial Proportion," *Statistical Science*, vol. 16, no. 2, pp. 101–133, 2001, doi: 10.1214/ss/1009213286.

[70] A. Gelman and P. N. Price, "All Maps of Parameter Estimates Are Misleading," *Statistics in Medicine*, vol. 18, no. 23, pp. 3221–3234, 1999, doi: 10.1002/(SICI)1097-0258(19991215)18:23<3221::AID-SIM312>3.0.CO;2-M.

[71] G. C. Cawley and N. L. C. Talbot, "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010.

[72] A. Talmor, J. Herzig, N. Lourie, and J. Berant, "CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4149–4158. [Online]. Available: https://arxiv.org/abs/1811.00937

[73] P. Clark *et al.*, "Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," *arXiv:1803.05457*, 2018, [Online]. Available: https://arxiv.org/abs/1803.05457

[74] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, "Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 2381–2391. [Online]. Available: https://arxiv.org/abs/1809.02789

[75] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 4791–4800. [Online]. Available: https://arxiv.org/abs/1905.07830

[76] R. Speer, J. Chin, and C. Havasi, "ConceptNet 5.5: An Open Multilingual Graph of General Knowledge," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence (AAAI)*, 2017, pp. 4444–4451. doi: 10.1609/aaai.v31i1.11164.

[77] C. Deng, Y. Zhao, X. Tang, M. Gerstein, and A. Cohan, "Investigating Data Contamination in Modern Benchmarks for Large Language Models," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 8706–8719. doi: 10.18653/v1/2024.naacl-long.482.

[78] L. Ben Allal *et al.*, "SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model," *arXiv:2502.02737*, 2025, [Online]. Available: https://arxiv.org/abs/2502.02737

[79] Gemma Team, "Gemma: Open Models Based on Gemini Research and Technology," *arXiv:2403.08295*, 2024, [Online]. Available: https://arxiv.org/abs/2403.08295

[80] Gemma Team, "Gemma 2: Improving Open Language Models at a Practical Size," *arXiv:2408.00118*, 2024, [Online]. Available: https://arxiv.org/abs/2408.00118

[81] Gemma Team, "Gemma 3 Technical Report," *arXiv:2503.19786*, 2025, [Online]. Available: https://arxiv.org/abs/2503.19786

[82] Qwen Team, "Qwen2.5 Technical Report," *arXiv:2412.15115*, 2024, [Online]. Available: https://arxiv.org/abs/2412.15115

[83] D. Guo *et al.*, "DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning," *Nature*, vol. 645, pp. 633–638, 2025, doi: 10.1038/s41586-025-09422-z.

[84] C. Spearman, "The Proof and Measurement of Association between Two Things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904, doi: 10.2307/1412159.

[85] W. J. Dixon and A. M. Mood, "The Statistical Sign Test," *Journal of the American Statistical Association*, vol. 41, no. 236, pp. 557–566, 1946, doi: 10.1080/01621459.1946.10501898.

[86] D. Chanin, T. Dulka, and A. Garriga-Alonso, "Feature Hedging: Correlated Features Break Narrow Sparse Autoencoders," *arXiv:2505.11756*, 2025, [Online]. Available: https://arxiv.org/abs/2505.11756

[87] D. Chanin, T. Dulka, and A. Garriga-Alonso, "A Bunch of Matryoshka SAEs," *LessWrong*, 2025, [Online]. Available: https://www.lesswrong.com/posts/3EQQpB7i2o3nu2Di6/a-bunch-of-matryoshka-saes

[88] R. A. Fisher, "The Logic of Inductive Inference," *Journal of the Royal Statistical Society*, vol. 98, no. 1, pp. 39–82, 1935, doi: 10.2307/2342435.

[89] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945, doi: 10.2307/3001968.

[90] H. B. Mann and D. R. Whitney, "On a Test of Whether One of Two Random Variables Is Stochastically Larger than the Other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947, doi: 10.1214/aoms/1177730491.

[91] Y. Benjamini and D. Yekutieli, "The Control of the False Discovery Rate in Multiple Testing under Dependency," *The Annals of Statistics*, vol. 29, no. 4, pp. 1165–1188, 2001, doi: 10.1214/aos/1013699998.

[92] C. J. Clopper and E. S. Pearson, "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934, doi: 10.1093/biomet/26.4.404.

[93] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep Reinforcement Learning from Human Preferences," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4299–4307. [Online]. Available: https://arxiv.org/abs/1706.03741

[94] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv:1909.08593*, 2019, [Online]. Available: https://arxiv.org/abs/1909.08593

[95] N. Stiennon *et al.*, "Learning to Summarize from Human Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://arxiv.org/abs/2009.01325

[96] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv:1707.06347*, 2017, [Online]. Available: https://arxiv.org/abs/1707.06347

[97] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and Editing Factual Associations in GPT," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://arxiv.org/abs/2202.05262

[98] K. Meng, A. Sen Sharma, A. Andonian, Y. Belinkov, and D. Bau, "Mass-Editing Memory in a Transformer," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.07229

[99] Z. Wu *et al.*, "ReFT: Representation Finetuning for Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: https://arxiv.org/abs/2404.03592

[100] Z. Wu, Q. Yu, A. Arora, C. D. Manning, and C. Potts, "Improved Representation Steering for Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[101] J. Braun, C. Eickhoff, D. Krueger, S. A. Bahrainian, and D. Krasheninnikov, "Understanding (Un)Reliability of Steering Vectors in Language Models," in *ICLR 2025 Workshop on Bidirectional Human-AI Alignment*, 2025.

[102] R. Bayat, A. Rahimi-Kalahroudi, M. Pezeshki, S. Chandar, and P. Vincent, "Steering Large Language Model Activations in Sparse Spaces," in *Conference on Language Modeling (COLM)*, 2025.

[103] S. Chalnev, M. Siu, and A. Conmy, "Improving Steering Vectors by Targeting Sparse Autoencoder Features," *arXiv:2411.02193*, 2024, [Online]. Available: https://arxiv.org/abs/2411.02193

[104] S. Soo, C. Guang, C. Balaganesh, W. Teng, G. Tan, and M. Yan, "Interpretable Steering of Large Language Models with Feature Guided Activation Additions," in *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025. [Online]. Available: https://arxiv.org/abs/2501.09929

[105] D. Arad, A. Mueller, and Y. Belinkov, "SAEs Are Good for Steering – If You Select the Right Features," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 10252–10270. doi: 10.18653/v1/2025.emnlp-main.519.

[106] Y. Zhao *et al.*, "Steering Knowledge Selection Behaviours in LLMs via SAE-Based Representation Engineering," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*

(Volume 1: Long Papers), 2025, pp. 5117–5136. [Online]. Available: https://arxiv.org/abs/2410.15999

[107] A. Muhamed, J. Bonato, M. Diab, and V. Smith, "SAEs Can Improve Unlearning: Dynamic Sparse Autoencoder Guardrails for Precision Unlearning in LLMs," *arXiv:2504.08192*, 2025, [Online]. Available: https://arxiv.org/abs/2504.08192

[108] E. Durmus *et al.*, "Evaluating Feature Steering: A Case Study in Mitigating Social Biases," *Anthropic Research*, 2024, [Online]. Available: https://www.anthropic.com/research/evaluating-feature-steering

[109] A. Stolfo, V. Balachandran, S. Yousefi, E. Horvitz, and B. Nushi, "Improving Instruction-Following in Language Models through Activation Steering," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[110] Z. Cheng *et al.*, "Steering When Necessary: Flexible Steering Large Language Models with Backtracking," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[111] C. Venhoff, I. Arcuschin, P. Torr, A. Conmy, and N. Nanda, "Understanding Reasoning in Thinking Language Models via Steering Vectors," in *ICLR 2025 Workshop on Reasoning and Planning for LLMs*, 2025.

[112] S. Azizi, E. Baghaei Potraghloo, S. Kundu, and M. Pedram, "Activation Steering for Chain-of-Thought Compression," in *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025.

[113] Y. Huang, H. Chen, S. Ruan, Y. Zhang, X. Wei, and Y. Dong, "Mitigating Overthinking in Large Reasoning Models via Manifold Steering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[114] Z. Zhao *et al.*, "Activation Control for Efficiently Eliciting Long Chain-of-Thought Ability of Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[115] J. Billa, "Predicting Where Steering Vectors Succeed," *arXiv:2604.15557*, 2026, [Online]. Available: https://arxiv.org/abs/2604.15557

[116] S. Gadgil, C. Lin, and S.-I. Lee, "Where to Steer: Input-Dependent Layer Selection for Steering Improves LLM Alignment," *arXiv:2604.03867*, 2026, [Online]. Available: https://arxiv.org/abs/2604.03867

[117] N. Das and M. Gaur, "Mechanistic Steering of LLMs Reveals Layer-wise Feature Vulnerabilities in Adversarial Settings," *arXiv:2604.23130v1*, 2026, [Online]. Available: https://arxiv.org/abs/2604.23130v1

[118] J. Brüggemann, "master-sae-experiments: Experiment Framework for SAE-Based Feature Steering," GitHub, Software repository, 2026. [Online]. Available: https://github.com/JonasBrue/master-sae-experiments

A SAE Inventory

Table 12 complements the model overview in Table 2 by grouping the 52 SAE checkpoints used across the 98 experiments into 27 release entries from 24 public repositories. Each entry represents one release recipe and may therefore cover checkpoints used at multiple sites or layers.

The *Architecture* column describes each release's training architecture where documented, which may differ from the activation rule executed by the pipeline. The andyrdt entry is executed with per-sample TopK, whereas the three chanind Matryoshka entries use their serialized JumpReLU thresholds.

SAE repository	Architecture	Width	Expansion factor	k	Site	Layers used	Experiments	Resolved commit
EleutherAI/pythia-70m								
EleutherAI/sae-pythia-70m-32k	TopK	32,768	64.0	16	both	2, 5	8	63999ba
HuggingFaceTB/SmolLM2-135M								
EleutherAI/sae-SmolLM2-135M-64x	TopK	36,864	64.0	32	mlp_out	15, 27	4	57ea2cb
EleutherAI/pythia-160m								
EleutherAI/Pythia-160m-SAE-k64-32k	TopK	32,768	42.7	64	mlp_out	6	1	325a8f8
EleutherAI/sae-pythia-160m-32k	TopK	32,768	42.7	32	resid_post	5, 11	4	2046768
EleutherAI/pythia-160m-deduped								
EleutherAI/sae-pythia-160m-deduped-32k	TopK	65,536	85.3	32	both	5, 11	8	b6c77b8
google/gemma-3-270m								
google/gemma-scope-2-270m-pt	JumpReLU	16,384	25.6	—	both	9, 15	8	b218cd5
EleutherAI/pythia-410m								
EleutherAI/sae-pythia-410m-65k	TopK	65,536	64.0	32	mlp_out	11, 23	4	8cd7db3
google/gemma-3-1b-pt								
chanind/gemma-3-1b-batch-topk-matryoshka-saes-w-32k-l0-40	BatchTopK	32,768	28.4	—	resid_post	12, 24	3	2375788
google/gemma-scope-2-1b-pt	JumpReLU	16,384	14.2	—	mlp_out resid_post	13 13, 22	3	b738dc0
meta-llama/Llama-3.2-1B								
chanind/sae-llama-3.2-1b-res	ReLU	16,384	8.0	—	resid_post	7, 8	4	6395ab3
seonglae/Llama-3.2-1B-sae	TopK	14,336	7.0	48	resid_post	11	2	257e17b
meta-llama/Llama-3.2-1B-Instruct								
EleutherAI/sae-Llama-3.2-1B-131k	TopK	131,072	64.0	32	mlp_out	7, 15	4	c0d4d78
qresearch/Llama-3.2-1B-Instruct-SAE-l9	ReLU (assumed)[4]	32,768	16.0	—	resid_post	9	2	4fd505e
deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B								
EleutherAI/sae-DeepSeek-R1-Distill-Qwen-1.5B-65k	TopK	65,536	42.7	32	mlp_out	13, 27	4	9fbac1d
google/gemma-2b								
jbloom/Gemma-2b-Residual-Stream-SAEs (anthropic)	ReLU	16,384	8.0	—	resid_post	0	2	2e64e91
jbloom/Gemma-2b-Residual-Stream-SAEs (anthropic_fast_lr)	ReLU	16,384	8.0	—	resid_post	6	2	2e64e91
jbloom/Gemma-2b-Residual-Stream-SAEs (plain)	ReLU	16,384	8.0	—	resid_post	10, 17	4	2e64e91
google/gemma-2b-it								
jbloom/Gemma-2b-IT-Residual-Stream-SAEs	ReLU	16,384	8.0	—	resid_post	12	2	16e463a
google/gemma-2-2b								
Ale21m9/gemma-2b-finetuned-sae	JumpReLU	16,384	7.1	—	resid_post	19	2	869aa9c
chanind/gemma-2-2b-batch-topk-matryoshka-saes-w-32k-l0-40 (snap)	BatchTopK	32,768	14.2	—	resid_post	12, 24	4	4c28e79
chanind/gemma-2-2b-batch-topk-matryoshka-saes-w-32k-l0-40 (standard)	BatchTopK	32,768	14.2	—	resid_post	12, 24	4	4c28e79
chanind/sae-gemma-2-2b-tied-topk	TopK	18,432	8.0	50	resid_post	1	2	16d13b5
google/gemma-scope-2b-pt-mlp	JumpReLU	16,384	7.1	—	mlp_out	12, 19	4	4c9df61
google/gemma-scope-2b-pt-res	JumpReLU	16,384	7.1	—	resid_post	12, 19	4	fd571b4
seonglae/gemma-2-2b-sae	TopK	18,432	8.0	64	resid_post	19	2	8226af8
meta-llama/Llama-3.1-8B-Instruct								
andyrdt/saes-llama-3.1-8b-instruct	BatchTopK	131,072	32.0	64	resid_post	15, 27	3	68226a9
meta-llama/Meta-Llama-3-8B-Instruct								
EleutherAI/sae-llama-3-8b-32x	TopK	131,072	32.0	192	resid_post	15, 30	4	3292654

Table 12: SAE release recipes used in the 98 experiments, grouped by the language model to which each SAE was applied and ordered by increasing model size.[5]

One additional loader deviation affects `chanind/sae-llama-3.2-1b-res`. Its release configuration specifies `apply_b_dec_to_input = true`, but the ReLU loader does not subtract the decoder bias before encoding. The four experiments at layers 7 and 8 therefore use uncentred activations. Across all releases, SAE parameters are cast to `bfloat16` when loaded.

Two EleutherAI releases were applied to instruction-tuned models although they were trained on the corresponding base models. `EleutherAI/sae-Llama-3.2-1B-131k` was applied to `meta-llama/Llama-3.2-1B-Instruct`, and `EleutherAI/sae-llama-3-8b-32x` to `meta-llama/Meta-Llama-3-8B-Instruct`. These experiments therefore apply each SAE to activations from a model other than its training model.

[4]The qresearch release identifies neither the SAE architecture nor its training site. The checkpoint was loaded through the ReLU implementation and applied at `resid_post`. The architecture is therefore marked as assumed, whereas the reported site denotes the intervention setting.

[5]The two seonglae rows are released as TransformerLens `hook_resid_pre` checkpoints. At block M this hook taps the block's input, which is the previous block's residual-stream output, the tensor this table calls `resid_post`. Only the indexing differs. The published `blocks.M` runs one higher than the zero-based layer whose output the checkpoint decomposes, so the table reports the hooked layer, 11 and 19, rather than the 12 and 20 appearing in the checkpoint paths.

B Benchmark Prompts

Section 4.1.1 describes the prompt format in prose. One function rendered every question of all four benchmarks, and Table 13 gives the two lines each prompt opens with.

Benchmark	Instruction line	Restriction line
CommonsenseQA	`Answer the following question.`	`Respond with ONLY the letter of the correct answer (A, B, C, D, or E).`
ARC-Challenge	`Answer the following science question.`	`Respond with ONLY the letter of the correct answer (A, B, C, or D).`
OpenBookQA	`Answer the following science question.`	`Respond with ONLY the letter of the correct answer (A, B, C, or D).`
HellaSwag	`Complete the following sentence with the most plausible ending.`	`Respond with ONLY the letter of the best ending (A, B, C, or D).`

Table 13: The opening lines of the four prompts.

The first question of the CommonsenseQA pool reaches every model as the text below.

```
Answer the following question.
Respond with ONLY the letter of the correct answer (A, B, C, D, or E).

Question: The sanctions against the school were a punishing blow, and they
seemed to what the efforts the school had made to change?

A. ignore
B. enforce
C. authoritarian
D. yell at
E. avoid

Answer:
```

Where a tokeniser ships a chat template, the prompt is placed in a single user message with a generation prompt appended. Tokenisation truncates at 2,048 tokens, a limit no item of the four benchmarks reaches.

C Experiment Results

Chapter 4 reports the study in aggregate, and Table 14 prints the 98 experiments behind it, one row apiece.

Table 14: The 98 experiments, grouped by model. Appendix A resolves each *SAE release* to its repository and commit. *Survival* gives the shortlisted features that survived Phase 2 against those tested, and *Rank* to α the configuration Phase 3 selected, whose features it ranked by frequency difference (*freq.*) or by causal effect (*causal*). *Selection* and *Gain* are accuracy changes in percentage points on the selection subset and on the holdout, with *Flips* counting the holdout questions improved and worsened. The last three columns are the transfer benchmarks, where *Other* is ARC-Challenge or CommonsenseQA, whichever is not its primary benchmark, *OBQA* is OpenBookQA, and *HS* is HellaSwag. p is the one-sided binomial of Phase 4 for the holdout gain, and a gain is printed bold where its own such test falls below 0.05.

Layer	Site	SAE release	Width	Bench.	Holdout	Baseline	Survival	Rank	k_+/k_-	α	Selection	Gain	Flips	p	Other	OBQA	HS
EleutherAI/pythia-70m (70M)																	
2	mlp_out	eleutherai	32,768	ARC	291	21.3%	0/40	freq.	10/0	1.0	+0.68	0.00	+0 / −0	1.000	0.00	−0.02	−0.01
2	resid_post	eleutherai	32,768	ARC	291	21.3%	0/83	causal	3/0	0.5	0.00	0.00	+0 / −0	1.000	0.00	0.00	0.00
5	mlp_out	eleutherai	32,768	ARC	291	21.3%	0/100	causal	3/0	0.5	0.00	0.00	+0 / −0	1.000	0.00	0.00	0.00
5	resid_post	eleutherai	32,768	ARC	291	21.3%	0/100	causal	3/0	0.5	0.00	0.00	+0 / −0	1.000	0.00	0.00	0.00
2	mlp_out	eleutherai	32,768	CSQA	1,380	21.6%	0/27	freq.	15/3	1.5	+0.43	−0.43	+14 / −20	0.885	+0.17	−0.24	+0.18
2	resid_post	eleutherai	32,768	CSQA	1,380	21.6%	0/62	causal	3/0	0.5	0.00	0.00	+0 / −0	1.000	0.00	0.00	0.00
5	mlp_out	eleutherai	32,768	CSQA	1,380	21.6%	0/100	causal	3/0	0.5	0.00	0.00	+0 / −0	1.000	0.00	0.00	0.00
5	resid_post	eleutherai	32,768	CSQA	1,380	21.6%	0/100	freq.	10/0	1.5	+0.07	0.00	+0 / −0	1.000	0.00	−0.02	0.00
HuggingFaceTB/SmolLM2-135M (135M)																	
15	mlp_out	eleutherai	36,864	ARC	357	23.2%	48/100	freq.	15/5	1.0	+2.52	+0.84	+6 / −3	0.254	−0.21	+0.33	−0.02
27	mlp_out	eleutherai	36,864	ARC	357	23.2%	32/100	causal	10/3	1.0	+1.40	+0.56	+4 / −2	0.344	−0.05	+0.16	−0.01
15	mlp_out	eleutherai	36,864	CSQA	2,761	11.4%	59/93	freq.	15/0	0.5	+0.33	**+0.36**	+14 / −4	0.015	+0.17	+0.20	−0.01
27	mlp_out	eleutherai	36,864	CSQA	2,761	11.4%	58/100	freq.	5/0	1.5	+0.36	0.00	+8 / −8	0.598	+0.31	−0.42	−0.01
EleutherAI/pythia-160m (160M)																	
5	resid_post	eleutherai	32,768	ARC	840	20.0%	1/96	freq.	5/0	1.5	+0.12	−0.24	+1 / −3	0.938	+0.02	+0.04	−0.07
11	resid_post	eleutherai	32,768	ARC	840	20.0%	0/100	causal	3/0	0.5	0.00	0.00	+0 / −0	1.000	0.00	0.00	0.00
5	resid_post	eleutherai	32,768	CSQA	3,728	17.3%	7/88	freq.	3/0	1.5	+0.19	−0.05	+5 / −7	0.806	−0.17	+0.07	−0.05
6	mlp_out	eleutherai	32,768	CSQA	3,538	17.1%	1/100	freq.	5/5	0.5	+0.20	−0.11	+0 / −4	1.000	+0.17	+0.09	−0.05
11	resid_post	eleutherai	32,768	CSQA	3,728	17.3%	0/100	freq.	15/3	1.5	+0.19	−0.11	+36 / −40	0.717	−0.70	−0.13	0.00
EleutherAI/pythia-160m-deduped (160M)																	
5	mlp_out	eleutherai	65,536	ARC	806	20.8%	28/100	causal	3/0	0.5	+0.25	+0.12	+1 / −0	0.500	−0.01	+0.05	+0.04
5	resid_post	eleutherai	65,536	ARC	806	20.8%	28/100	causal	10/0	0.5	+0.25	0.00	+1 / −1	0.750	+0.01	+0.04	+0.05
11	mlp_out	eleutherai	65,536	ARC	806	20.8%	1/100	freq.	7/0	1.5	+0.12	+0.12	+2 / −1	0.500	−0.01	0.00	0.00
11	resid_post	eleutherai	65,536	ARC	806	20.8%	7/100	freq.	5/5	1.5	+0.50	+0.12	+1 / −0	0.500	+0.05	−0.04	+0.02
5	mlp_out	eleutherai	65,536	CSQA	3,211	17.6%	17/90	causal	3/3	1.0	0.00	−0.06	+5 / −7	0.806	+0.13	0.00	+0.08
5	resid_post	eleutherai	65,536	CSQA	3,211	17.6%	24/88	freq.	10/5	1.5	+0.12	+0.06	+17 / −15	0.430	+0.26	−0.02	−0.03
11	mlp_out	eleutherai	65,536	CSQA	3,211	17.6%	12/100	freq.	5/0	1.5	0.00	−0.03	+12 / −13	0.655	0.00	+0.04	+0.05
11	resid_post	eleutherai	65,536	CSQA	3,211	17.6%	25/100	causal	10/5	1.0	+0.16	−0.19	+24 / −30	0.830	+0.04	+0.05	0.00

Layer	Site	SAE release	Width	Bench.	Holdout	Baseline	Survival	Rank	k_+/k_-	α	Selection	Gain	Flips	p	Other	OBQA	HS
google/gemma-3-270m (270M)																	
9	mlp_out	gemma scope	16,384	ARC	391	22.5%	11/100	freq.	5/0	1.5	+1.02	−0.51	+2 / −4	0.891	+0.01	−0.22	+0.17
9	resid_post	gemma scope	16,384	ARC	391	22.5%	1/27	freq.	5/5	1.5	+1.28	−0.51	+0 / −2	1.000	−0.04	+0.05	+0.06
15	mlp_out	gemma scope	16,384	ARC	391	22.5%	5/98	causal	3/0	0.5	+1.28	−0.51	+1 / −3	0.938	+0.05	−0.13	+0.14
15	resid_post	gemma scope	16,384	ARC	391	22.5%	9/100	freq.	5/0	1.5	+1.53	−0.77	+0 / −3	1.000	+0.09	−0.22	**+0.27**
9	mlp_out	gemma scope	16,384	CSQA	2,810	12.2%	21/100	freq.	5/0	1.0	+0.25	+0.04	+6 / −5	0.500	−0.26	−0.20	+0.06
9	resid_post	gemma scope	16,384	CSQA	2,810	12.2%	2/25	freq.	15/3	1.0	+0.25	**+0.36**	+12 / −2	0.006	−0.13	−0.11	+0.01
15	mlp_out	gemma scope	16,384	CSQA	2,810	12.2%	0/82	freq.	5/5	1.0	+0.25	+0.11	+5 / −2	0.227	−0.22	−0.13	+0.14
15	resid_post	gemma scope	16,384	CSQA	2,810	12.2%	6/100	freq.	3/0	1.5	+0.28	−0.07	+2 / −4	0.891	−0.09	−0.04	+0.18
EleutherAI/pythia-410m (410M)																	
11	mlp_out	eleutherai	65,536	ARC	819	26.1%	48/94	freq.	5/3	1.0	+0.49	+0.24	+2 / −0	0.250	−0.05	−0.04	+0.06
23	mlp_out	eleutherai	65,536	ARC	819	26.1%	21/100	freq.	5/3	1.5	+0.24	0.00	+1 / −1	0.750	−0.04	−0.11	−0.14
11	mlp_out	eleutherai	65,536	CSQA	4,625	21.0%	61/85	causal	15/0	0.5	+0.11	−0.02	+3 / −4	0.773	0.00	−0.05	+0.04
23	mlp_out	eleutherai	65,536	CSQA	4,625	21.0%	29/100	freq.	3/0	0.5	+0.04	+0.04	+2 / −0	0.250	−0.04	−0.04	+0.09
google/gemma-3-1b-pt (1B)																	
12	resid_post	chanind	32,768	ARC	294	24.1%	46/73	freq.	7/0	1.0	+2.03	−0.34	+4 / −5	0.746	−0.25	−0.26	+0.06
13	mlp_out	gemma scope	16,384	ARC	294	24.1%	65/100	freq.	7/5	1.5	+1.69	+1.02	+4 / −1	0.188	−0.13	−0.07	+0.06
24	resid_post	chanind	32,768	ARC	294	24.1%	75/100	freq.	10/0	1.5	+1.36	0.00	+1 / −1	0.750	−0.20	0.00	+0.02
12	resid_post	chanind	32,768	CSQA	1,932	14.0%	0/69	freq.	5/0	1.5	+0.52	−0.10	+14 / −16	0.708	−0.35	+0.04	−0.14
13	resid_post	gemma scope	16,384	CSQA	1,932	14.0%	0/29	freq.	10/0	1.5	+0.36	+0.36	+21 / −14	0.155	−0.04	−0.37	−0.05
22	resid_post	gemma scope	16,384	CSQA	1,932	14.0%	0/100	freq.	5/5	1.5	+0.36	−0.36	+10 / −17	0.939	+0.04	−0.04	+0.11
meta-llama/Llama-3.2-1B (1B)																	
7	resid_post	chanind	16,384	ARC	501	30.1%	1/43	freq.	15/0	1.5	+1.00	+0.20	+12 / −11	0.500	−0.87	+0.15	+0.13
8	resid_post	chanind	16,384	ARC	501	30.1%	0/54	freq.	3/0	0.5	+0.40	0.00	+3 / −3	0.656	+0.09	−0.27	+0.02
11	resid_post	seonglae	14,336	ARC	501	30.1%	21/96	freq.	10/3	0.5	+2.40	+1.60	+16 / −8	0.076	−2.96	**+1.85**	**+1.28**
7	resid_post	chanind	16,384	CSQA	2,487	28.3%	12/34	causal	10/0	1.5	+0.52	−0.16	+31 / −35	0.731	−0.39	+0.07	−0.03
8	resid_post	chanind	16,384	CSQA	2,487	28.3%	20/44	causal	3/3	1.5	+0.64	+0.40	+22 / −12	0.061	+0.22	+0.05	−0.09
11	resid_post	seonglae	14,336	CSQA	2,487	28.3%	31/97	causal	3/0	0.5	+0.16	+0.56	+100 / −86	0.170	+0.48	**+1.81**	−0.10
meta-llama/Llama-3.2-1B-Instruct (1B)																	
7	mlp_out	eleutherai	131,072	ARC	771	38.4%	17/100	causal	3/5	1.0	+2.46	+1.95	+49 / −34	0.062	−4.68	**+9.73**	**+3.10**
9	resid_post	qresearch	32,768	ARC	747	40.6%	9/32	causal	5/3	0.5	+1.47	**+1.74**	+18 / −5	0.005	**+1.87**	**+6.41**	**+0.56**
15	mlp_out	eleutherai	131,072	ARC	771	38.4%	22/100	freq.	5/0	1.5	+1.56	**+2.08**	+24 / −8	0.004	**+3.95**	**+6.43**	**+1.16**
7	mlp_out	eleutherai	131,072	CSQA	3,350	44.7%	50/87	causal	15/5	0.5	+2.27	**+3.70**	+233 / −109	8.8×10^{-12}	**+12.13**	**+8.94**	**+2.03**
9	resid_post	qresearch	32,768	CSQA	3,304	45.8%	4/29	causal	3/0	0.5	+0.27	+0.30	+47 / −37	0.163	+0.09	**+1.25**	−0.05
15	mlp_out	eleutherai	131,072	CSQA	3,304	45.8%	47/100	causal	10/3	1.0	+1.15	**+1.45**	+116 / −68	2.5×10^{-4}	**+7.51**	**+7.86**	**+1.84**

Layer	Site	SAE release	Width	Bench.	Holdout	Baseline	Survival	Rank	k_+/k_-	α	Selection	Gain	Flips	p	Other	OBQA	HS
deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B (1.5B)																	
13	mlp_out	eleutherai	65,536	ARC	633	23.2%	28/90	causal	10/5	1.0	+0.63	**+1.74**	+16 / −5	0.013	+0.10	−0.13	−0.33
27	mlp_out	eleutherai	65,536	ARC	633	23.2%	21/30	freq.	3/0	0.5	+0.16	+0.47	+3 / −0	0.125	−0.03	−0.04	−0.14
13	mlp_out	eleutherai	65,536	CSQA	2,436	12.1%	2/65	freq.	15/3	1.5	+0.62	+0.12	+81 / −78	0.437	+0.57	−0.29	−0.45
27	mlp_out	eleutherai	65,536	CSQA	2,436	12.1%	7/32	freq.	7/0	1.5	+0.33	+0.08	+7 / −5	0.387	−0.04	−0.09	−0.20
google/gemma-2-2b (2B)																	
1	resid_post	chanind topk	18,432	ARC	738	58.1%	40/44	causal	7/0	1.0	+0.68	+0.81	+10 / −4	0.090	**+0.27**	**+0.37**	+0.07
12	mlp_out	gemma scope	16,384	ARC	738	58.1%	76/100	causal	3/0	0.5	+0.54	−0.14	+3 / −4	0.773	**+0.19**	−0.13	−0.09
12	resid_post	gemma scope	16,384	ARC	738	58.1%	74/100	freq.	7/3	0.5	+0.54	+0.14	+4 / −3	0.500	**+0.17**	−0.18	−0.10
12	resid_post	snap loss	32,768	ARC	738	58.1%	78/100	freq.	7/3	1.5	+0.54	+0.54	+9 / −5	0.212	**+0.44**	**+0.71**	+0.13
12	resid_post	standard	32,768	ARC	738	58.1%	59/100	causal	7/3	0.5	+0.54	+0.54	+5 / −1	0.109	**+0.22**	+0.11	+0.08
19	mlp_out	gemma scope	16,384	ARC	738	58.1%	77/100	causal	3/5	1.5	+0.68	−0.27	+3 / −5	0.855	**+0.90**	−0.16	+0.19
19	resid_post	ale21m9	16,384	ARC	738	58.1%	64/100	freq.	7/3	1.5	+0.41	+0.41	+7 / −4	0.274	**+0.37**	**+0.48**	**+0.37**
19	resid_post	gemma scope	16,384	ARC	738	58.1%	51/100	causal	3/0	1.0	+0.41	0.00	+2 / −2	0.688	+0.09	−0.11	+0.02
19	resid_post	seonglae	18,432	ARC	738	58.1%	79/100	freq.	15/3	1.0	+0.41	+0.27	+7 / −5	0.387	**+0.47**	**+0.73**	**+0.27**
24	resid_post	snap loss	32,768	ARC	738	58.1%	92/100	freq.	3/0	1.0	+0.27	0.00	+3 / −3	0.656	+0.04	+0.15	−0.05
24	resid_post	standard	32,768	ARC	738	58.1%	94/100	freq.	5/3	1.5	+0.27	−0.41	+1 / −4	0.969	+0.05	+0.05	+0.04
1	resid_post	chanind topk	18,432	CSQA	3,774	47.4%	10/34	causal	3/3	1.5	+0.40	**+0.72**	+36 / −9	3.3×10^{-5}	**+2.10**	**+1.32**	+0.11
12	mlp_out	gemma scope	16,384	CSQA	3,774	47.4%	24/100	causal	10/0	1.5	+0.32	+0.21	+18 / −10	0.092	**+0.57**	**+0.62**	+0.09
12	resid_post	gemma scope	16,384	CSQA	3,774	47.4%	33/100	causal	15/0	1.5	+0.45	**+0.34**	+25 / −12	0.024	**+1.05**	**+0.95**	+0.16
12	resid_post	snap loss	32,768	CSQA	3,774	47.4%	34/100	causal	15/5	1.5	+0.66	**+0.64**	+35 / −11	2.7×10^{-4}	**+1.96**	**+1.59**	−0.07
12	resid_post	standard	32,768	CSQA	3,774	47.4%	45/100	causal	10/3	1.5	+0.37	**+0.37**	+23 / −9	0.010	**+1.31**	**+1.08**	+0.05
19	mlp_out	gemma scope	16,384	CSQA	3,774	47.4%	32/100	causal	10/5	1.5	+0.34	+0.21	+15 / −7	0.067	**+0.96**	**+1.12**	+0.08
19	resid_post	ale21m9	16,384	CSQA	3,774	47.4%	67/100	freq.	15/0	1.5	+0.34	+0.26	+24 / −14	0.072	**+1.48**	**+1.65**	**+0.40**
19	resid_post	gemma scope	16,384	CSQA	3,774	47.4%	71/100	causal	3/3	1.5	+0.64	**+0.48**	+29 / −11	0.003	**+1.53**	**+1.45**	**+0.36**
19	resid_post	seonglae	18,432	CSQA	3,774	47.4%	64/99	causal	15/5	1.5	+0.69	**+0.48**	+28 / −10	0.003	**+1.70**	**+1.47**	**+0.59**
24	resid_post	snap loss	32,768	CSQA	3,774	47.4%	91/100	causal	15/3	1.5	+0.40	+0.26	+22 / −12	0.061	**+1.13**	**+0.70**	+0.21
24	resid_post	standard	32,768	CSQA	3,774	47.4%	91/100	causal	15/0	1.5	+0.34	**+0.26**	+16 / −6	0.026	**+0.65**	**+0.92**	**+0.19**
google/gemma-2b (2B)																	
0	resid_post	jbloom anthropic	16,384	ARC	442	25.1%	10/23	freq.	10/3	1.5	+0.68	−0.23	+5 / −6	0.726	**+0.83**	**+0.44**	+0.17
6	resid_post	jbloom fast lr	16,384	ARC	442	25.1%	43/100	freq.	15/5	1.5	+0.68	−0.23	+3 / −4	0.773	**+0.74**	**+0.51**	−0.22
10	resid_post	jbloom plain	16,384	ARC	442	25.1%	83/100	freq.	10/5	1.0	+3.16	−1.36	+17 / −23	0.866	**+0.55**	+0.44	−0.67
17	resid_post	jbloom plain	16,384	ARC	442	25.1%	48/100	freq.	3/5	0.5	+2.71	−0.68	+18 / −21	0.739	**+1.40**	**+0.75**	**+0.57**
0	resid_post	jbloom anthropic	16,384	CSQA	2,366	12.3%	13/15	freq.	10/5	1.5	+1.06	+0.08	+29 / −27	0.447	−0.22	+0.02	−0.08
6	resid_post	jbloom fast lr	16,384	CSQA	2,366	12.3%	79/89	causal	7/3	0.5	+0.55	+0.08	+21 / −19	0.437	−0.04	+0.09	−0.11

Layer	Site	SAE release	Width	Bench.	Holdout	Baseline	Survival	Rank	k_+/k_-	α	Selection	Gain	Flips	p	Other	OBQA	HS
10	`resid_post`	jbloom plain	16,384	CSQA	2,366	12.3%	75/100	causal	5/3	1.5	+1.18	+0.34	$+147/-139$	0.340	+1.05	+0.38	−0.13
17	`resid_post`	jbloom plain	16,384	CSQA	2,366	12.3%	81/100	causal	3/0	1.5	+6.59	**+8.83**	$+476/-267$	7.9×10^{-15}	−0.26	−0.97	−0.88
`google/gemma-2b-it` (2B)																	
12	`resid_post`	jbloom	16,384	ARC	1,118	45.7%	29/100	causal	7/0	1.5	+0.27	+0.45	$+20/-15$	0.250	−2.78	+0.02	+0.26
12	`resid_post`	jbloom	16,384	CSQA	5,370	46.5%	49/100	causal	7/3	0.5	0.00	+0.02	$+134/-133$	0.500	−0.35	−0.53	−0.24
`meta-llama/Llama-3.1-8B-Instruct` (8B)																	
15	`resid_post`	andyrdt	131,072	ARC	1,076	84.1%	79/100	freq.	5/5	0.5	−0.56	−0.74	$+3/-11$	0.994	+0.09	+0.42	−1.37
15	`resid_post`	andyrdt	131,072	CSQA	5,123	77.2%	97/100	freq.	3/0	0.5	−0.25	−0.61	$+9/-40$	1.0000	+0.04	−0.57	−0.58
27	`resid_post`	andyrdt	131,072	CSQA	5,123	77.2%	94/100	causal	15/0	0.5	0.00	−0.08	$+4/-8$	0.927	0.00	+0.05	+0.14
`meta-llama/Meta-Llama-3-8B-Instruct` (8B)																	
15	`resid_post`	eleutherai	131,072	ARC	1,109	83.0%	42/100	freq.	3/0	0.5	+0.36	+0.27	$+4/-1$	0.188	−0.05	−0.02	+0.04
30	`resid_post`	eleutherai	131,072	ARC	1,109	83.0%	60/100	causal	10/0	0.5	+0.45	+0.18	$+3/-1$	0.313	−0.02	−0.15	+0.07
15	`resid_post`	eleutherai	131,072	CSQA	5,272	75.7%	52/100	freq.	3/0	0.5	−0.04	−0.08	$+11/-15$	0.837	+0.31	−0.13	**+0.58**
30	`resid_post`	eleutherai	131,072	CSQA	5,272	75.7%	64/100	freq.	10/3	1.5	+0.25	+0.02	$+17/-16$	0.500	+0.04	−0.05	0.00